

Computing IV Sec 201: Project Portfolio

Nathan Khoury

Spring 2024

Contents

1	Before You Read	2
2	PS0: Hello SFML	3
3	PS1: PhotoMagic	8
4	PS2: Pythagorean Tree	19
5	PS3: Sokoban	29
6	PS4: N-Body Simulation	51
7	PS5: DNA Sequence Alignment	69
8	PS6: RandWriter	80
9	PS7: Kronos Log Parsing	93

Time to Complete Portfolio: approx. 40 hours

1 Before You Read

This section exists to help whoever is reading this understand the formatting within each section. I will present a list below which will list the section names below in the order that you will encounter them while reading, and I'll also briefly explain what the section is supposed to detail. I would also like to mention here to friends, family, TAs, employers, or anyone else who may be reading (or, more likely, skimming) this the projects I completed in my Computing IV class were challenging and I learned a lot from them. That being said, they *are* in chronological order, so it would be unfair to judge the quality of PS0 in section 2 or PS2 in section 4 compared against projects like PS6 in section 8 or PS7 in section 9. I have grown a lot this semester and intend to continue doing so, and I hope that as a reader that will come across to you genuinely.

Functionality will generally tackle the problem at hand, or the objective that the program will achieve. It may handle broad ideas, or math/science concepts that are specific, but are not in terms of the program itself. This section is generally to give you an idea of what's in store, and to (hopefully) provide the necessary context to understand the goal.

Features handles the actual implementation of the program. It is very wordy and references variable names, class names, etc. It is *generally* structured in such a way that it will bring you along with the flow of the program, starting in `main` and progressing through the control statements and all relevant functions, explaining what is necessary as it goes. Every project is a little different, so this section is more flexible in that regard.

Personal Reflection is a broader subsection with three additional subsubsections:

The Struggles is where I will detail issues during implementation. This includes bugs, poor design choices, inefficient implementation, etc. It tries to cover all of the bases of what went wrong all around. No program of mine is perfect, this section always has something to say.

What I Learned will bounce off of the previous section, expressing maybe alternate solutions to poor implementation, new ideas that I've since learned about, things I learned during the process which helped circumvent/avoid issues, etc.

The Takeaway is more of a personal, broader outlook on the program in my opinion. I might mention if I liked the assignment, how it impacted me and my growth as a programmer, and really any other remarks that I may choose to throw in there.

Running The Program displays proper command-line formatting in order to run the program, and explains what arguments are required and how to best supply them as a user. Even if you are not going to run the program, it may help clarify what control the user may have with a certain program.

Proof of Functionality typically will provide some amount of images to show what the program looks like when it is executed, maybe what ways the user can impact the output of a given program, etc. It is also used to prove to some extent that I really did make and execute the program.

Source Code will first list the files that comprise the final program, and then will display the contents of each file individually so that anyone can see how I implemented the program.

Unit Testing is where I may express any issues that occurred when trying to write my unit tests for the program, perhaps what some tests aim to accomplish, or if unit testing was even applicable to the project being discussed at all. After my remarks the code within the test file itself will be displayed for viewing

Bonus will only show up if I fulfilled some bonus opportunity for the project, hence why it is near the end. I will explain what the bonus opportunity was and how I implemented it into my version, as well as maybe any issues/challenges that resulted.

I will also mention at the end here that this portfolio was produced in L^AT_EX using MikTeX and TeXworks.

2 PS0: Hello SFML

2.1 Functionality

The Hello SFML assignment was a simple tutorial assignment which was intended to allow everyone to get their new tools set up for the semester without the pressure of a strenuous assignment on top of that. The goal of this tutorial assignment was to get one's IDE, access to Linux, Lint, etc. fully operational, and to be able to run the SFML "Did I install everything correctly?" code without issue. By extension, it was also an expectation that we could all add some feature to express not only that SFML, the GNU Compiler, and all the rest of our tools were functional, but that we also could access and interpret resources such as the SFML documentation which proved a pivotal resource throughout this semester.

My implementation of this assignment featured an image of a funny cat that I found of the internet named *Maxwell*. The user is able to make Maxwell move in four separate directions relative to the screen by using the **W**, **A**, **S**, **D** keys to move Maxwell **Up**, **Left**, **Down**, and **Right** respectively. I decided that my extra feature of choice would be the ability to let maxwell do a backflip. In the presented version of the program, the user is able to hit the **space bar** key which will cause Maxwell to enter a backflipping state which he will automatically exit once he notices he has completed the flip.



Figure 1: Maxwell the Cat, provided as "sprite.png" in the program

2.2 Features

Since this was just an introductory sort of assignment, there is not really too much going on under the hood. Just to reiterate, the program's simplicity is expressed through rudimentary **W**, **A**, **S**, **D** controls which move Maxwell at linear velocity in the direction of choice. The backflipping mechanism essentially boils down to the following: If the user hits spacebar, or if Maxwell is already mid-backflip during this frame, check if the newly calculated distance to continue the backflip to this frame would exceed a 360° rotation. If so, Maxwell should snap back to his original orientation since he should have completed the flip by this point. Otherwise, Maxwell should begin/continue a backflip by updating his rotation by the amount of his linear flipping speed of -180 (negative so that it is a counterclockwise rotation, displaying a *backflip*) multiplied by the seconds that have elapsed since the last frame. The elapsed seconds are tracked by using the `sf::Clock` and `sf::Time` objects from the SFML library, and resetting the count every frame.

I decided to add, for a little bit of extra content for the program and experience navigating the SFML Documentation, a text block in the top right of the screen which provides instruction on the controls, as well as an updating count of the amount of backflips that Maxwell has completed. It's not much but it helped to gain a bit more understanding early on of the nature of SFML, which proved to be vital in later assignments. Beyond these features there were no notable algorithms, data structures, or object-oriented designs applied in this assignment.

2.3 Personal Reflection

2.3.1 The Struggles

In parallel with the scare features, the struggles were also fairly scarce in this assignment. That being said, they were not nonexistent. I struggled very much getting SFML to work, primarily because my original setup could not get Windows Subsystem for Linux *WSL* working on my original laptop. Then the virtual machine started causing problems and took multiple installs to get it working (I swear I'm not the problem here, you have to believe me), and when it finally *did* start working, it hardly was since my laptop had only `textbf8` gb of RAM, which meant that the virtual machine was only getting `4`. This is all without mentioning when it was working, something regarding my laptop's power coincidentally broke such that the laptop would be powered on; Yet, the screen would not turn on. I ended up having to get a whole new laptop, which I was actually able to get WSL operating on which made everything incredibly more straightforward. I was able to use WSL for the rest of my assignments in this course, making the challenges in this course much more digestible given that I wouldn't have to fight with the virtual machine for every single thing.

Separately, I also had issues with my stopping condition for Maxwell's backflip. Many of my early attempts at trying to get him to stop backflipping were futile since I wasn't accounting for the fact that the value of SFML `sf::Sprite` rotation remains strictly within the range of 0° - 360° , meaning that simply checking if the next frame of rotation went past 0° or 360° was not a viable option. I ended up having to have to do intermediate math that checks if the rotation will go past 360° *before* applying it. If so, just set the sprite back to a rotation of 0° and end the backflipping process. Overall, not a hard challenge at all, but considering how little there was to this assignment, it was worth mentioning. Below is my full, final implementation of the backflipping process:

```
1 if (sf::Keyboard::isKeyPressed(sf::Keyboard::Space) || sprite.getRotation() != 0) {
2     tempRotation = (-180 * static_cast<float>(time.asSeconds()));
3     if (sprite.getRotation() != 0 && sprite.getRotation() - (-1 * tempRotation) <= 0) {
4         sprite.setRotation(0); // if new rotation will pass 360, set to 0
5         numFlips++;
6         // update the flip counter
7         flipCounter.setString(flipString + std::to_string(numFlips));
8     } else {
9         sprite.rotate(tempRotation);
10    }
11 }
```

Aside from the fact that this shows how poor my skills were at the start of this semester (ouch), and conversely how much I improved by the end, the nested *if else* statement shows the final implementation of the relatively tricky stopping condition. It first checks that the rotation is not already 0 (retrospectively, this is redundant since the rotation must be nonzero to even enter the outer loop, let alone the nested one) and also verifies that the sprite's current rotation, minus the intermediately calculated `tempRotation`, would end up completing a full rotation (dipping below 0°). Subtraction is used since the sprite is rotating backwards with respect to SFML. Positive rotation in SFML is clockwise whereas in different contexts, such as the unit circle, positive rotation would be counterclockwise. This means after the initial first rotation backwards, the sprite goes from 0° to 360° and counts down from 360° . A good brain-teaser to kick off the course; However, all things considered: not too bad.

2.3.2 What I Learned

I gained a lot of early surface-level understanding of SFML, especially what it can, and *can't*, do. Learning how Sprites, Textures, and transformative actions all apply together in the context of a simple SFML program were invaluable. Even a little usage of the SFML clock proved to be handy later on.

I got to apply a fair chunk of basic syntax that I learned when we first learned C++ in Computing III in a small, independent project. This not only dusted off the knowledge I had from the previous semester, but it reinforced some more basic concepts that are integral

to the C++ programming process.

It is also important to mention that I learned the value and methodology of creating make-files, as well as how to use the GNU Compiler at all. The absence of this knowledge would have made the rest of this course impossible, so it was great to at least get some idea of how it all works early on.

2.3.3 The Takeaway

Overall, this program was short, sweet, and handy for getting back into the swing of programming. Where I had previously had very little experience making C++ programs at all, let alone independently, this proved to be a very helpful reminder of some of the important concepts that go behind starting up a new program.

2.4 Running The Program

After compiling with *make*, command line input to run the Hello SFML program follows the following formula:

1 `./sfml-app`

This program utilized no command line arguments, so it is only required that the user run the executable produced which is named **sfml-app**

2.5 Proof of Functionality

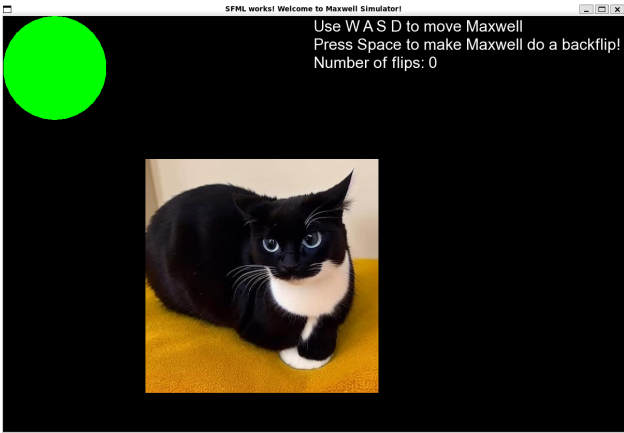
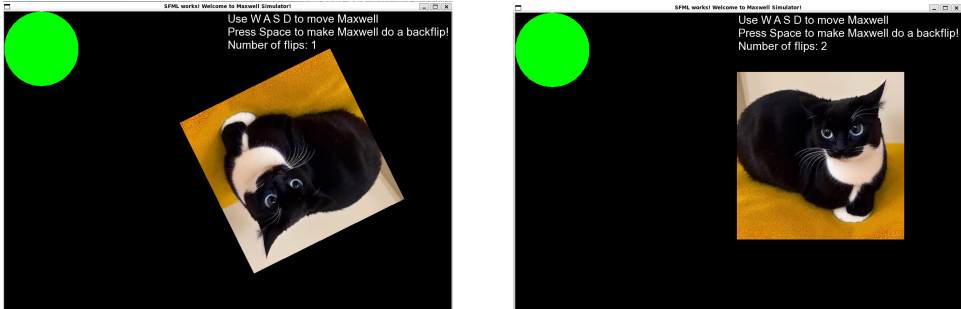


Figure 2: Screenshot of the program’s appearance after startup



(a) Maxwell in the middle of his second backflip

(b) Maxwell after his second backflip (with the updated count in the upper right)

2.6 Source Code

The following presents the source code for this program. For this program there were no supporting .hpp or .cpp files, it was simply just the main driver in **main.cpp** and the make-file routine in **Makefile**.

Makefile

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -lboost_unit_test_framework
4 # Your .hpp files
5 DEPS =
6 # Your compiled .o files
7 OBJECTS =
8 # The name of your program
9 PROGRAM = sfml-app
10
11 .PHONY: all clean lint
12
13
14 all: $(PROGRAM)
15
16 # Wildcard recipe to make .o files from corresponding .cpp file
17 %.o: %.cpp $(DEPS)
18     $(CC) $(CFLAGS) -c $<
19
20 $(PROGRAM): main.o $(OBJECTS)
21     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
22
23 clean:
24     rm *.o $(PROGRAM)
25
26 lint:
27     cpplint *.cpp *.hpp
```

main.cpp

```
1 // Copyright 2024 Nathan Khoury
2 #include <iostream>
3 #include <string>
4 #include <SFML/Graphics.hpp>
5
6
7 int main() {
8     sf::RenderWindow window(sf::VideoMode(1200, 800), "SFML works! Welcome to Maxwell Simulator!");
9     sf::CircleShape shape(100.f);
10    shape.setFillColor(sf::Color::Green);
11
12    // initialize clock
13    sf::Clock clock;
14    sf::Time time;
15
16    // float used for preventing infinite spinning
17    float tempRotation;
18
19    // adding texture
20    sf::Texture spriteTexture;
21    if (!(spriteTexture.loadFromFile("./sprite.png"))) {
22        std::cout << "Couldn't load spriteTexture" << std::endl;
23    }
24
25    // create sprite and wrap it with our texture
26    sf::Sprite sprite;
27    sprite.setTexture(spriteTexture); // add texture to sprite
28    sprite.setOrigin(225.f, 225.f); // the sprite.jpeg is 225px x 225px, set origin to center
29    sprite.setPosition(500.f, 500.f); // move sprite (as to not cover circle)
30
31    // access Arial font for text display, then apply to text
32    sf::Font font;
33    font.loadFromFile("arial.ttf");
34    sf::Text instructionText
35        ("Use W A S D to move Maxwell\nPress Space to make Maxwell do a backflip!", font);
36    instructionText.setPosition(600, 0);
```

```

37
38 // create unsigned int variable for storing number of backflips, and new corresponding text
39 unsigned int numFlips = 0;
40 const std::string flipString = "Number of flips: ";
41 sf::Text flipCounter((flipString + std::to_string(numFlips)), font);
42 flipCounter.setPosition(600, 70);
43
44 // limit framerate for consistency
45 window.setFramerateLimit(60);
46
47 while (window.isOpen()) {
48     sf::Event event;
49     time = clock.getElapsedTime(); // get time between frames
50     clock.restart(); // restart clock for this frame
51     while (window.pollEvent(event)) {
52         if (event.type == sf::Event::Closed)
53             window.close();
54     }
55     // Check for W A S D keypresses for movement
56     if (sf::Keyboard::isKeyPressed(sf::Keyboard::S))
57         sprite.move(0.f, 100 * time.asSeconds());
58
59     if (sf::Keyboard::isKeyPressed(sf::Keyboard::W))
60         sprite.move(0.f, -100 * time.asSeconds());
61
62     if (sf::Keyboard::isKeyPressed(sf::Keyboard::A))
63         sprite.move(-100 * time.asSeconds(), 0.f);
64
65     if (sf::Keyboard::isKeyPressed(sf::Keyboard::D))
66         sprite.move(100 * time.asSeconds(), 0.f);
67
68     if (sf::Keyboard::isKeyPressed(sf::Keyboard::Space) || sprite.getRotation() != 0) {
69         tempRotation = (-180 * static_cast<float>(time.asSeconds()));
70         if (sprite.getRotation() != 0 && sprite.getRotation() - (-1 * tempRotation) <= 0) {
71             sprite.setRotation(0); // if new rotation will pass 360, set to 0
72             numFlips++;
73             // update the flip counter
74             flipCounter.setString(flipString + std::to_string(numFlips));
75         } else {
76             sprite.rotate(tempRotation);
77         }
78     }
79
80     window.clear();
81     window.draw(shape);
82     window.draw(sprite); // draw our new sprite
83     window.draw(instructionText); // draw instruction text
84     window.draw(flipCounter); // draw flip counter
85     window.display();
86 }
87
88 return 0;
89 }

```

2.7 Unit Testing

Unit Testing was not necessary for this assignment.

3 PS1: PhotoMagic

3.1 Functionality

This first serious project of the semester is called **PhotoMagic**, and was split into two notable sections. The first portion of this project required students to implement a Fibonacci Linear Feedback Shift Register **FibLFSR** in the form of a class appropriately named *FibLFSR*. The latter half of the project prompted students to apply the FibLFSR that they developed earlier as a means of taking an input image, and outputting what would appear to be non-sensical, random pixels in its place. The output image is actually an encrypted version of the first image, for which the FibLFSR was utilized to provide the encryption technique. Decryption is, likewise, just as replicable since the process is not random at all. The FibLFSR class is contained within a namespace called *PhotoMagic*, and the encryption/decryption portion of the program is added within the namespace, but not within the class.

The user of the program supplies not only the filenames of the image to be encrypted/decrypted and the output image, but also a 16-bit string of binary numbers to use as a seed for initializing the FibLFSR. Since the algorithm within the FibLFSR is consistent this means that running the same seed on the same original image multiple times will yield the same encrypted output image. It's also important to note that, as will be discussed in 3.2, image reversal is possible since the methodology used within a FibLFSR yields identical results when the same seed is used.

3.2 Features

My implementation of the Photomagic program begins by asserting sufficient command-line arguments, and then promptly initializing an object of type `PhotoMagic::FibLFSR`. The FibLFSR class features only a value constructor which accepts a string assumed to be of the appropriate 16-bit binary digit format. My implementation simply stores the string as a private member variable `std::string _register`. Section 3.3.2 discusses hypothetical improvements that I did not implement, but have since learned to be more efficient than working with a string object, although more technical in terms of the proposed implementation.

With the FibLFSR initialized, the rest of the program is able to proceed after performing a call to `PhotoMagic::transform()` which accepts a reference to the input image as SFML's image data type `sf::Image`, as well as a pointer to the FibLFSR itself. The `transform()` function governs the whole process of image encryption/decryption; Although, notably, it does not need to know which it is completing. This is because the way that the encryption/decryption is completed within `transform()` is by iterating over every pixel within the image object, and using the **XOR** ($\wedge$) operation to XOR the red, green, and blue color values of each pixel, with three separately generated numbers from the FibLFSR. As an aside, the FibLFSR generates numbers via the following process.

Within the 16-bit string seed held inside the FibLFSR, it should be understood that the farthest left bit is known to be the *most significant bit* **MSB**, and corresponds to position 15 in the 16-bit string (on an indexed range of 0-15). Conversely, the farthest right bit is known to be the *least significant bit* **LSB** and corresponds to position 0 in the 16-bit string. With this in mind, the FibLFSR operates by taking the MSB and performing an XOR operation on locations known as *taps*. For this implementation of a FibLFSR, the taps are at positions **10**, **12**, and **13**. The MSB at position 15 goes down the line, first undergoing a XOR operation with the tap at position 13, the result of which proceeds to be XOR'd with the next tap at 12. The result of that operation is finally XOR'd with the tap at position 10, yielding the final resulting bit. All the bits shift left one position to replace the empty slot left by the removed MSB (with the former resident of position 14 becoming the *new* MSB). The resulting bit from the XOR operations is added in position 0 as the new LSB.

The above-mentioned process, in the context of the PhotoMagic program, is found within the `PhotoMagic::FibLFSR::step()` public member function, and the newly generated LSB is returned as an integer value of either 1 or 0. The application of this lies within the

`PhotoMagic::FibLFSR::generate()` public member function, which takes a `int k` as its sole parameter. The `generate()` function simulates k iteration of the `step()` function. Starting with a running total of 0, the value of the total doubled, and the newly generated bit is added to the total. This process seamlessly extracts the right-most k bits from the register and represents them as an integer. The implementation for the `generate()` function is found displayed below.

```
1 int PhotoMagic::FibLFSR::generate(int k) {
2     int total = 0;
3     for (int i = 0; i < k; i++) {
4         total *= 2; // simulate left shift <<
5         total += this->step();
6     }
7     return total;
8 }
```

By passing the value 8 to the `generate()` function, an integer value within the range 0-255 can be obtained, which can then be XOR'd with one of the singular color values, also within the range of 0-255, for the current pixel being encrypted/decrypted. To loop back around to the central point of this all: the `transform()` function iterates over every pixel within the input image, calling `generate()` for every red, green, and blue color value for all the pixels. When this process completes, the image is saved under the output filename specified in the command line, and two SFML windows are opened: The left window displays the original image, while the right window displays the encrypted/decrypted image. Note: Due to the nature of the XOR operation, and the fact that providing the same seed to the FibLFSR yields the same sequence of generated bits, this means that an image that has been encrypted with a certain seed can also be decrypted with that exact same seed.

3.3 Personal Reflection

3.3.1 The Struggles

I found this first project to be a daunting task initially. I felt that, personally, I was in over my head and this is where my Computer Science path would end. I'm not sure if it was just the intimidation of such a challenging concept (the functionality of a FibLFSR), my *very* imperfect knowledge of C++ at the time, my lack of confidence with bitwise operations, or some horribly effective combination of them all, but I felt very stressed to even start this assignment up. I had real genuine fears of where my future would lie just based off of my impressions of reading the requirements for this assignment. I know this is an awfully personal divulgence of myself; Although, I feel that it what is most important to highlight in this first assignment. Sometimes getting started, or summoning confidence, can be the most difficult part. Once I finally worked up the confidence to begin sinking time into the project, the aspects of the project that seems impossibly complex regressed to something within the realm of possibility. That is not to say that I became a wizard on the keyboard and wrote code that would blow generations away; It was still very challenging and time consuming. The point is that I struggled with letting the fear of the *idea* of the project govern my confidence to tackle the project at all. This, unfortunately, was an issue I dealt with the whole semester, but gradually became more and more reasonable the more I learned and, likewise, the more confidence I gained.

With respect to the code itself: I felt that, even at the time, my storage and usage of a string of binary digits rather than a singular integer itself was notably inefficient. That being said, I simply couldn't understand at the time how to apply the bitwise operators to extract the bits I needed from a singular integer representation of the binary seed. I am glad to say that I have grown and learned *a lot* since then and in section 3.3.2 I mention the alternate solution. Aside from this, I found the remaining aspects of implementing the FibLFSR, as well as the actual PhotoMagic encryption/decryption functionality itself, to be *relatively* straightforward.

The only other aspect of the project that I struggled with significantly was the unit testing. It was my first time ever using the Boost library, so that was a challenge in its own right, but figuring out what to test and how to effectively do so was difficult in a handful of ways. I

describe more of this in the Unit Testing section 3.7 at the end of section 3

3.3.2 What I Learned

I learned a significant amount about not only the importance of certain bitwise operations, but also the application of them in a real project. I also became familiar with the importance of the XOR operator $\wedge$, as well as some of the nuances of it. I learned that you can swap two numbers by only using the XOR operator three times, requiring no additional memory. I also learned about the reversibility-aspect of the XOR operator which was imperative for this project. By this I am referring to the fact that if you XOR a number by the same separate number twice, it will return to the original starting number. This is the basis of the decryption aspect of the PhotoMagic program, so knowledge of this was essential.

Probably the most valuable skill I learned from this assignment was how to accept and process command-line arguments. Prior to this assignment, I had seen `main()` have parameters such as:

```
1 int main(int argc, char* argv[]) {
2     /* some program */
3 }
```

Although I had never understood the purpose of those parameters, since many of my projects before this just had those omitted due to them not being utilized. I am glad to say I truly do understand now the purpose of the parameters, as well as the importance of command-line arguments as a whole. This became a key player in most of the future projects, so I'm glad I was able to figure it all out during this first real project.

I also learned of the more efficient approach to the program that I was seeking before and during development. By utilizing the ($\gg$) right shift operator you can move desired bits for extraction to the end, at which point you can mod (%) by 2 to obtain the LSB bit, which is either a 1 or 0. I drafted up a short example which extracts the specified bits and outputs them to the `std::cout` stream to assert this understanding, which is displayed below:

```
1 #include <iostream>
2
3 int main() {
4     // Example 1
5     int x = 44;    // 101100
6                   //   ^ extract bit at position 3
7     int bit = (x >> 3) % 2;    // 1
8     std::cout << bit << std::endl;
9
10    // Example 2
11    int y = 315;   // 100111011
12                  //   ^ extract bit at position 5
13    bit = (y >> 5) % 2;    // 1
14    std::cout << bit << std::endl;
15
16    // Example 3
17    int z = 239;   // 11101111
18                  //   ^ extract bit at position 4
19    bit = (z >> 4) % 2;    // 0
20    std::cout << bit << std::endl;
21
22    return 0;
23 }
```

3.3.3 The Takeaway

Overall this project was, admittedly, very intimidating, especially at first. I found that, despite that, it proved to be a very insightful and educational project, and inspired some drive in me to keep pushing through despite the confidence I was lacking. I can say now, looking back on all my projects, that it was certainly worth it and I have come so far from where I was just a few months ago. I found that this assignment helped solidify understanding

of the fundamental functions of some of the bitwise operators, as well as concept of linear feedback registers and their significance. I also gained considerably more understanding of the anatomy of a makefile, the process of accepting command-line arguments, and at least some understanding of how SFML works.

3.4 Running The Program

The program can be compiled using *make* and the Makefile provided. An executable named **PhotoMagic** is produced, as well as a static library file **PhotoMagic.a**. The program can be executed using a command of the format:

```
1 ./PhotoMagic <input-filename-with-extention> <output-filename-with-extention> <seed>
```

Where the first parameter is the name of the input file, including its respective file extention (i.e. .png, .jpeg, .bmp, etc.)

The second parameter is the name of the output file to be created/overwritten, as well as the extention for the desired format to write to (see first parameter for examples).

And lastly a third parameter which is an exactly 16 character string of binary digits to be supplied as the seed of the FibLFSR

3.5 Proof of Functionality

Below are two figures showing how the output windows look when an image is encrypted, but also when an image is decrypted. The same photo is run through the program twice in the figures, first encrypting it, then taking the output of the first program and accepting it as input the second time, utilizing the same seed for both operations.

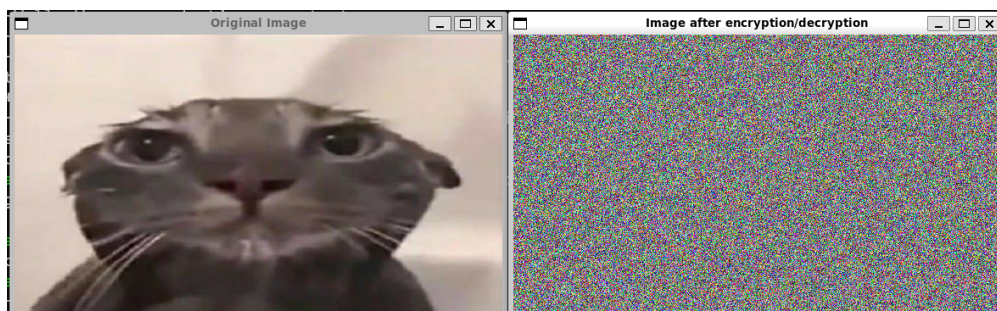


Figure 3: Image of a cat with water being poured on his head (left) gets converted into an encrypted image (right) using the PhotoMagic program

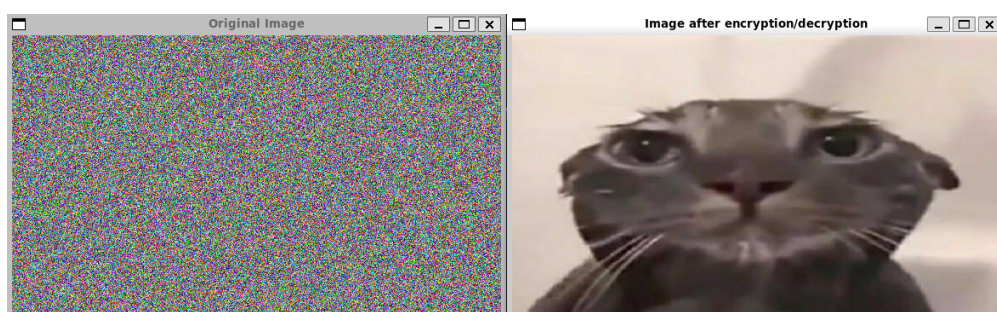


Figure 4: Encrypted image displaying pixels of random color values (left) is revealed to be a silly photo of a cat getting water poured on his head (right) after decryption

3.6 Source Code

The following pages present the source code files for the project. The FibLFSR is within its corresponding `FibLFSR.hpp` and `FibLFSR.cpp` files. `PhotoMagic.hpp` and `PhotoMagic.cpp` contains the encryption/decryption extention to the FibLFSR, like the `PhotoMagic::transform()` function. A Makefile is supplied too, and the main driver is within `main.cpp`

Makefile

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -lboost_unit_test_framework
4 # Your .hpp files
5 DEPS = FibLFSR.hpp PhotoMagic.hpp
6 # Your compiled .o files
7 OBJECTS = FibLFSR.o PhotoMagic.o
8 # The name of your programs
9 PROGRAM = PhotoMagic test
10
11 .PHONY: all clean lint
12
13 all: test PhotoMagic PhotoMagic.a
14
15 # Wildcard recipe to make .o files from corresponding .cpp file
16 %.o: %.cpp $(DEPS)
17     $(CC) $(CFLAGS) -c $<
18
19 PhotoMagic: main.o $(OBJECTS)
20     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
21
22 PhotoMagic.a: $(OBJECTS)
23     ar rcs $@ $^
24
25 test: test.o $(OBJECTS)
26     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
27
28 clean:
29     rm *.o PhotoMagic test PhotoMagic.a output-file.png
30
31 lint:
32     cpplint *.cpp *.hpp
```

main.cpp

```
1 // Copyright 2024 Nathan Khoury
2 #include <iostream>
3 #include <string>
4
5 #include "FibLFSR.hpp"
6 #include "PhotoMagic.hpp"
7
8 int main(int argc, char* argv[]) {
9     // accept exactly 3 inputs (4 technically)
10    if (argc != 4) {
11        std::cout << "error, bad input" << std::endl;
12        return -1;
13    }
14    std::string input = argv[1];
15    std::string output = argv[2];
16    std::string seed = argv[3];
17    // load input.txt to image
18    sf::Image img;
19    if (!img.loadFromFile(input)) {
20        return -1;
21    }
22
23    // create LFSR
24    PhotoMagic::FibLFSR myLFSR(seed);
25
26    // encrypt image in transform()
27    PhotoMagic::transform(img, &myLFSR);
28
29    // save image to output-file.png
30    if (!img.saveToFile(output)) {
31        std::cout <<
```

```

32         "error, couldn't save encrypted image to: "
33         << output << std::endl;
34         return -1;
35     }
36
37     // get size of images
38     sf::Vector2u size = img.getSize();
39
40     // load as textures and sprites
41     sf::RenderWindow windowOriginal
42         (sf::VideoMode(size.x, size.y), "Original Image");
43     sf::RenderWindow windowEncrypted
44         (sf::VideoMode(size.x, size.y), "Image after encryption/decryption");
45
46     sf::Texture textureOriginal;
47     sf::Texture textureEncrypted;
48
49     textureOriginal.loadFromFile(input);
50     textureEncrypted.loadFromImage(img);
51
52     sf::Sprite spriteOriginal;
53     sf::Sprite spriteEncrypted;
54     spriteOriginal.setTexture(textureOriginal);
55     spriteEncrypted.setTexture(textureEncrypted);
56
57     // set window positions
58     windowOriginal.setPosition({200, static_cast<int>(size.y)});
59     windowEncrypted.setPosition({210 + static_cast<int>(size.x), static_cast<int>(size.y)});
60
61     // display both images
62     while (windowOriginal.isOpen() && windowEncrypted.isOpen()) {
63         sf::Event event;
64         while (windowOriginal.pollEvent(event)) {
65             if (event.type == sf::Event::Closed)
66                 windowOriginal.close();
67         }
68         while (windowEncrypted.pollEvent(event)) {
69             if (event.type == sf::Event::Closed)
70                 windowEncrypted.close();
71         }
72         windowOriginal.clear();
73         windowOriginal.draw(spriteOriginal);
74         windowOriginal.display();
75         windowEncrypted.clear();
76         windowEncrypted.draw(spriteEncrypted);
77         windowEncrypted.display();
78     }
79
80     return 0;
81 }

```

FibLFSR.hpp

```

1  // Copyright 2024 Dr. Daly
2  // Sourced from Dr. Daly, used for ps1a/b
3  // and slightly edited by Nathan Khoury
4  #ifndef FIBLFSR_H
5  #define FIBLFSR_H
6
7  #include <iostream>
8
9  namespace PhotoMagic {
10     class FibLFSR {
11     public:
12         // Constructor to create LFSR with the given initial seed
13         explicit FibLFSR(std::string seed);
14         // Simulate one step and return the new bit as 0 or 1

```

```

15     int step();
16     // Simulate k steps and return a k-bit integer
17     int generate(int k);
18     // return register value
19     std::string getRegister() const;
20     // helper fuction for step()
21     char compareBits(char a, char b) const;
22 private:
23     std::string _register;
24 };
25 std::ostream& operator<<(std::ostream&, const FibLFSR& lfsr);
26
27 } // namespace PhotoMagic
28
29
30
31 #endif

```

FibLFSR.cpp

```

1 // Copyright 2024 Nathan Khoury
2 #include <string>
3 #include <SFML/Graphics.hpp>
4
5 #include "FibLFSR.hpp"
6
7
8 PhotoMagic::FibLFSR::FibLFSR(std::string seed) {
9     this->_register = seed;
10 }
11
12 int PhotoMagic::FibLFSR::step() {
13     char MSB = _register.at(0);
14     MSB = compareBits(MSB, _register.at(2));
15     MSB = compareBits(MSB, _register.at(3));
16     MSB = compareBits(MSB, _register.at(5));
17
18     // shift all the bits to the left
19     for (int i = 0; i < 15; i++)
20         _register.at(i) = _register.at(i + 1);
21
22     _register.at(15) = MSB; // MSB becomes LSB
23
24     // subtract '0' ascii value from ascii of either
25     // '0' or '1' to get the int 0 or 1
26     return static_cast<int>(MSB - '0');
27 }
28
29 int PhotoMagic::FibLFSR::generate(int k) {
30     int total = 0;
31     for (int i = 0; i < k; i++) {
32         total *= 2; // simulate left shift <<
33         total += this->step();
34     }
35     return total;
36 }
37
38 std::string PhotoMagic::FibLFSR::getRegister() const {
39     return _register;
40 }
41
42 char PhotoMagic::FibLFSR::compareBits(char a, char b) const {
43     if (a == b)
44         return '0';
45     return '1';
46 }
47

```

```

48 std::ostream& PhotoMagic::operator<<(std::ostream& out, const FibLFSR& lfsr) {
49     out << lfsr.getRegister();
50     return out;
51 }

```

PhotoMagic.hpp

```

1  // Copyright 2024 Nathan Khoury
2  #ifndef PHOTOMAGIC_H
3  #define PHOTOMAGIC_H
4
5  #include <SFML/Graphics.hpp>
6
7  #include "FibLFSR.hpp"
8
9  namespace PhotoMagic {
10 // encode an provided image reference
11 void transform(sf::Image&, FibLFSR*);
12 // helper function for transform
13 void pixelTransform(sf::Color&, FibLFSR* l);
14 }
15
16 #endif

```

PhotoMagic.cpp

```

1  // Copyright 2024 Nathan Khoury
2  #include <string>
3  #include <SFML/Graphics.hpp>
4
5  #include "PhotoMagic.hpp"
6  #include "FibLFSR.hpp"
7
8  void PhotoMagic::transform(sf::Image& img, FibLFSR* l) {
9      // get dimensions of image
10     sf::Vector2u size = img.getSize();
11     // convert from unsigned int to int
12     int xBounds = static_cast<int>(size.x);
13     int yBounds = static_cast<int>(size.y);
14     sf::Color p;
15     // loop through entire image
16     if (xBounds >= 1 && yBounds >= 1) {
17         for (int x = 0; x < xBounds; x++) {
18             for (int y = 0; y < yBounds; y++) {
19                 // get new pixel
20                 p = img.getPixel(x, y);
21                 // get new color
22                 PhotoMagic::pixelTransform(p, l);
23                 // reassign pixel
24                 img.setPixel(x, y, p);
25             }
26         }
27     }
28 }
29
30 void PhotoMagic::pixelTransform(sf::Color& p, FibLFSR* l) {
31     // XOR red
32     p.r ^= l->generate(8);
33     // XOR green
34     p.g ^= l->generate(8);
35     // XOR blue
36     p.b ^= l->generate(8);
37 }

```

3.7 Unit Testing

This was my first time ever using the Boost library for unit testing, let alone at all. I found it a bit tricky at first, since the only other time I had written any unit tests was a year prior to this in the Computing II lab for my C code, although I don't think I entirely understood the prevalence or significance of unit testing in programming at the time. These tests are rough around the edges, but they get the job done. My tests assert that individual steps through the FibLFSR are completed as expected using `step()` manually, that `generate()` performs not only the correct number of steps but yields the correct values based on the input seed, that interweaving manual stepping with `step()` as well as `generate()` does not cause any issues (i.e. they update the internal register appropriately), and that actions such as FibLFSR construction and calling `generate()` with a very large number do not throw any exceptions.

Alongside this first handful of tests that I described above I also had to test the overloaded (`<<`) stream insertion operator that I developed for the FibLFSR, which should output the current internal state of the register to the specified output stream. I would say this is where my biggest issue arised, because I had never handled a situation like this before. I really struggled to understand how to even go about this, but after more time than I'd like to admit I discovered via C++ documentation that a `std::ostringstream` object was actually perfect for what I wanted. With the `std::ostringstream` I was able to just write to it the contents of the FibLFSR and compare the string held by the object to a string with the expected state of the register. I had never really used stringstream prior to this but they actually turned out to be a very valuable asset for the projects that followed.

The remaining tests focused on verifying that individual pixels were getting properly encrypted properly. I did this by having tests that took a pixel with three specific color values, calling `transform()`, and verifying that the three color values became the new expected color values based on the seed that the register was given, and my manual calculation of the expected values before hand. The tests try different scenarios and combinations of edge cases to verify total functionality. Although, again, it is evident that these are not the greatest tests ever written since I was *very* inexperienced at the time of writing these. I'm still proud of how they came out though, with all things considered. Below is the `test.cpp` file which contains my Boost unit test module for this project: `test.cpp`

```
1 // Copyright 2024 Nathan Khoury
2
3 #include <iostream>
4 #include <string>
5
6 #include "FibLFSR.hpp"
7 #include "PhotoMagic.hpp"
8
9 #define BOOST_TEST_DYN_LINK
10 #define BOOST_TEST_MODULE Main
11 #include <boost/test/unit_test.hpp>
12
13 using PhotoMagic::FibLFSR;
14
15 // First two tests were provided
16 BOOST_AUTO_TEST_CASE(testStepInstr) {
17     FibLFSR l("1011011000110110");
18     BOOST_REQUIRE_EQUAL(l.step(), 0);
19     BOOST_REQUIRE_EQUAL(l.step(), 0);
20     BOOST_REQUIRE_EQUAL(l.step(), 0);
21     BOOST_REQUIRE_EQUAL(l.step(), 1);
22     BOOST_REQUIRE_EQUAL(l.step(), 1);
23     BOOST_REQUIRE_EQUAL(l.step(), 0);
24     BOOST_REQUIRE_EQUAL(l.step(), 0);
25     BOOST_REQUIRE_EQUAL(l.step(), 1);
26 }
27
28 BOOST_AUTO_TEST_CASE(testGenerateInstr) {
29     FibLFSR l("1011011000110110");
```

```

30 BOOST_REQUIRE_EQUAL(l.generate(9), 51);
31 }
32
33 // The next 5 tests were for ps1a
34 BOOST_AUTO_TEST_CASE(testGeneratePDFExample) {
35     // Uses the example in the pdf to test
36     // valid generate() behavior
37     FibLFSR myLFSR("1100011011000011");
38     BOOST_REQUIRE_EQUAL(myLFSR.generate(5), 6);
39     BOOST_REQUIRE_EQUAL(myLFSR.generate(5), 14);
40     BOOST_REQUIRE_EQUAL(myLFSR.generate(5), 24);
41     BOOST_REQUIRE_EQUAL(myLFSR.generate(5), 1);
42     BOOST_REQUIRE_EQUAL(myLFSR.generate(5), 13);
43     BOOST_REQUIRE_EQUAL(myLFSR.generate(5), 28);
44 }
45
46 BOOST_AUTO_TEST_CASE(testConstructor) {
47     // Very rudimentary test to assure
48     // LSFR can be constructed
49     BOOST_REQUIRE_NO_THROW(FibLFSR myLFSR1("0000000000000000"));
50     BOOST_REQUIRE_NO_THROW(FibLFSR myLFSR2("1111111111111111"));
51     BOOST_REQUIRE_NO_THROW(FibLFSR myLFSR3("0100011000100011"));
52     BOOST_REQUIRE_NO_THROW(FibLFSR myLFSR4("1111011101100000"));
53     BOOST_REQUIRE_NO_THROW(FibLFSR myLFSR5("1010110100011101"));
54 }
55
56 BOOST_AUTO_TEST_CASE(testStepAndGenerate) {
57     // Tests combination of manual stepping and generating
58     FibLFSR myLFSR("11011010010001011");
59     BOOST_REQUIRE_EQUAL(myLFSR.step(), 0);
60     BOOST_REQUIRE_EQUAL(myLFSR.generate(3), 2);
61     BOOST_REQUIRE_EQUAL(myLFSR.step(), 1);
62 }
63
64 BOOST_AUTO_TEST_CASE(testManyGenerations) {
65     // Test a high number of generations
66     FibLFSR myLFSR("11011010010001011");
67     BOOST_CHECK_NO_THROW(myLFSR.generate(9999));
68 }
69
70 BOOST_AUTO_TEST_CASE(testOutput) {
71     // Check for proper output
72     FibLFSR myLFSR("0111011101100001");
73     std::ostringstream oss;
74     oss << myLFSR;
75     BOOST_REQUIRE_EQUAL(oss.str(), "0111011101100001");
76 }
77
78 // The remaining tests are new for ps1b
79 BOOST_AUTO_TEST_CASE(test1x1Image) {
80     FibLFSR myLFSR("1101100011011001");
81     sf::Color color(24, 99, 255);
82     sf::Image img;
83
84     // Create 1x1 test image with given color
85     img.create(1, 1, color);
86     PhotoMagic::transform(img, &myLFSR);
87
88     sf::Color p;
89     p = img.getPixel(0, 0);
90
91     // Manually verify every color value is XOR'd correctly
92     BOOST_REQUIRE_EQUAL(p.r, 126);
93     BOOST_REQUIRE_EQUAL(p.g, 56);
94     BOOST_REQUIRE_EQUAL(p.b, 249);
95 }

```

```
96
97 BOOST_AUTO_TEST_CASE(test1x1ImageReverse) {
98     FibLFSR myLFSR("1101100011011001");
99     sf::Color color(126, 56, 249);
100     sf::Image img;
101
102     // Create 1x1 test image with color from previous test
103     img.create(1, 1, color);
104     PhotoMagic::transform(img, &myLFSR);
105
106     sf::Color p;
107     p = img.getPixel(0, 0);
108
109     // Manually verify every color returns to original
110     // value before previous encryption
111     BOOST_REQUIRE_EQUAL(p.r, 24);
112     BOOST_REQUIRE_EQUAL(p.g, 99);
113     BOOST_REQUIRE_EQUAL(p.b, 255);
114 }
```

4 PS2: Pythagorean Tree

4.1 Functionality

The Pythagorean Tree program is aimed at creating a visually impressive tree based off of squares and recursion. The concept behind the premise is very simple: Given a starting length L , a square with $L \times L$ dimensions is constructed and centered at the bottom center of the display window. From there, the top left and top right corner are given to a recursive function call which will iterate N times as is specified by the user. From every square that is generated, with respect to that square, its outward-facing top two corners are also passed to two more recursive function calls, which continues until N iterations are satisfied, at which point the calls return no value and exit their recursive branches. At the end of the recursive calls, the tree is displayed on the screen for the user to view, with colors that fade from black to purple to white with the more iterations that have occurred (Example output visible in figure 5). Within my implementation of the Pythagorean Tree program, the user may *Optionally* specify as a third parameter a desired angle A which will represent the angle of rotation that a **left** child square will rotate away from its parent. The user does not need to worry about the rotation B of the right child square since it is automatically calculated by solving $B = 90 - A$ internally.

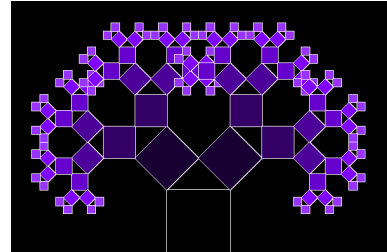


Figure 5: Sample output with parameters: $L = 100$,
 $N = 6$

A slightly more nuanced flow of the program is as follows: After collecting and verifying sufficient command-line input, the window's dimensions are dynamically determined at runtime based on the length of the base square L , the methodology of which is described in section 4.2. An instance of the `Tree` class is created and named `myPTree`. It is constructed using the required value constructor which expects an argument of type `double A` which represents the angle of rotation that the *left* child square will rotate away from its parent. This is stored within private data member `_A`, along with private data member `_B` which is derived using the method explained in the previous passage. The constructor also requires a second argument of type `integer N` which stores the amount of iterations to perform specified by the user. This parameter is stored in private data member `_N` and is primarily used to calculate the current depth of the tree, which is further used to determine the color to be applied to the branches at this depth.

Once the tree object is instantiated, a call to `Tree::PTree()` is made which requires the arguments: Length of side of the base square (`double L`), amount of iterations to complete (`integer N`), width of the display window (`unsigned integer wx`), and lastly the height of the display window (`unsigned integer wy`). It is worth mentioning that the significance of supplying the display window dimensions is to allow the base of the tree to always be placed in the center of the viewport; Therefore, it is required to inform `PTree()` of the dimensions of the window since they are determined dynamically at runtime.

Within `PTree()` the base square is created and placed appropriately in the center of the bottom of the display window. The position of the upper two corners of this base square are calculated, and two recursive calls to a helper function are made supplying the calculated upper corners as starting positions for each square. Each recursive call generates a child square in the appropriate position, rotated the proper amount, and also *shrunk* by an amount calculated by considering the parent's side length, as well as the angle of rotation. Each recursive call will make two more recursive calls based off of its respective upper corners, continuing until the desired number of iterations is reached. All the recursive functions return, and the display window is opened. Within the tree object, a storage container holding the tree and an overridden SFML `sf::draw()` function are used to draw the tree to the window every frame until the user closes the window which terminates the program. For a greater and more technical review of the overridden `sf::draw()` function, the container storing the branches of the tree, and the math behind generating each iteration, proceed to section 4.2.

4.2 Features

A more detailed explication of the process of generating each square per iteration would yield the following series of steps:

1. Square is created from the `sf::RectangleShape` type from the SFML graphics library
2. Square size is created with dimensions $L \times L$ (L is provided by the caller)
3. Aesthetics like color and outline are added
4. Origin is shifted to bottom left of square
5. Position is set to `pos` (`pos` is a `sf::Vector2f` supplied by the caller denoting the position the callee square should originate from)
6. Square is rotated by the `totalRot` to catch it up to speed with the parent (if it is a right child square, it is flipped an additional -90 degrees to lie on top of the parent square, not doing so would leave it hanging over the right side since the origin is bottom left)
7. Square is rotated by the constant angle amount (Either A° or B°)
8. `totalRot` is incremented/decremented appropriately, depending on if it is a left/right child square
9. Square is pushed into vector (`_treeVector` is a private class data member)
10. Check if more iterations are required. If so, get info needed to continue, and then reiterate (passing $N - 1$ as the total iterations remaining). If not, `return`

Now that it is clear how the squares come to be, it is important to explain how their properties are derived per iteration. By this, it's most important to examine how side length and positions are calculated. Beginning with the more rudimentary of the two, side length is calculated using two simple calculus formulas: To calculate the side length of the left child square, we can use the formula $L' = L * \cos \theta$ where L' represents the left child's side length, L represents the parent's side length, and θ is the angle of rotation A which the left child rotates by each iteration. As a reminder, the value of A is 45° by default unless otherwise specified as a third argument in the command line. To calculate the side length of the right child square, the formula $L' = L * \sin(90^\circ - \theta)$ is utilized. L' and L represent the same values as with the left child square; However, the $90^\circ - \theta$ is represented by the value of B within the program. As another reminder, B is derived from the angle A meaning that if A were to be 45° , B would also be 45° . Figure 6 displays what one iteration of the tree would look like given that A is left default or specifically set to 45° . Figure 6 also contextualizes the formulas by showing the angles formed by the child squares with respect to the parent square, as well as the greater triangle formed by the combination of all three squares.

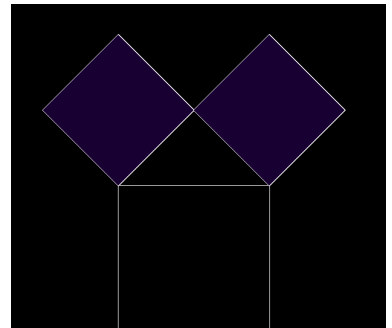


Figure 6: Example of one iteration, showing the triangle formed by the empty space between squares

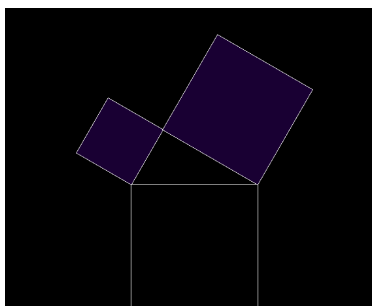


Figure 7: Exemplification of the triangle formed by the two child squares and the parent when angle A is 60° , and B is 30° as a result

Figure 7 provides a visual example of what a value for **A** that is not 45° would yield. It depicts not only the skewed triangle that is formed between all of squares, but it also reveals the significance of the separate formulas since the squares are very clearly not identical in length.

The more challenging aspect of the child square generation was determining the positions of the next two child squares. Recall that the locations of where to grow child squares from are derived from the parent's starting point. It should be noted that total rotation (**totalRot**) is stored with respect to the Unit Circle's counterclockwise rotations, so a negative value for **totalRot** is **negative** for the *unit circle*, yet **positive** for *SFML rotation*. The total rotation and the angle produced *this* iteration, with respect to any additional angular offsets due to the corners of the newly produced child squares, are input into **cos** to get the new x offset, and **sin** to get the new y offset. This should calculate the point of the first child, **p1**. **p1** is then used as the starting point of the next calculation, which uses **cos** and **sin** to calculate x and y offset again (while considering new offsets that occur moving relative to the child square's corners, such as 90° or 180°), storing the resulting position in **p2**. **p1** and **p2** are used as arguments to call the next iteration of child squares. Below is a small snippet from the code which shows this process in the program, within the helper function for *left* child squares called **PTreeIterateLeft()**.

```

1 sf::Vector2f p1, p2;    // store next growth points
2
3 p1 = pos;    // start at parent's position for reference
4 p1.x += L * cos(((totalRot + 90) / 180) * M_PI);
5 p1.y -= L * sin(((totalRot + 90) / 180) * M_PI);
6
7 p2 = p1;    // start at p1's position for reference
8 p2.x += L * cos(((totalRot) / 180) * M_PI);
9 p2.y -= L * sin(((totalRot) / 180) * M_PI);

```

The occurrence, and quite prominent redundancy, of the separate **PTreeIterateLeft()** and **PTreeIterateRight()** helper functions is discussed in section 4.3, as there are slight differences that could've been handled much more appropriately in retrospect which would not have warranted two separate helper functions.

Finally, I would like to mention two things: First of all, the container used for storing the child squares was a **std::vector<sf::RectangleShape>** named **_treeVector**. A vector was a very simple and easy choice for this sort of situation; I don't need to index the container for any reason during the program, I simply need something that can hold all the squares generated, and something that I can easily iterate over to call **draw()** on all the elements within. The second thing I'd like to expand on is the overridden **virtual void draw()** function that I was required to override due to the fact that my class inherits from **sf::Drawable** which provides the before-mentioned **pure** virtual function. I actually, for better or for worse, inlined the definition for this which features a ranged for loop that uses the **sf::RenderTarget&** provided (i.e. the display window in main) to call **draw()** on each square within the vector. If I were to refactor this code now, I don't think I would stand by the inlining of this function. There's no harm in moving it to the .cpp file; However, there is harm in cluttering the .hpp file.

4.3 Personal Reflection

4.3.1 The Struggles

For me, the math was very challenging to wrap my head around. I don't think it's because the math is intrinsically challenging for students like myself who have made it to this class, I feel it's the aspect of trying to apply it to SFML which we were all still relatively new at, as well as implementing it in C++ in general which I personally still felt somewhat lost within, which posed the greatest difficulty. I spent way more time than I will ever admit trying to draw diagrams, implement new ideas, adding and subtracting new angular offsets, and convincing myself that negatives belonged where they certainly did not. In the end it was not frantically trying everything I could to break through which solved it, it was the step by step dissection and analysis of what was going wrong in the **p1** and **p2** calculations which allowed me to correct slight mistakes, which corrected the whole thing. Probably the

biggest reason why it was so complex to understand what was going wrong was the fact that it would make it about three iterations deep just fine, but then spontaneously at the fourth iteration everything would go wrong. It took a lot of time to break through, but nothing felt better than finally squashing the bug and finishing the program.

4.3.2 What I Learned

Most notably I learned to break down the problem in way more ways than I thought possible. Drawing diagrams is very helpful, yes, and loading up a "playground" program I made to understand how setting/rotating around new origin points works was great; But it wasn't these that debugged the program, it was a stable, level-minded, thorough comb through the program which finally revealed the issues. That's not to say that the takeaway was combing through the program will fix every bug, there were certainly issues with even that process when it came to debugging future assignments. Rather, I'm saying that panicking got me nowhere, and sinking hours into drawing pictures and verifying the math on paper is not the only thing that counts. Approaching the program with a fresh, dedicated mind, and digging deep into what I knew about C++ and SFML is what got me through this.

I would also like to acknowledge the separated left/right helper functions. I think this was part of a frantic panic to fix the bug that consumed my life for a couple days, and I absolutely should have gone back to refactor it. The two helper functions are almost identical, and the ways they differ could be solved with perhaps an additional parameter, or modifying an existing parameter to transmit that information. At the very least, the large chunks of identical code could be split into another function to not only shrink the massive size of these helper functions, but to reduce the significant amount of redundant code in this project.

More general things that I learned included new experience overriding a pure virtual function in a class I made on my own, experience manipulating objects from an external library like SFML in the context of a C++ container like a vector, and also working with the `sf::Vector2f` data type in the SFML library which was one of the most important data types within the SFML library I got familiar with this semester.

4.3.3 The Takeaway

I felt that personally this was my worst project in terms of how the end result turns out. I don't mean on the user's end, I think the tree looks great and I enjoyed implementing the colors. I'm referring to the cleanliness of the code and reluctance to refactor. I learned a lot from this assignment just after submission, and I'm learning a lot more looking back on it. The next assignment, PS3 Sokoban, is when I felt like I started to up my game in a major way. Sokoban is far from perfect, but I feel there are many redeeming qualities found within it which help outweigh some of the worse aspects of the code. For this project, the Pythagorean Tree, I feel it is poor in too many ways for me to say I'm proud of most aspects of it. I do not think it was a waste of time, but I learned a lot about not only the importance of revisiting and refactoring code, but the value of writing good, clean code **the first time**.

4.4 Running The Program

The Pythagorean Tree program can be compiled with *make* in correspondence with the provided Makefile. This will produce an executable named **PTree** which can be executed in the terminal using the following format:

```
1 ./PTree L N <(optionally) left angle of inner triangle>
```

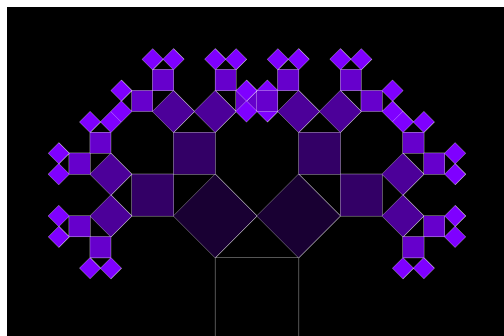
The first parameter is the length of the base square as double value **L**

The second parameter specifies the depth of recursive calls to make as an integer value **N**

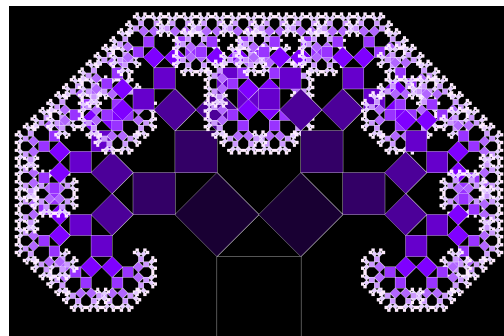
The final parameter is completely **optional** and specifies the angle of rotation for the left angle of the inner triangle. This also determines the measurement of the right angle of the inner triangle, as well as the side lengths of the child squares. Figure 7 displays this, and more examples occur in section 4.5.

4.5 Proof of Functionality

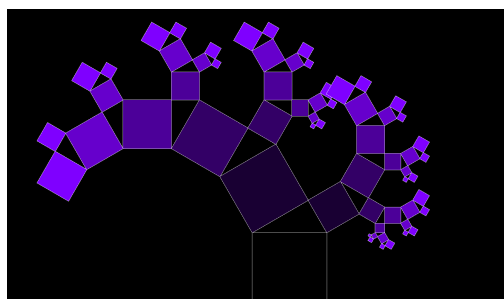
Figure 8: Examples of sample program output



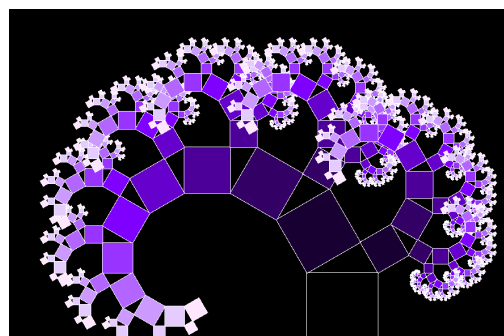
(a) Tree generated with $L = 250$, $N = 5$



(b) Tree generated with $L = 250$, $N = 10$



(c) Tree generated with $L = 250$, $N = 5$, $\theta = 30$



(d) Tree generated with $L = 125$, $N = 10$, $\theta = 30$

Please note that the figures above are not scaled proportionally to each other. The base squares for each tree look very similar when, in practice, they actually generate quite differently when the L value changes. A leap from $L = 250$ to $L = 125$ should produce base squares with a 2:1 side length ratio when compared to each other in the actual display window.

It is also a good time to quickly address undesirable visual issues such as the awkward overlap that occurs when the branches of the tree interact with one another. The artifact I am referring to is most prevalent in trees with many iterations like figure 8b. This is simply due to the order in which the squares are added to the internal vector, and the order in which they are drawn (which just steps through the whole vector). Overall, not a huge deal, nor was it a concern for the project itself, but certainly worth acknowledging.

4.6 Source Code

The following displays the source code for the Pythagorean Tree program. Alongside a Makefile and main.cpp, the PTree.hpp and PTree.cpp files are also showcased, which contain the full implementation of the recursive tree object

Makefile

```

1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -lboost_unit_test_framework
4 # Your .hpp files
5 DEPS = PTree.hpp
6 # Your compiled .o files
7 OBJECTS = PTree.o
8 # The name of your programs
9 PROGRAM = PTree
10
11 .PHONY: all clean lint
12
```

```

13 all: $(PROGRAM)
14
15 # Wildcard recipe to make .o files from corresponding .cpp file
16 %.o: %.cpp $(DEPS)
17     $(CC) $(CFLAGS) -c $<
18
19 PTree: main.o $(OBJECTS)
20     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
21
22 clean:
23     rm *.o PTree
24
25 lint:
26     cpplint *.cpp *.hpp

```

main.cpp

```

1  // Copyright 2024 Nathan Khoury
2  #include <iostream>
3  #include <string>
4  #include <SFML/Graphics.hpp>
5
6  #include "PTree.hpp"
7
8  int main(int argc, char *argv[]) {
9      if (argc < 3) { // need all arguments to be passed
10         std::cerr << "please pass all arguments, returning -1"
11             << std::endl;
12         return -1;
13     }
14     double L = std::stod(argv[1]); // get side length of starting square
15     int N = std::stoi(argv[2]);    // get recursion depth
16
17     double angleSupplied = 45;    // set default angle
18     unsigned int windowHeight = 6 * L; // set default window height
19     unsigned int windowWidth = 4 * L; // set default window width
20
21     if (argc >= 4) {
22         angleSupplied = std::stod(argv[3]); // input custom angle
23         if (angleSupplied != 45) {
24             windowWidth = 1900;
25             windowHeight = 1000;
26         }
27     }
28     // Create tree with supplied parameters
29     Tree myPTree(angleSupplied, N);
30     myPTree.PTree(L, N, windowWidth, windowHeight);
31
32     // Render the window for drawing
33     sf::RenderWindow window
34         (sf::VideoMode(windowWidth, windowHeight), "PTree drawing program");
35
36     window.setPosition({0, 0});
37
38     while (window.isOpen()) {
39         sf::Event event;
40         while (window.pollEvent(event)) {
41             if (event.type == sf::Event::Closed) {
42                 window.close();
43             }
44         }
45         window.clear();
46         window.draw(myPTree);
47         window.display();
48     }
49     return 0;
50 }

```

PTree.hpp

```
1 // Copyright 2024 Nathan Khoury
2 #ifndef PTREE_H
3 #define PTREE_H
4
5 #include <iostream>
6
7 #include <vector>
8 #include <SFML/Graphics.hpp>
9
10 class Tree : public sf::Drawable {
11 public:
12     explicit Tree(double A = 45, int N = 0) : _A(A), _B(90 - A), _N(N) {}
13     void PTree(double L, int N, unsigned int wx, unsigned int wy);
14
15 private:
16     // Overridden draw function from sf::Drawable
17     virtual void draw
18         (sf::RenderTarget& target, sf::RenderStates states) const {
19         for (auto it : _treeVector)
20             target.draw(it);
21     }
22     // Helper functions for PTree()
23     void PTreeIterateLeft(double L, int N, sf::Vector2f pos,
24         float totalRot, double rot);
25     void PTreeIterateRight(double L, int N, sf::Vector2f pos,
26         float totalRot, double rot);
27     sf::Color determineColor(int depth);
28     // Internal storage of squares
29     std::vector<sf::RectangleShape> _treeVector;
30     // Internal representation of the two inner angles of the inner triangle
31     const double _A, _B;
32     // Internal memory of the total depth to recurse to
33     const int _N;
34 };
35
36 #endif
```

PTree.cpp

```
1 // Copyright 2024 Nathan Khoury
2 #include <iostream>
3 #include <cmath>
4 #include <SFML/Graphics.hpp>
5 #include "PTree.hpp"
6
7 void Tree::PTree(double L, int N, unsigned int wx, unsigned int wy) {
8     sf::RectangleShape square;
9     sf::Vector2f v;
10     // position the base square in bottom-center of the screen
11     v.x = L;
12     v.y = L;
13     square.setSize(v);
14     square.setOrigin(L / 2, L);
15     v.x = wx / 2;
16     v.y = wy;
17     square.setPosition(v);
18     square.setOutlineThickness(1);
19     square.setOutlineColor(sf::Color::White);
20     square.setFillColor(determineColor(N));
21     _treeVector.push_back(square);
22
23     if (N <= 0) {
24         return;
25     } else {
26         // Left child branch
27         v.x -= (L / 2);
```

```

28     v.y -= L;
29     // new side len = cos(angle in radians)
30     double LNew = L * cos((_A / 180.0) * M_PI);
31     PTreeIterateLeft(LNew, N - 1, v, 0, _A);
32
33     // Right child branch
34     v.x += L;
35     // new side len = sin(angle in radians)
36     LNew = L * sin((_A / 180.0) * M_PI);
37     PTreeIterateRight(LNew, N - 1, v, 0, _B);
38 }
39 }
40
41 void Tree::PTreeIterateLeft(double L, int N, sf::Vector2f pos,
42 float totalRot, double rot) {
43     // create next square
44     sf::RectangleShape square;
45     sf::Vector2f v(L, L);
46     square.setSize(v);
47     square.setOutlineThickness(1);
48     square.setOutlineColor(sf::Color::White);
49     square.setFillColor(determineColor(N));
50     // tranform new square
51     sf::Vector2f orig(0, L);
52     square.setOrigin(orig);
53     square.setPosition(pos);
54     square.rotate(-totalRot); // counterclockwise
55     // rotate square counterclockwise _A degrees
56     square.rotate(-rot);
57     totalRot += rot; // create a new total
58     _treeVector.push_back(square);
59     // iterate again?
60     if (N <= 0) {
61         return; // no
62     } else {
63         // yes
64         sf::Vector2f p1, p2; // store next growth points
65
66         p1 = pos; // start at parent's position for reference
67         p1.x += L * cos(((totalRot + 90) / 180) * M_PI);
68         p1.y -= L * sin(((totalRot + 90) / 180) * M_PI);
69
70         p2 = p1; // start at p1's position for reference
71         p2.x += L * cos(((totalRot) / 180) * M_PI);
72         p2.y -= L * sin(((totalRot) / 180) * M_PI);
73
74         // call left branch
75         double LNew = L * cos((_A / 180.0) * M_PI);
76         PTreeIterateLeft(LNew, N - 1, p1, totalRot, _A);
77         // call right branch
78         LNew = L * sin((_A / 180.0) * M_PI);
79         PTreeIterateRight(LNew, N - 1, p2, totalRot, _B);
80     }
81 }
82
83 void Tree::PTreeIterateRight(double L, int N, sf::Vector2f pos,
84 float totalRot, double rot) {
85     // create next square
86     sf::RectangleShape square;
87     sf::Vector2f v(L, L);
88     square.setSize(v);
89     square.setOutlineThickness(1);
90     square.setOutlineColor(sf::Color::White);
91     square.setFillColor(determineColor(N));
92     // tranform new square
93     sf::Vector2f orig(0, L);

```

```

94 square.setOrigin(orig);
95 square.setPosition(pos);
96 square.rotate(-90); // flip to be on top of parent
97 square.rotate(-totalRot); // clockwise
98 // rotate square clockwise _B degrees
99 square.rotate(rot);
100 totalRot -= rot; // create a new total
101 _treeVector.push_back(square);
102 // iterate again?
103 if (N <= 0) {
104     return; // no
105 } else {
106     // yes
107     sf::Vector2f p1, p2; // store next growth points
108
109     p1 = pos; // start at parent's position for reference
110     p1.x += L * cos(((90 + totalRot) / 180) * M_PI);
111     p1.y -= L * sin(((90 + totalRot) / 180) * M_PI);
112
113     p2 = p1; // start at p1's position for reference
114     p2.x += L * cos(((180 + totalRot) / 180) * M_PI);
115     p2.y -= L * sin(((180 + totalRot) / 180) * M_PI);
116
117     // call left branch
118     double LNew = L * cos((_A / 180.0) * M_PI);
119     PTreeIterateLeft(LNew, N - 1, p2, totalRot, _A);
120     // call right branch
121     LNew = L * sin((_A / 180.0) * M_PI);
122     PTreeIterateRight(LNew, N - 1, p1, totalRot, _B);
123 }
124 }
125
126 sf::Color Tree::determineColor(int depth) {
127     int x = _N - depth;
128     sf::Color c(0, 0, 0);
129     switch (x) {
130         case 0: // black
131             return c;
132             break;
133         case 1: // deeper purple
134             c.r = 25;
135             c.b = 51;
136             return c;
137             break;
138         case 2: // deep purple
139             c.r = 51;
140             c.b = 102;
141             return c;
142             break;
143         case 3: // purple
144             c.r = 76;
145             c.b = 153;
146             return c;
147             break;
148         case 4: // slightly brighter purple
149             c.r = 102;
150             c.b = 204;
151             return c;
152             break;
153         case 5: // bright purple
154             c.r = 127;
155             c.b = 255;
156             return c;
157             break;
158         case 6: // vibrant purple
159             c.r = 153;

```

```

160         c.g = 51;
161         c.b = 255;
162         return c;
163         break;
164     case 7:      // darker lavender
165         c.r = 178;
166         c.g = 102;
167         c.b = 255;
168         return c;
169         break;
170     case 8:      // lavender
171         c.r = 204;
172         c.g = 153;
173         c.b = 255;
174         return c;
175         break;
176     case 9:      // faint lavender
177         c.r = 229;
178         c.g = 204;
179         c.b = 255;
180         return c;
181         break;
182     case 10:     // essence of lavender
183         c.r = 255;
184         c.g = 230;
185         c.b = 255;
186         return c;
187         break;
188     default:
189         return sf::Color::White;
190         break;
191 }
192 }

```

4.7 Unit Testing

Unit tests were not necessary for this project.

4.8 Bonus

For this assignment I tackled two of the bonus requirements. The first of which, the optional ability for the user to specify the measurement of the left angle within the inner triangle (angle **A**), I discussed sufficiently in sections 4.1 and 4.2 so I won't reiterate too much here. In short, the class calculates the right angle of the inner triangle, **B**, and then uses **A** and **B** when calculating rotations during the recursion. I baked it into my implementation the first time, so I didn't have to try and edit anything to make it work afterwards. The second bonus requirement I completed was the custom coloration of the tree. A helper function called `Tree::determineColor()` takes an integer `depth` as a parameter and determines using a switch statement what color to apply. 10 layers of unique colors are supported, but starting at the 11th layer onward the default case sets the squares to be white. This is because I manually chose the RGB color values for each layer, and I could not possibly account for an infinite amount of recursion. I am certain there are ways to sustain a gradient in correlation to the depth `N` selected by the user, but I feel that my implementation is fine (albeit a hefty switch statement), and the end result is quite attractive.

5 PS3: Sokoban

5.1 Functionality

Sokoban is a fairly well-known puzzle game which allows the player to navigate a warehouse-like environment, with the ultimate goal of moving all the storage crates into their respective storage locations. PS3 required that a somewhat simplified version of the game be reproduced, such that it would require a level filename as input through the command-line, load the file, interpret the symbols, and produce a playable game level. The game should verify consistently if the game has been won, and if so, a victory message should be displayed for the user and movement should be disabled.

An example of appropriate level format can be found below (level1.lvl file):

```
1 10 10
2 #####
3 #...a...#
4 #...A...#
5 #.....#
6 #...#...#
7 #...#...#
8 #..@..A..#
9 #.....a#
10 #.....#
11 #####
```

The first number provided denotes the *height* of the game board, with the unit of measurement being game tiles. The second number is representative of the *width* of the game board. The remaining lines of symbol express what tile should be placed in that slot on the board. The list of valid symbols, as well as what tile they represent respectively, are provided now:

- @ indicates the location in which **the player** should begin the level in
- . indicates empty space, which should visually be represented by the **ground tile**
- # indicates a **wall** tile
- A indicates a **box** tile which the player will likely attempt to interact with to achieve victory
- a indicates a **storage location** tile, which the player can push a box onto as part of the attempt to win. Boxes are not bound to storage locations when moved onto one
- 1 indicates a **box in storage location** already, so the program should keep this in mind when the player goes to push the box. This is simply drawn as a box during the program despite this, there is no unique tile for this scenario.

When the level is completely loaded in to some internal representation, which I will explicate further in section 5.2, an SFML `sf::RenderWindow` object is instantiated and dynamically sized to match the size of the game board exactly. Details on this, as well as future processes mentioned here, are also elaborated on in section 5.2. The main game loop begins at this point, and the player should be able to start attempting to solve the puzzle. The main loop uses the SFML `sf::Event` type to check if a `sf::Event::KeyPressed` event has occurred. If not, the game is redrawn as a new frame and it checks again. When the player *does* press any key, a switch statement determines what appropriate action should be processed in the game to correspond to the key. The following displays the full set of controls which yield responses from the game, other inputs are not processed and will not impact the game state (aside from closing the window... that will definitely impact the game state).

- W or Up Arrow can be used to move the player **up** one tile.
- S or Down Arrow can be used to move the player **down** one tile.
- A or Left Arrow can be used to move the player to the **left** one tile.
- D or Right Arrow can be used to move the player to the **right** one tile.

R is used to **restart** the game state back to the state presented in the original level file

Q is used to **undo** up to *five* of the most recent moves (I opted for Q rather than the intuitive U which would've more clearly correlated to the **undo** functionality due to the distance away from WASD each key was. I felt that convenience for the user was more important than a naming convention).

Within each function that performs a corresponding action based on input is a check for if the game has been won, denying the movement from being processed if so. At the end of each frame, after the game has been drawn, victory text is also drawn if the game was won. Proceed to section 5.2 for a more in-depth analysis of the design of the program as well as some of the major decisions that went into implementing aspects of the game.

5.2 Features

Moving along with the flow of the program, I'll explain all of the major design choices made within the program. This includes, but is not limited to: How the game is represented internally, the anatomy of the `SB::Sokoban` class which governs the loading, modification, display, storage, and other states of the game, and even my very first lambda that I wrote.

Starting at the very beginning, it is verified that a level was actually supplied to the command-line, and if not, we shame the user appropriately (customer service 101), and kick them out of the program to let them think about their actions and try again. With the level supplied, a `SB::Sokoban` object is created and a file is opened. The `Sokoban` class is found within the custom `SB` namespace and manages the vast majority of the game with the public functions it provides such as `movePlayer()`, `undo()`, `isWon()`, and more. With the file open, the overridden stream extraction operator (`»`) is used to read the level file into the internal storage of the `Sokoban` class.

It is important to explain how the actual Sokoban game is represented internally at this point. A singular `std::vector<char>` named `_data` is used to represent the game state internally as a **two-dimensional matrix**. Although there are two-dimensional matrices built into C++ itself, the contents of them *must* be determined at **compile-time**, *not* **run-time**. It is similarly possible to use a `std::vector<std::vector<char>>` for easy indexing in the form of `[i][j]`; However, I just felt this was not necessary for a relatively simple task like this. You are able to manipulate a one-dimensional array/vector to behave as if it were two-dimensional by applying the following formula in place of the desired index:

$$i = x + y * w$$

The equation yields that the desired index *i* is equal to the y position of the desired index *y* multiplied by the width of the matrix *w*, with the x position of the desired index *x* added to that result. Furthermore, this equation expresses how to calculate the index when using a one-dimensional array/vector as if it were two-dimensional, specifically in the context of a *row-major order* implementation of the matrix. Row-major order reads the matrix from left to right, moving across the columns, stepping down one *row* only after all of the *columns* in that row have been utilized. The alternate perspective for manipulating a one-dimensional array/vector as a two-dimensional matrix is known as *column-major order*. Column-major order is functionally the opposite of row-major order, in the sense that it instead reads the matrix from top to bottom, moving down the rows, stepping over one *column* only after all of the *rows* in that column have been utilized. In short, the method that I adhered to is similar to reading the matrix like a book if it's contents were to be organized in front of you in a grid accurately representing the organization of the contents. A short example below demonstrates how the differences described above impact the order that elements are indexed by while using the formula above. Keep in mind that the formula above is designed for *row-major order* not column-major order. Column-major order would warrant slight changes to accomodate the differences.

	Row-Major Order	Column-Major Order
1	//	
2	[1][2][3][4][5]	[1][4][7][10][13]
3	[6][7][8][9][10]	[2][5][8][11][14]
4	[11][12][13][14][15]	[3][6][9][12][15]

With that out of the way, finally, lets return back to the overridden stream extraction operator (`>>`). The way it functions is by first reading the height and width in, since those are the first two values provided. Then, while grabbing one line of symbols at a time, the current line of symbols is broken into individual characters which are read into the internal matrix. When all of the characters are stored in the matrix, the `std::istream&` returns.

At this point, the miscellaneous SFML elements required are created like the window, the victory text, etc. The main game loop begins at this point. Every frame the game first sets the `gameWon` boolean flag according to the value returned by `SB::Sokoban::isWon()`. `isWon()` determines if the game is won based on if there are either **no more storage locations** left (the boxes have filled all of the storage locations, remaining boxes do not matter for the win condition) or **no more boxes left** (all of the boxes have been placed in storage locations, extra storage locations do not matter for the win condition). The `isWon()` function also contains not only the first lambda that I wrote, but also the second (which is almost identical to the first, so it hardly counts). Below is the body of the `isWon()` function, showing how the lambdas are used to determine if the game is finished or not.

```
1 auto boxIterator = std::find_if(_data.begin(), _data.end(), [](char x) { return x == 'A'; });
2 if (boxIterator == _data.end())
3     return true;    // win if there are no more boxes
4
5 auto storageIterator = std::find_if(_data.begin(), _data.end(), [](char x) { return x == 'a'; });
6 if (storageIterator == _data.end() && !_standingOnStorage)
7     return true;    // win if there are no more storage areas
8
9 return false;    // otherwise the game is not won.
```

Using the `std::find_if()` algorithm from the standard template library `<algorithm>` library, I added my first custom lambda expression which captures no values, accepts the current character being iterated upon as its sole parameter, and returns the value generated by the boolean expression `x == 'A'`. If `std::find_if()` returns `_data.end()` this signifies that there was no unmatched box left on the game board, meaning that the game has been won, so the function returns `true`. Otherwise, if the character `'A'` is found within the text, we move onto the next `std::find_if()` statement which is almost identical to the first but returns the value generated by the boolean expression `x == 'a'`, which searches for storage locations. The same logic for determining if the state of the boxes satisfies the win condition also applies to the storage locations. If there is still an unmatched storage location *and* an unmatched box on the game board, the game has not yet been won, and the function returns `false`.

Back to the main loop now, after the win condition has been checked, the program checks if a key has been pressed, and if so, what key it was. If it is any of the keys that the program is concerned about, the program executes the corresponding function. Pressing `'R'` causes a restart of the game state, which is fairly straightforward since we just reopen the file, load the data back in, and reset all of our internal boolean flags back to a state that reflects the beginning of the game. Although reopening the file is expensive in theory, I feel it is not a considerable issue in this case since the level sizes are consistently small, the data is lightweight (just a grid of characters), and also, in practice, the player would realistically be restarting rather sparingly. Pressing `'Q'` allows you to undo a move, with the limit being up to five of your most recent moves. This is explained in detail in section 5.8, so I will skip over the implementation of it here. The last process worth analyzing is the function `SB::Sokoban::movePlayer()`.

Within `movePlayer()` there are two significant `if` statements worth looking at. The first checks for input to move up/down, the second checks for input to move left/right. The reason these were separated was... no good reason at all. I suppose at the time I thought it was better to keep them separate, but this is blatantly so egregiously redundant in retrospect

that it kind of amazes me. This also holds true to a certain degree for all of the helper functions within `movePlayer()` like `isOutOfBoundsX()` vs `isOutOfBoundsY()`, `isWallX()` vs `isWallY()`, etc. To be clear, I stand by the helper functions existing for the same purpose of ones like `isWallX/Y()`; However, the separation of X from Y is sufficient, sure, but it just yields so much redundant code, and doubles the amount of helper functions which just makes the whole thing *that* much messier. Moving past all of the issues, which I will reiterate in section 5.3.1, both of the `if` statements follow the same flow which I will now expand below as a series of steps. Note that all of the following helper functions mentioned will take the parameters: `p` representing the player location as a `sf::Vector2i`, and `offset` as an integer representing the direction the player is attempting to move in. Also note that I will omit the X/Y suffix of the helper functions (i.e. `isBoxX()`→`isBox()`) since this process applies to both directions. The steps for determining if movement is valid or not is as follows:

- Check if the move will take the player out of bounds using `isOutOfBounds()`, if not, continue to next step.
- Check if the move is blocked by a wall `isWall()`, if not, continue to next step.
- Check if the player will walk into a box with `isBox()`. If so, verify that the box will not be pushed out of bounds with `isOutOfBounds()`, that it will not be blocked by a wall with `isWall()`, and that it will not collide with another box with `isBox()`. If any of these are true, do not move the player or the box.
 - If all is well, check if there is a storage location in the tile past the one the player is attempting to walk into. If so, move successfully and push the box onto the storage location
 - Otherwise, just push the box. If the player is now on a storage location set the internal `_standingOnStorage` boolean flag to `true`. If the player was standing on a storage location, be sure to replace the storage location within the internal matrix
- If nothing is blocking the player, and the player is not trying to push a box, just step forward one tile. Check if the player is stepping on/off a storage location and set the internal `_standingOnStorage` boolean flag appropriately.

This is a rather complex tree of nested `if` statements, and certainly is not optimal at all, but it covers all the cases and gets the job done. After the player movement is determined, the function returns and a new frame is drawn, causing the loop to continue. The loop continues until the `isWon()` condition is met, at which point movement is disabled and a victory message pops up. Congrats! You beat Sokoban! :)



Figure 9: Victory text that displays when the player completes a level successfully

5.3 Personal Reflection

5.3.1 The Struggles

One of the aspects of this project that challenged me the most was tracking the player trampling the storage locations. Since I represent the player internally in my matrix as the @ symbol, and storage locations as the a character, I had an issue with storage locations being trampled when a player would step on them, since the @ would overwrite the a, and not replace it when the player stepped off. My solution was to have an internal boolean flag called `_standingOnStorage` which tracked if the player was on storage. I then modified all of the movement logic to set/unset the flag accordingly, and replace the storage location on the ground when necessary. Did this work? Yes. But like most initial ideas, it was not really the best idea. The largest drawback to this that I acknowledge now is that it not only makes the code significantly more messy, but there is now an obligation to set/unset a boolean flag in the midst of most of the operations. Keeping track of this is annoying at best, and can lead to many bugs at worst. My more intuitive solution now would be just to choose another symbol to represent a player standing on storage. For example, rather than a player on a storage space being represented by an @ internally and a `_standingOnStorage` boolean flag set to true, it could all just be represented within the internal matrix as a ! symbol (or really any character/symbol). This removes the need for excessive boolean flags, keeping the code neat and simple.

Similarly, the need for each helper function to have an X and a Y version is extremely redundant. If nothing else, a check could be performed within one helper function to determine which direction is being referred to. As an extension to this, the enumerated direction being moved in could be passed to the helper, which uses it to determine if the `offset` integer variable is in terms of X or Y.

5.3.2 What I Learned

As I discussed in the previous section 5.3.1, I realized to a certain degree that less is more. Of course, this is far from a novel idea, but I think it's significant since learning to code can be daunting due to the nature of some of the concepts/problems you may encounter. Learning new things is challenging, so trying to *fix* something you still aren't comfortable with can be a nightmare. With this in mind, I started to realize around this point in the semester that it's easy to get caught up in the details of coding, such that when you have finally fixed an issue you've encountered, you may be left with an even bigger mess than you started with (albeit, a functional one, but messy code is sometimes just as bad as broken code). I really tried to keep this in mind as I worked on more and more projects later in the semester, since I could save a lot of time by just working out simple and neat solutions, rather than technically sufficient but horrid code. Debugging 4 helper functions, for instance, would be way easier than trying to debug 8 which are split by subtle X/Y differences.

5.3.3 The Takeaway

I really enjoyed this assignment since it was a fun way to approach C++ coding that I had never really experienced before. I had mostly only really worked on simple console applications, or really challenging mathematically-focused programs. It was great to not only have something that wasn't a massive headache, but to have an end product that feels fun to share afterwards. I found the process to be much more intuitive, and I can certainly accredit some of this to the fact that it was a game and I had played a lot of games growing up, but I will attribute some of it to the fact that I really felt like contextualizing a program helped me learn a lot. I also think, by extension to this, that the realization that *yes* this can certainly be very challenging, but I *am* learning a lot whether I realize it or not, was very reassuring. I started to gain some satisfaction with the programs I was creating around this point in the semester, which is something I had really not let myself have at all prior to this. Although I knew there was an incredibly far amount left for me to grow, it felt motivating knowing that I was progressing as a programmer, at least to some acknowledgeable level.

5.4 Running The Program

The Sokoban game can be compiled using the Makefile provided and the *make* command. This will provide the executable file **Sokoban** and the static library **Sokoban.a**. You can then use the following format to run the game:

```
1 ./Sokoban <level-filename>
```

The only parameter than must be supplied to the command line is the name of the level that the user wishes to play. Running the program with this format will automatically open the level and load the game, allowing one to simply get right into the action.

5.5 Proof of Functionality

Figure 10: Snippits of gameplay within the custom level I made



(a) Default starting position, only one box and one storage location in this level



(b) Oh man... (Player will have to restart or undo some moves to get out of here)



(c) Hooray!!!

5.6 Source Code

The Sokoban program was comprised of a set of **Sokoban.hpp** and **Sokoban.cpp** files containing the **Sokoban** class, the **SB** namespace, and the custom **Direction** enumeration. The main gameplay loop is found within the **main.cpp** text file. As always, a **Makefile** accompanies the program as well. Below is the source code for all of the mentioned files

Makefile

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -lboost_unit_test_framework
4 # Your .hpp files
5 DEPS = Sokoban.hpp
6 # Your compiled .o files
7 OBJECTS = Sokoban.o
8 # The name of your programs
9 PROGRAM = Sokoban test
10
11 .PHONY: all clean lint
12
13 all: $(PROGRAM) Sokoban.a
14
15 # Wildcard recipe to make .o files from corresponding .cpp file
```

```

16 %.o: %.cpp $(DEPS)
17     $(CC) $(CFLAGS) -c $<
18
19 Sokoban: main.o $(OBJECTS)
20     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
21
22 test: test.o $(OBJECTS)
23     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
24
25 Sokoban.a: $(OBJECTS)
26     ar rcs $@ $^
27
28 clean:
29     rm *.o Sokoban test Sokoban.a
30 lint:
31     cpplint *.cpp *.hpp

```

main.cpp

```

1  // Copyright 2024 Nathan Khoury
2  #include <iostream>
3  #include <fstream>
4
5  #include <SFML/Window.hpp>
6  #include <SFML/Audio.hpp>
7
8  #include "Sokoban.hpp"
9
10 int main(int argc, char *argv[]) {
11     // Check if sufficient arguments were supplied
12     if (argc < 2) {
13         std::cerr << "must supply level to load, returning 1"
14             << std::endl;
15         return 1;
16     }
17     std::string lvl = argv[1];
18
19     // Create Sokoban object
20     SB::Sokoban sb;
21
22     // Read level into internal storage
23     std::ifstream f;
24     f.open(lvl);
25     f >> sb;
26     // Print level to terminal
27     // std::cout << sb;
28
29     f.close();
30
31     // Start Sokoban Game
32     sf::RenderWindow window(sf::VideoMode(sb.width() * 64, sb.height() * 64),
33         "Sokoban game, playing level: " + lvl);
34     window.setPosition({0, 0});
35     window.setFramerateLimit(30);
36
37     // load font for victory text
38     sf::Font font;
39     font.loadFromFile("arial.ttf");
40     sf::Text victoryText("Victory!!!", font);
41     victoryText.setOrigin(victoryText.getLocalBounds().width / 2,
42         victoryText.getLocalBounds().height / 2); // set origin to center of text
43     victoryText.setPosition(sb.width() * 32, sb.height() * 32);
44
45     while (window.isOpen()) {
46         sf::Event event;
47         bool gameWon = sb.isWon();
48         while (window.pollEvent(event)) {

```

```

49         if (event.type == sf::Event::Closed)
50             window.close();
51         if (event.type == sf::Event::KeyPressed) {
52             switch (event.key.code) {
53                 case sf::Keyboard::Up:
54                 case sf::Keyboard::W:
55                     // Move up if W or Up Arrow pressed
56                     sb.movePlayer(SB::Up);
57                     break;
58                 case sf::Keyboard::Down:
59                 case sf::Keyboard::S:
60                     // Move up if W or Up Arrow pressed
61                     sb.movePlayer(SB::Down);
62                     break;
63                 case sf::Keyboard::Left:
64                 case sf::Keyboard::A:
65                     // Move up if W or Up Arrow pressed
66                     sb.movePlayer(SB::Left);
67                     break;
68                 case sf::Keyboard::Right:
69                 case sf::Keyboard::D:
70                     // Move up if W or Up Arrow pressed
71                     sb.movePlayer(SB::Right);
72                     break;
73                 case sf::Keyboard::R:
74                     // Reset game
75                     f.open(lvl);
76                     f >> sb;
77                     f.close();
78                     sb.resetUndo(); // reset undo deque
79                     break;
80                 case sf::Keyboard::Q:
81                     // Undo move
82                     sb.undo();
83                     break;
84                 default: break;
85             }
86         }
87         window.clear();
88         window.draw(sb);
89         if (gameWon) { window.draw(victoryText); }
90         window.display();
91     }
92 }
93 return 0;
94 }

```

Sokoban.hpp

```

1 // Copyright 2024 Nathan Khoury
2 #ifndef SOKOBAN_H
3 #define SOKOBAN_H
4
5 #include <iostream>
6 #include <string>
7 #include <vector>
8 #include <deque>
9
10 #include <SFML/Graphics.hpp>
11
12 namespace SB {
13     typedef enum Direction {Up, Down, Left, Right} Direction;
14
15     class Sokoban : public sf::Drawable {
16     public:
17         // default constructor
18         Sokoban();

```

```

19 // overloaded extraction operator to read level into internal storage
20 friend std::istream& operator>>(std::istream& lvl, Sokoban& sb);
21 // overloaded insertion operator to print level to ostream
22 friend std::ostream& operator<<(std::ostream& os, Sokoban& sb);
23 // return height
24 size_t width() const { return _width; }
25 // return width
26 size_t height() const { return _height; }
27 // determine if game has been won
28 bool isWon() const;
29 // give player location as a sf::Vector2i
30 sf::Vector2i playerLoc() const;
31 // move the player according to the direction provided;
32 void movePlayer(Direction d);
33 // undo up to 5 moves (BONUS)
34 void undo();
35 // clear undo buffer (BONUS)
36 void resetUndo();
37
38 protected:
39     void draw(sf::RenderTarget& window, sf::RenderStates states) const override;
40
41 private:
42     size_t _height;
43     size_t _width;
44     std::vector<char> _data;
45     std::deque<std::vector<char>> _moveHistory; // last 5 moves
46     std::deque<sf::Texture> _orientationHistory; // direction facing on last 5 moves
47     std::deque<bool> _storageHistory; // was player on storage during those moves?
48     bool _standingOnStorage;
49
50     bool isOutOfBoundsY(sf::Vector2i p, int offset);
51     bool isOutOfBoundsX(sf::Vector2i p, int offset);
52     bool isWallY(sf::Vector2i p, int offset);
53     bool isWallX(sf::Vector2i p, int offset);
54     bool isBoxY(sf::Vector2i p, int offset);
55     bool isBoxX(sf::Vector2i p, int offset);
56     bool isStorageLocY(sf::Vector2i p, int offset);
57     bool isStorageLocX(sf::Vector2i p, int offset);
58
59     // textures
60     sf::Texture _wall;
61     sf::Texture _box;
62     sf::Texture _groundEmpty;
63     sf::Texture _groundStorage;
64     sf::Texture _playerDown;
65     sf::Texture _playerUp;
66     sf::Texture _playerLeft;
67     sf::Texture _playerRight;
68
69     sf::Texture _playerOrientation;
70 };
71
72     std::istream& operator>>(std::istream& is, Sokoban& sb);
73
74     std::ostream& operator<<(std::ostream& os, Sokoban& sb);
75 } // namespace SB
76 #endif

```

Sokoban.cpp

```

1 // Copyright 2024 Nathan Khoury
2 #include <fstream>
3 #include <iostream>
4 #include <algorithm>
5
6 #include "Sokoban.hpp"

```

```

7
8 SB::Sokoban::Sokoban() {
9     _wall.loadFromFile("block_06.png");
10    _box.loadFromFile("crate_03.png");
11    _groundEmpty.loadFromFile("ground_01.png");
12    _groundStorage.loadFromFile("ground_04.png");
13    _playerDown.loadFromFile("player_05.png");
14    _playerUp.loadFromFile("player_08.png");
15    _playerLeft.loadFromFile("player_20.png");
16    _playerRight.loadFromFile("player_17.png");
17
18    _playerOrientation = _playerDown;
19    _standingOnStorage = false;
20 }
21
22 std::istream& SB::operator>>(std::istream& lvl, SB::Sokoban& sb) {
23     // read height and width from top of file
24     lvl >> sb._height;
25     lvl >> sb._width;
26
27     // create space in data for the game board
28     sb._data.resize(sb._width * sb._height);
29
30     // player gets starting values (in case of restart)
31     sb._standingOnStorage = false;
32     sb._playerOrientation = sb._playerDown;
33
34     // store symbols in game board
35     std::string str;
36     size_t i = 0;
37     while (lvl) {
38         std::getline(lvl, str);
39         for (size_t j = 0; j < str.size(); j++) {
40             if (str.at(j) == ' ' || str.at(j) == '\n') {
41                 continue;
42             } else {
43                 sb._data.at(i) = str.at(j);
44                 i++;
45             }
46         }
47     }
48     return lvl;
49 }
50
51 std::ostream& SB::operator<<(std::ostream& os, SB::Sokoban& sb) {
52     os << sb._height << " ";
53     os << sb._width;
54     for (size_t y = 0; y < sb._height; y++) {
55         os << std::endl;
56         for (size_t x = 0; x < sb._width; x++) {
57             os << sb._data.at(x + (y * sb._width));
58         }
59         std::flush(os);
60     }
61     os << std::endl;
62     return os;
63 }
64
65 bool SB::Sokoban::isWon() const {
66     auto boxIterator = std::find_if(_data.begin(), _data.end(),
67         [](char x) { return x == 'A'; });
68     if (boxIterator == _data.end())
69         return true;    // win if there are no more boxes
70
71     auto storageIterator = std::find_if(_data.begin(), _data.end(),
72         [](char x) { return x == 'a'; });

```

```

73     if (storageIterator == _data.end() && !_standingOnStorage)
74         return true;    // win if there are no more storage areas
75
76     // otherwise the game has not been won
77     // there is still a box and a storage area to be matched
78     return false;
79 }
80
81 sf::Vector2i SB::Sokoban::playerLoc() const {
82     sf::Vector2i p;
83     for (size_t y = 0; y < _height; y++) {
84         for (size_t x = 0; x < _width; x++) {
85             if (_data.at(x + (y * _width)) == '@') {
86                 p.x = x;
87                 p.y = y;
88                 return p;
89             }
90         }
91     }
92     std::cerr << "Couldn't find player, returning (-1, -1) for playerLoc()" << std::endl;
93     p.x = -1;
94     p.y = -1;
95     return p;
96 }
97
98 void SB::Sokoban::movePlayer(SB::Direction d) {
99     if (isWon())
100         return;
101     if (d == SB::Up || d == SB::Down) {    // IF MOVING UP OR DOWN
102         if (_moveHistory.size() >= 5) {
103             // FOR UNDOING PLAYER MOVES
104             // more than 5 moves in storage, pop oldest
105             _moveHistory.pop_back();
106             _moveHistory.push_front(_data);
107             _orientationHistory.pop_back();
108             // add current state to history
109             _orientationHistory.push_front(_playerOrientation);
110             _storageHistory.pop_back();
111             _storageHistory.push_front(_standingOnStorage);
112         } else {
113             // add current state to history
114             _moveHistory.push_front(_data);
115             _orientationHistory.push_front(_playerOrientation);
116             _storageHistory.push_front(_standingOnStorage);
117         }
118
119         int offset;
120         // decide direction to be moved (up/down)
121         if (d == Up) {
122             offset = -1;
123             _playerOrientation = _playerUp;
124         } else {
125             offset = 1;
126             _playerOrientation = _playerDown;
127         }
128
129         // get current player location
130         sf::Vector2i p = playerLoc();
131         if (isOutOfBoundsY(p, offset)) {
132             // if out of bounds, don't move
133             return;
134         } else if (isWallY(p, offset)) {
135             // if a wall above/below, don't move
136             return;
137         } else if (isBoxY(p, offset)) {
138             // try to push box

```

```

139         if (isOutOfBoundsY(p, offset + offset) || isWallY(p, offset + offset)
140             || isBoxY(p, offset + offset)) {
141             // if there is a wall/box blocking the box, or it would be out of bounds,
142             // then do not move the box
143             return;
144         } else {
145             // push the box succesfully
146             if (isStorageLocY(p, offset + offset)) {
147                 // if pushed into a storage location, mark as '1'
148                 _data.at(p.x + ((p.y + offset * 2) * _width)) = '1';
149             } else {
150                 // otherwise mark as just 'A'
151                 _data.at(p.x + ((p.y + offset * 2) * _width)) = 'A';
152             }
153             if (_standingOnStorage) {
154                 // if player was standing on storage spot, be sure to replace it
155                 _data.at(p.x + (p.y * _width)) = 'a';
156             } else {
157                 // if not storage, has to be empty space
158                 _data.at(p.x + (p.y * _width)) = '.';
159             }
160             if (isStorageLocY(p, offset)) {
161                 // track if player just moved on a storage spot
162                 _standingOnStorage = true;
163             } else {
164                 _standingOnStorage = false;
165             }
166             _data.at(p.x + ((p.y + offset) * _width)) = '@';
167         }
168     } else {
169         // if nothing else, just move one step
170         if (_standingOnStorage) {
171             // if player was standing on storage spot, be sure to replace it
172             _data.at(p.x + (p.y * _width)) = 'a';
173         } else {
174             // if not storage, has to be empty space
175             _data.at(p.x + (p.y * _width)) = '.';
176         }
177         if (isStorageLocY(p, offset)) {
178             // track if player just moved on a storage spot
179             _standingOnStorage = true;
180         } else {
181             _standingOnStorage = false;
182         }
183         _data.at(p.x + ((p.y + offset) * _width)) = '@';
184     }
185 } else if (d == SB::Left || d == SB::Right) { // IF MOVING LEFT OR RIGHT
186     if (_moveHistory.size() >= 5) {
187         // FOR UNDOING PLAYER MOVES
188         // more than 5 moves in storage, pop oldest
189         _moveHistory.pop_back();
190         _moveHistory.push_front(_data);
191         _orientationHistory.pop_back();
192         // add current state to history
193         _orientationHistory.push_front(_playerOrientation);
194         _storageHistory.pop_back();
195         _storageHistory.push_front(_standingOnStorage);
196     } else {
197         // add current state to history
198         _moveHistory.push_front(_data);
199         _orientationHistory.push_front(_playerOrientation);
200         _storageHistory.push_front(_standingOnStorage);
201     }
202     int offset;
203     // decide direction to be moved (left/right)
204     if (d == Left) {

```

```

205         offset = -1;
206         _playerOrientation = _playerLeft;
207     } else {
208         offset = 1;
209         _playerOrientation = _playerRight;
210     }
211
212     // get current player location
213     sf::Vector2i p = playerLoc();
214     if (isOutOfBoundsX(p, offset)) {
215         // if out of bounds, don't move
216         return;
217     } else if (isWallX(p, offset)) {
218         // if a wall left/right, don't move
219         return;
220     } else if (isBoxX(p, offset)) {
221         // try to push box
222         if (isOutOfBoundsX(p, offset + offset) || isWallX(p, offset + offset)
223             || isBoxX(p, offset + offset)) {
224             // if there is a wall/box blocking the box, or it would be out of bounds,
225             // then do not move the box
226             return;
227         } else {
228             // push the box succesfully
229             if (isStorageLocX(p, offset + offset)) {
230                 // if pushed into a storage location, mark as '1'
231                 _data.at((p.x + offset * 2) + (p.y * _width)) = '1';
232             } else {
233                 // otherwise mark as just 'A'
234                 _data.at((p.x + offset * 2) + (p.y * _width)) = 'A';
235             }
236             if (_standingOnStorage) {
237                 // if player was standing on storage spot, be sure to replace it
238                 _data.at(p.x + (p.y * _width)) = 'a';
239             } else {
240                 // if not storage, has to be empty space
241                 _data.at(p.x + (p.y * _width)) = '.';
242             }
243             if (isStorageLocX(p, offset)) {
244                 // track if player just moved on a storage spot
245                 _standingOnStorage = true;
246             } else {
247                 _standingOnStorage = false;
248             }
249             _data.at((p.x + offset) + (p.y * _width)) = '@';
250         }
251     } else {
252         if (_standingOnStorage) {
253             // if player was standing on storage spot, be sure to replace it
254             _data.at(p.x + (p.y * _width)) = 'a';
255         } else {
256             // if not storage, has to be empty space
257             _data.at(p.x + (p.y * _width)) = '.';
258         }
259         if (isStorageLocX(p, offset)) {
260             // track if player just moved on a storage spot
261             _standingOnStorage = true;
262         } else {
263             _standingOnStorage = false;
264         }
265         _data.at((p.x + offset) + (p.y * _width)) = '@';
266     }
267 } else {
268     std::cerr << "Bad input for movePlayer()" << std::endl;
269     return;
270 }

```

```

271 }
272
273 bool SB::Sokoban::isOutOfBoundsY(sf::Vector2i p, int offset) {
274     return ((p.y + offset) * _width < 0 || (p.y + offset) * _width >= _data.size());
275 }
276
277 bool SB::Sokoban::isOutOfBoundsX(sf::Vector2i p, int offset) {
278     return (p.x + offset) < 0 || (p.x + offset) >= static_cast<int>(_width);
279 }
280
281 bool SB::Sokoban::isWallY(sf::Vector2i p, int offset) {
282     return (_data.at(p.x + ((p.y + offset) * _width)) == '#');
283 }
284
285 bool SB::Sokoban::isWallX(sf::Vector2i p, int offset) {
286     return (_data.at((p.x + offset) + (p.y * _width)) == '#');
287 }
288
289 bool SB::Sokoban::isBoxY(sf::Vector2i p, int offset) {
290     if ((_data.at(p.x + ((p.y + offset) * _width)) == 'A')           // if box
291         || (_data.at(p.x + ((p.y + offset) * _width)) == '1'))       // or if box in storage
292         return true;
293     return false;
294 }
295
296 bool SB::Sokoban::isBoxX(sf::Vector2i p, int offset) {
297     if ((_data.at((p.x + offset) + (p.y * _width)) == 'A')           // if box
298         || (_data.at((p.x + offset) + (p.y * _width)) == '1'))       // or if box in storage
299         return true;
300     return false;
301 }
302
303 bool SB::Sokoban::isStorageLocY(sf::Vector2i p, int offset) {
304     if ((_data.at(p.x + ((p.y + offset) * _width)) == 'a')           // if storage location
305         || (_data.at(p.x + ((p.y + offset) * _width)) == '1'))       // or if box in storage
306         return true;
307     return false;
308 }
309
310 bool SB::Sokoban::isStorageLocX(sf::Vector2i p, int offset) {
311     if ((_data.at((p.x + offset) + (p.y * _width)) == 'a')           // if storage location
312         || (_data.at((p.x + offset) + (p.y * _width)) == '1'))       // or if box in storage
313         return true;
314     return false;
315 }
316
317 void SB::Sokoban::draw(sf::RenderTarget& window, sf::RenderStates states) const {
318     for (size_t y = 0; y < _height; y++) {
319         for (size_t x = 0; x < _width; x++) {
320             // load current character within internal grid
321             char ch = _data.at(x + (y * _width));
322             sf::Sprite tile;
323             sf::Sprite backdrop(_groundEmpty);
324             bool hasBackdrop = false;
325             // determine which texture to load
326             switch (ch) {
327                 case '#':
328                     tile.setTexture(_wall);
329                     break;
330                 case '.':
331                     tile.setTexture(_groundEmpty);
332                     break;
333                 case '@':
334                     if (_standingOnStorage)
335                         backdrop.setTexture(_groundStorage);
336                     tile.setTexture(_playerOrientation);

```

```

337         hasBackdrop = true;
338         break;
339     case 'A':
340         tile.setTexture(_box);
341         hasBackdrop = true;
342         break;
343     case 'a':
344         tile.setTexture(_groundStorage);
345         break;
346     case '1':
347         tile.setTexture(_box);
348         backdrop.setTexture(_groundStorage);
349         hasBackdrop = true;
350         break;
351     default:
352         break;
353 }
354 if (hasBackdrop) {
355     backdrop.setPosition(x * 64, y * 64);
356     window.draw(backdrop);
357 }
358 tile.setPosition(x * 64, y * 64);
359 window.draw(tile, states);
360 }
361 }
362 }
363
364 void SB::Sokoban::undo() {
365     if (_moveHistory.size() <= 0)
366         return; // no more game states to restore
367     _data = _moveHistory.front();
368     _moveHistory.pop_front();
369     // restore direction that player was facing
370     _playerOrientation = _orientationHistory.front();
371     _orientationHistory.pop_front();
372     // be sure to remember storage locations
373     _standingOnStorage = _storageHistory.front();
374     _storageHistory.pop_front();
375 }
376
377 void SB::Sokoban::resetUndo() {
378     _moveHistory.erase(_moveHistory.begin(), _moveHistory.end());
379     _orientationHistory.erase(_orientationHistory.begin(), _orientationHistory.end());
380     _storageHistory.erase(_storageHistory.begin(), _storageHistory.end());
381 }

```

5.7 Unit Testing

Unit Testing for the Sokoban game was not conceptually very tough, although it definitely needed to be *very* thorough to cover all of the edge cases (although, this sort of goes without saying, since that *is* the point of unit testing). The tests verify that proper matrix size is initialized, regular win conditions are met, automatic win conditions on game startup are met, normal player movement is allowed, and that restricted player movement is handled appropriately. Stranger cases like out of bounds movement and the ability to move wall tiles need to be verified as well, as well as double box movement. Finally, the most obscure case that needed to be ascertained was the trampling of storage locations. Below, the `test.cpp` file is presented with the implementations of all of the necessary tests.

```

1 // Copyright 2024 Nathan Khoury
2 #include <fstream>
3 #include "Sokoban.hpp"
4
5 #define BOOST_TEST_DYN_LINK
6 #define BOOST_TEST_MODULE Sokoban_Testing
7 #include <boost/test/unit_test.hpp>
8
9 using SB::Sokoban;
10 using SB::Direction;
11
12 BOOST_AUTO_TEST_CASE(initHeightAndWidth1) {
13     Sokoban sb;
14     std::ifstream f;
15     f.open("level1.lvl");
16     f >> sb;
17
18     BOOST_REQUIRE_EQUAL(sb.height(), 10);
19     BOOST_REQUIRE_EQUAL(sb.width(), 10);
20 }
21
22 BOOST_AUTO_TEST_CASE(initHeightAndWidth2) {
23     Sokoban sb;
24     std::ifstream f;
25     f.open("level2.lvl");
26     f >> sb;
27
28     BOOST_REQUIRE_EQUAL(sb.height(), 10);
29     BOOST_REQUIRE_EQUAL(sb.width(), 12);
30 }
31
32 BOOST_AUTO_TEST_CASE(initHeightAndWidth3) {
33     Sokoban sb;
34     std::ifstream f;
35     f.open("level3.lvl");
36     f >> sb;
37
38     BOOST_REQUIRE_EQUAL(sb.height(), 12);
39     BOOST_REQUIRE_EQUAL(sb.width(), 10);
40 }
41
42 BOOST_AUTO_TEST_CASE(autowin1) {
43     Sokoban sb;
44     std::ifstream f;
45     f.open("autowin.lvl");
46     f >> sb;
47
48     BOOST_REQUIRE_EQUAL(sb.isWon(), true);
49 }
50
51 BOOST_AUTO_TEST_CASE(autowin2) {
52     Sokoban sb;
53     std::ifstream f;
54     f.open("autowin2.lvl");
55     f >> sb;
56
57     BOOST_REQUIRE_EQUAL(sb.isWon(), true);
58 }
59
60 BOOST_AUTO_TEST_CASE(autowin3) {
61     Sokoban sb;
62     std::ifstream f;
63     f.open("autowin3.lvl");
64     f >> sb;
65
66     BOOST_REQUIRE_EQUAL(sb.isWon(), true);

```

```

67 }
68
69 BOOST_AUTO_TEST_CASE(autowin4) {
70     Sokoban sb;
71     std::ifstream f;
72     f.open("autowin4.lv1");
73     f >> sb;
74
75     BOOST_REQUIRE_EQUAL(sb.isWon(), true);
76 }
77
78 BOOST_AUTO_TEST_CASE(testPlayerLoc1) {
79     Sokoban sb;
80     std::ifstream f;
81     f.open("pushup.lv1");
82     f >> sb;
83
84     sf::Vector2i expectedLoc({2, 2});
85
86     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
87     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
88 }
89
90 BOOST_AUTO_TEST_CASE(testPlayerLoc2) {
91     Sokoban sb;
92     std::ifstream f;
93     f.open("level3.lv1");
94     f >> sb;
95
96     sf::Vector2i expectedLoc({3, 8});
97
98     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
99     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
100 }
101
102 BOOST_AUTO_TEST_CASE(pushUpWin) {
103     Sokoban sb;
104     std::ifstream f;
105     f.open("pushup.lv1");
106     f >> sb;
107
108     sf::Vector2i expectedLoc({2, 1});
109     sb.movePlayer(SB::Direction::Up);
110
111     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
112     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
113     BOOST_REQUIRE_EQUAL(sb.isWon(), true);
114 }
115
116 BOOST_AUTO_TEST_CASE(pushDownWin) {
117     Sokoban sb;
118     std::ifstream f;
119     f.open("pushdown.lv1");
120     f >> sb;
121
122     sf::Vector2i expectedLoc({2, 3});
123     sb.movePlayer(SB::Direction::Down);
124
125     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
126     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
127     BOOST_REQUIRE_EQUAL(sb.isWon(), true);
128 }
129
130 BOOST_AUTO_TEST_CASE(pushLeftWin) {
131     Sokoban sb;
132     std::ifstream f;

```

```

133     f.open("pushleft.lvl");
134     f >> sb;
135
136     sf::Vector2i expectedLoc({1, 2});
137     sb.movePlayer(SB::Direction::Left);
138
139     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
140     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
141     BOOST_REQUIRE_EQUAL(sb.isWon(), true);
142 }
143
144 BOOST_AUTO_TEST_CASE(pushRightWin) {
145     Sokoban sb;
146     std::ifstream f;
147     f.open("pushright.lvl");
148     f >> sb;
149
150     sf::Vector2i expectedLoc({3, 2});
151     sb.movePlayer(SB::Direction::Right);
152
153     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
154     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
155     BOOST_REQUIRE_EQUAL(sb.isWon(), true);
156 }
157
158 BOOST_AUTO_TEST_CASE(outOfBoundsNoWallsUp) {
159     Sokoban sb;
160     std::ifstream f;
161     // nowalls00B.lvl (out of bounds) is NOT winnable
162     // I made the level just for a small region to run into the walls of
163     f.open("testnowalls00B.lvl");
164     f >> sb;
165
166     sf::Vector2i expectedLoc({1, 0});
167     for (size_t i = 0; i < 10; i++)
168         sb.movePlayer(SB::Direction::Up);
169
170     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
171     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
172     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
173 }
174
175 BOOST_AUTO_TEST_CASE(outOfBoundsNoWallsDown) {
176     Sokoban sb;
177     std::ifstream f;
178     // nowalls00B.lvl (out of bounds) is NOT winnable
179     // I made the level just for a small region to run into the walls of
180     f.open("testnowalls00B.lvl");
181     f >> sb;
182
183     sf::Vector2i expectedLoc({1, 2});
184     for (size_t i = 0; i < 10; i++)
185         sb.movePlayer(SB::Direction::Down);
186
187     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
188     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
189     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
190 }
191
192 BOOST_AUTO_TEST_CASE(outOfBoundsNoWallsLeft) {
193     Sokoban sb;
194     std::ifstream f;
195     // nowalls00B.lvl (out of bounds) is NOT winnable
196     // I made the level just for a small region to run into the walls of
197     f.open("testnowalls00B.lvl");
198     f >> sb;

```

```

199
200     sf::Vector2i expectedLoc({0, 1});
201     for (size_t i = 0; i < 10; i++)
202         sb.movePlayer(SB::Direction::Left);
203
204     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
205     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
206     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
207 }
208
209 BOOST_AUTO_TEST_CASE(outOfBoundsNoWallsRight) {
210     Sokoban sb;
211     std::ifstream f;
212     // nowalls00B.lv1 (out of bounds) is NOT winnable
213     // I made the level just for a small region to run into the walls of
214     f.open("testnowalls00B.lv1");
215     f >> sb;
216
217     sf::Vector2i expectedLoc({2, 1});
218     for (size_t i = 0; i < 10; i++)
219         sb.movePlayer(SB::Direction::Right);
220
221     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
222     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
223     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
224 }
225
226 BOOST_AUTO_TEST_CASE(outOfBoundsWithWallsUp) {
227     Sokoban sb;
228     std::ifstream f;
229     // testwalls00B.lv1 (out of bounds) is, similarly, NOT WINNABLE
230     f.open("testwalls00B.lv1");
231     f >> sb;
232
233     sf::Vector2i expectedLoc({2, 2});
234     for (size_t i = 0; i < 10; i++)
235         sb.movePlayer(SB::Direction::Up);
236
237     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
238     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
239     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
240 }
241
242 BOOST_AUTO_TEST_CASE(outOfBoundsWithWallsDown) {
243     Sokoban sb;
244     std::ifstream f;
245     // testwalls00B.lv1 (out of bounds) is, similarly, NOT WINNABLE
246     f.open("testwalls00B.lv1");
247     f >> sb;
248
249     sf::Vector2i expectedLoc({2, 2});
250     for (size_t i = 0; i < 10; i++)
251         sb.movePlayer(SB::Direction::Down);
252
253     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
254     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
255     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
256 }
257
258 BOOST_AUTO_TEST_CASE(outOfBoundsWithWallsLeft) {
259     Sokoban sb;
260     std::ifstream f;
261     // testwalls00B.lv1 (out of bounds) is, similarly, NOT WINNABLE
262     f.open("testwalls00B.lv1");
263     f >> sb;
264

```

```

265     sf::Vector2i expectedLoc({2, 2});
266     for (size_t i = 0; i < 10; i++)
267         sb.movePlayer(SB::Direction::Left);
268
269     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
270     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
271     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
272 }
273
274 BOOST_AUTO_TEST_CASE(outOfBoundsWithWallsRight) {
275     Sokoban sb;
276     std::ifstream f;
277     // testwalls00B.lvl (out of bounds) is, similarly, NOT WINNABLE
278     f.open("testwalls00B.lvl");
279     f >> sb;
280
281     sf::Vector2i expectedLoc({2, 2});
282     for (size_t i = 0; i < 10; i++)
283         sb.movePlayer(SB::Direction::Right);
284
285     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
286     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
287     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
288 }
289
290 BOOST_AUTO_TEST_CASE(pushDoubleBoxUp) {
291     Sokoban sb;
292     std::ifstream f;
293     // testdoublebox.lvl is NOT WINNABLE and serves the same purpose
294     // as testnowalls00B.lvl and testwalls00B.lvl
295     f.open("testdoublebox.lvl");
296     f >> sb;
297
298     sf::Vector2i expectedLoc({3, 3});
299     for (size_t i = 0; i < 10; i++)
300         sb.movePlayer(SB::Direction::Up);
301
302     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
303     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
304     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
305 }
306
307 BOOST_AUTO_TEST_CASE(pushDoubleBoxDown) {
308     Sokoban sb;
309     std::ifstream f;
310     // testdoublebox.lvl is NOT WINNABLE and serves the same purpose
311     // as testnowalls00B.lvl and testwalls00B.lvl
312     f.open("testdoublebox.lvl");
313     f >> sb;
314
315     sf::Vector2i expectedLoc({3, 3});
316     for (size_t i = 0; i < 10; i++)
317         sb.movePlayer(SB::Direction::Down);
318
319     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
320     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
321     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
322 }
323
324 BOOST_AUTO_TEST_CASE(pushDoubleBoxLeft) {
325     Sokoban sb;
326     std::ifstream f;
327     // testdoublebox.lvl is NOT WINNABLE and serves the same purpose
328     // as testnowalls00B.lvl and testwalls00B.lvl
329     f.open("testdoublebox.lvl");
330     f >> sb;

```

```

331
332     sf::Vector2i expectedLoc({3, 3});
333     for (size_t i = 0; i < 10; i++)
334         sb.movePlayer(SB::Direction::Left);
335
336     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
337     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
338     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
339 }
340
341 BOOST_AUTO_TEST_CASE(pushDoubleBoxRight) {
342     Sokoban sb;
343     std::ifstream f;
344     // testdoublebox.lvl is NOT WINNABLE and serves the same purpose
345     // as testnowalls00B.lvl and testwalls00B.lvl
346     f.open("testdoublebox.lvl");
347     f >> sb;
348
349     sf::Vector2i expectedLoc({3, 3});
350     for (size_t i = 0; i < 10; i++)
351         sb.movePlayer(SB::Direction::Right);
352
353     BOOST_REQUIRE_EQUAL(sb.playerLoc().x, expectedLoc.x);
354     BOOST_REQUIRE_EQUAL(sb.playerLoc().y, expectedLoc.y);
355     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
356 }
357
358 BOOST_AUTO_TEST_CASE(manyBoxWin) {
359     // Test 10 boxes and 10 storage
360     Sokoban sb;
361     std::ifstream f;
362     f.open("testmanyboxes.lvl");
363     f >> sb;
364
365     for (size_t i = 0; i < 10; i++) {
366         BOOST_REQUIRE_EQUAL(sb.isWon(), false);
367         sb.movePlayer(SB::Direction::Right);
368         sb.movePlayer(SB::Direction::Left);
369         sb.movePlayer(SB::Direction::Down);
370     }
371     BOOST_REQUIRE_EQUAL(sb.isWon(), true);
372 }
373
374 BOOST_AUTO_TEST_CASE(swapoffWin) {
375     // Test 10 boxes and 10 storage
376     Sokoban sb;
377     std::ifstream f;
378     f.open("swapoff.lvl");
379     f >> sb;
380
381     sb.movePlayer(SB::Direction::Up);
382     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
383     sb.movePlayer(SB::Direction::Down);
384     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
385     sb.movePlayer(SB::Direction::Right);
386     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
387     sb.movePlayer(SB::Direction::Right);
388     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
389     sb.movePlayer(SB::Direction::Up);
390     BOOST_REQUIRE_EQUAL(sb.isWon(), false);
391     sb.movePlayer(SB::Direction::Left);
392     BOOST_REQUIRE_EQUAL(sb.isWon(), true);
393 }

```

5.8 Bonus

One of the bonus options that I completed was the functionality for the player sprite to change depending on which way the player most recently moved. For instance, if the player moved left, the player sprite would appear to face left, and would sustain this until the player gave another movement input. To do this, I simply had a private member variable `_playerOrientation` which is of type `sf::Texture` and would update it's texture each move according to what direction that the player attempted to move in. This could be improved by having a pointer to one of the internal player textures rather than copying the value each time.

The other bonus opportunity that I fulfilled was the ability to undo one or more moves. I decided to allow the player to undo up the five of the most recent moves, and I accomplished this by creating a private member variable `_moveHistory` which is a `std::deque<std::vector<char>>` container. The deque holds five of the most recent gamestates, which are obtained from the `_data` internal vector in that frame. Although storing multiple entire vectors is conceptually very expensive, the limited size of the game board due to the window size, as well as the fact that the vector only store characters, makes it somewhat reasonable in my opinion. Attentive readers, particularly those who viewed the `Sokoban.hpp` class header, would notice two more deque containers awkwardly present: `_orientationHistory` and `_storageHistory`. Due to the nature of some of my more lackluster design choices, two issues presented themselves when it came to implementing the undo history deque. The first of these was that the direction that the player was facing during that move was not preserved, which I felt was awkward when undoing moves. I solved this by having another deque track the textures alongside the game states. The second issue was that the undo action did not take into account if the player was standing on a storage space at the time of undoing, meaning that undoing while standing on a storage location would cause the player to move the storage location with themselves. I fixed this with yet *another* deque which tracked the boolean history of `_standingOnStorage`.

The abundance of deques not only causes clutter within the class header, but causes a massive headache when you look at the `SB::Sokoban::movePlayer()` function. The requirement to modify three separate deques is an absolute disaster. We can easily eliminate the `_storageHistory` deque by implementing the representation within the vector as was described in section 5.3.1, which arguable was the worse of the two. My initial thought now is that the remaining `_orientationHistory` deque could be condensed into the first, possibly by having the first deque become a `std::deque<std::pair<std::vector<char>, sf::Texture>>`, although this seems to have its own detriments.

6 PS4: N-Body Simulation

6.1 Functionality



Figure 11: The "planets.txt" simulation at startup. The sun and its four nearest planets are simulated in this instance

The N-Body Simulation program instructed students to implement the design for a physics simulation which accurately reflected the behavior of celestial bodies interacting with one another in space over a certain duration of time. As a user, one can provide redirected input to `stdin` from a file of the appropriate format, which the program will read and determine the number of particles that will be displayed in the simulation. Then, it will create objects of `CelestialBody` type with appropriate values corresponding to those sourced from the input and store them in an overarching `Universe` object. Afterwards, the `Universe` object will be used to run the simulation according to the time intervals specified by the user in the command line, as well as the time to stop the simulation at which is also provided by the user in the command line.

The basis of the mathematical calculations comes from our usage of the **leapfrog finite difference approximation scheme** which proves to be an effective way to approximate integration within our simulation. Rather than operating under infinite precision, as is the nature of integration, we instead use some value of Δt seconds which is given to the program by the user in the command line. This value of Δt will allow us to know what precision to calculate each step of the simulation at per frame, allowing the user to customize their balance of precision and runtime accordingly. The simulation iterates up to a value of T seconds, adding Δt to the running total of time, `timeElapsed` in seconds, until `timeElapsed = T`, at which point the simulation terminates and displays the final x position, y position, x velocity, y velocity, mass, and image file of each and every particle in the simulation to the terminal.

The following is a list of important mathematical/physics concepts that are employed in the calculations within the program, their implementations in the program will be detailed further in 6.2 if necessary.

- The **gravitational constant** G , which yields a value of $6.67 \times 10^{-11} Nm^2/kg^2$
- The **distance between two planets**, r , can be calculated using $r = \sqrt{\Delta x^2 + \Delta y^2}$ which is derived from the Pythagorean Theorem ($a^2 + b^2 = c^2$)
- **Force** F can be calculated by using the formula $F = \frac{Gm_1m_2}{r^2}$ where m_1 is the mass of the first planet, and m_2 is the mass of the second planet
- Force can also be broken into its x and y components by using the following formulas
 - The x component can be obtained using $F_x = F \frac{\Delta x}{r}$
 - The y component can be obtained using $F_y = F \frac{\Delta y}{r}$
- The principle of superposition allows us to determine the **net force** acting on a single particle is the total sum of *all* of the forces acting on it. This can further be dismantled into:
 - **Net force** on a particle in the x direction is determined by the **sum of all** F_x acting upon it

- **Net force** on a particle in the ***y*** direction is determined by the **sum of all F_y** acting upon it
- **Acceleration** in the x and y directions can be calculated by
 - $a_x = \frac{F_x}{m}$
 - $a_y = \frac{F_y}{m}$
- **Newton’s Laws of Motion** are also very significant since all aspects of the simulation obey them (via. NASA)
 - **Newton’s First Law:** An object at rest will remain at rest, and an object in motion will remain in motion, at a constant velocity until an unbalanced force acts upon it
 - **Newton’s Second Law:** Acceleration of an object depends on the mass of the object being considered, as well as the force that is applied (this is expressed in the above-mentioned acceleration formulas)
 - **Newton’s Third Law:** Every action yields an equal *and* opposite reaction

Proceed to the next section, section 6.2, to see how all of these concepts come together to allow for an accurate simulation of particles with respect to the laws of physics within our universe.

6.2 Features

Before taking a tour of the flow of the program, it is important to examine the classes involved. The implementation of the N-Body Simulation program utilizes one namespace **NB** which contains two classes: **Universe** and **CelestialBody**. **Universe** acts as a governing figure for the program, and contains all of the **CelestialBody** objects that are created in preparation of the simulation within a simple `std::vector<std::shared_ptr<CelestialBody>>` named **particles**. Part of the expectation for PS4 was that students would learn how to create and work with smart pointers, so I chose to utilize the `std::shared_ptr` type since my needs were simple, and it provided good automatic management of manually allocated memory. The **Universe** class most notably manages the flow of the program with a public **step()** method which recalculates all of the positions and velocities of the particles using the Δt provided by the user as the time interval to compute the changes with respect to (i.e. if the user provides a Δt of 100 seconds, this would mean that every frame the simulation would recalculate the new positions and velocities of all of the particles as if every frame were representative of 100 seconds elapsed. Note that regardless of the value that Δt is, the simulation always steps once every frame).

An object from the **CelestialBody** class represents a single particle from the input file. It has internal representations of the x and y position, x and y velocities, and the mass of the particle. It also features helpful functions like **distance()** which takes itself and another **CelestialBody** and calculates the distance between them, or **netFIncrement** which takes a `sf::Vector2<double>` and adds the x/y components of force to its internal running total of net force acting upon itself. Both the **Universe** and the **CelestialBody** classes feature overridden stream insertion (`<<`) and stream extraction (`>>`) operators. The **Universe** overridden extraction operator reads from the redirected **stdin** input and determines the number of particles that will be simulated, as well as the radius of the universe to be simulated. It then calls the **CelestialBody** stream extraction operator once for every particle that will be simulated. Within the **CelestialBody** stream extraction operator, it will read from the redirected **stdin** input the x position, y position, x velocity, y velocity, mass, and the image file to display *all in that order exactly*. It will then load the texture, and return. After all of the **CelestialBody** objects are created as `std::shared_ptr` objects and stored within the internal vector in the **Universe**, the `std::istream&` returns. The overridden stream insertion operator works similarly for both, but in reverse. The **Universe** stream insertion operator calls the **CelestialBody** stream insertion operator once for each particle that exists, and each **CelestialBody** will write back all of their internal fields in the exact same format they were read in, except with the updated current values.

With all of this in mind, let's begin a swift tour of the flow of the program. Command line input is verified, and once this is complete, the universe reads from the redirected input using the overridden stream extraction operator. After, the display window is created, the backdrop is loaded, and the main loop begins with the `timeElapsed` variable starting at 0. Just prior to the loop, the user's inputs for Δt and T are stored in constant double variables `dt` and `T` respectively. While `timeElapsed >= T`, the program will call `Universe::step()` with the sole argument `dt`. After `step()`, `timeElapsed` is incremented by `dt`, and the loop reiterates.

Now let's take a look at what is going on within `Universe::step()`. Each `CelestialBody` has a private member variable `_netF` which tracks the running total of the net force being applied to itself for all the calculations in that step. This variable is temporary and is used for the calculations, so the first ranged for loop within `Universe::step()` assures that `_netF` is set to 0 for *all* of the `CelestialBody` objects. Then a for loop begins, starting at `i = 1`, setting `currCB` to the `i - 1` `CelestialBody` within the internal vector, and then running a `std::for_each()` operation from the `<algorithm>` library. The `for_each()` operation starts `i` elements past the start of the vector, and iterates to the end, applying a custom lambda expression to each which takes the `currCB`, as well as the `CelestialBody` being iterated on by the `for_each()` algorithm, and calls a helper function `pairwise()` on them. The loop and algorithm above-mentioned are displayed below for better understanding.

```

1  // Calculate net force for each CelestialBody
2  for (size_t i = 1; i < particles.size(); i++) {
3      CelestialBody& currCB = *particles.at(i - 1);
4      std::for_each(particles.begin() + i, particles.end(),
5                  [&](auto pcb){ pairwise(currCB, *pcb); });
6  }
```

The `pairwise()` helper function calculates the distance between the two particles. Then, it calculates the force being applied to both of the particles. Due to Newton's third law of motion, the force being applied to both of the particles is equivalent in absolute value, but they are opposite in terms of signage. For this reason, the first numerical value calculated is representative of the *first* particle only. Afterwards, the x and y components of the force vector calculated are both multiplied by -1, which accurately yields the force being applied to the *second* particle.

The interaction between the for loop and the `for_each()` algorithm allows us to save a considerable amount of time by calculating the net force applied for every combination of two particles being simulated only once. After all of the `CelestialBody` objects contain an accurate representation of the net force acting upon them within their `_netF` variable, a final ranged for loop calls `CelestialBody::step()` on each `CelestialBody`.

Within `CelestialBody::step()`, the acceleration is calculated. The x value of the velocity vector is updated by performing $v_x' = v_x + \Delta t \times a_x$. The y value of the velocity vector is done similarly, with y values respectively. Then, the x position is recalculated by using $p_x' = p_x + \Delta t \times v_x'$, with the y value being calculated by replacing the x inputs with y inputs respectively. After this, the internal positions and velocities of all of the `CelestialBody` objects are fully updated, and `Universe::step()` is entirely complete.

Returning back to the main loop, it is now worth mentioning that just before anything relating to `Universe::step()` occurs, an overridden `draw()` function which `Universe` and `CelestialBody` inherit from `sf::Drawable` is called. The `Universe::draw()` function calls `CelestialBody::draw()` once for each particle that exists. `CelestialBody::draw()` functions by creating a `sf::Sprite` with the corresponding texture, setting the sprite origin to the center of the sprite, and then setting the position of the sprite to the appropriate position reflected by the inner position vector inside the `CelestialBody`. It must be noted that not only must a calculation be done to scale the position stored internally (which is stored in possibly very large scientific notation double values) down to a position which is in the correct place relative to the scale of the window, but another transformation must occur to translate the particle half the size of the display window to the right and down within the

window. This second transformation is required since the SFML coordinate plane origin lies in the top left of the display window, where the origin of the universe we are simulating is in the center of the display window. After the appropriate scaling and translation, the sprite is drawn, which allows the user to see the beautiful universe they are simulating!

It is worth quickly mentioning at the end here that this program implements a design pattern known as **Model-View-Controller** *MVC*. The MVC design pattern is split into three distinct parts:

Model manages the internal representation of the data being simulated. In this program, that would be the `Universe` class which manages all of the `CelestialBody` objects and updates their positions and velocities according to the physics we would like to apply.

View manages how the data is presented to the universe. This would be the SFML window in this project, which works cooperatively with the overridden `draw()` functions that the two classes provide.

Controller is able to respond to user input and governs the intervals at which the model will update. This is found within the main looping condition, and the parameters that the user specifies.

6.3 Personal Reflection

6.3.1 The Struggles

I had some difficulties implementing the `std::for_each()` algorithm in the loop within `Universe::step()` which calculates the net force applied to each planet. For what I wanted to achieve, this lambda ended up being decently more complex than the first one I wrote, which I discussed back in section 5.2. Overall, however, I was quite satisfied with how it came out. I feel that the usage of `for_each()` in this situation may make it slightly more complex for a reader to understand what is going on, but it keeps the code much more concise and removes extra chances for error caused by me trying to implement another loop myself.

Aside from this, most of the process of designing the N-Body Simulator was fairly painless. The only other hurdle I encountered, which absolutely should *not* have been an issue, was my gravitational constant. The very fact that this was a problem at all is astonishing since it is *constant* in value. It just so happens that I messed up the scientific notation format for the gravitational constant. I accidentally wrote `const double G = 6.67 * 10e-11` instead of the appropriate scientific notation format in C++ which is `const double G = 6.67e-11`. This took an absolutely embarrassingly large amount of time to weed out, because I parsed every line of the actual complex math at least a dozen times, since that would reasonably be where something wrong was occurring. Despite this, I was actually very proud that I took my time and implemented considerably better code the first time than my other previous projects, since I got all of the complex mathematical computations correct the first time around.

6.3.2 What I Learned

I learned, perhaps later than others, that a `double` variable can accept scientific notation as a valid immediate value. I had no clue that it was a supported feature, and I felt so much relief knowing that was possible. During this project, I also got a lot better at implementing clean and purposeful helper functions. I feel that the helper functions in this project are significantly better than the ones in previous assignments. I think that the overall cleanliness of the code was also greatly improved upon during the course of this assignment.

6.3.3 The Takeaway

Overall, this was definitely one of my favorite assignments. Not only because it was really cool to get to see and use the end product, but I really felt like I grew a lot as a programmer during just this one assignment. I think I approached this project with considerable diligence, which I may have been lacking for previous assignments. I tried to work slower and implement good,

clean code that would conform to good visual standards as well as pose a reduced amount of debugging the first time, and it really paid off. Although it is not nearly perfect, it serves as a great example of the growth that I experienced over just a few weeks. If you don't believe me, just compare the cleanliness of the `Universe::step()` function in this program to the `SB::Sokoban::movePlayer()` function in PS3, Sokoban.

6.4 Running The Program

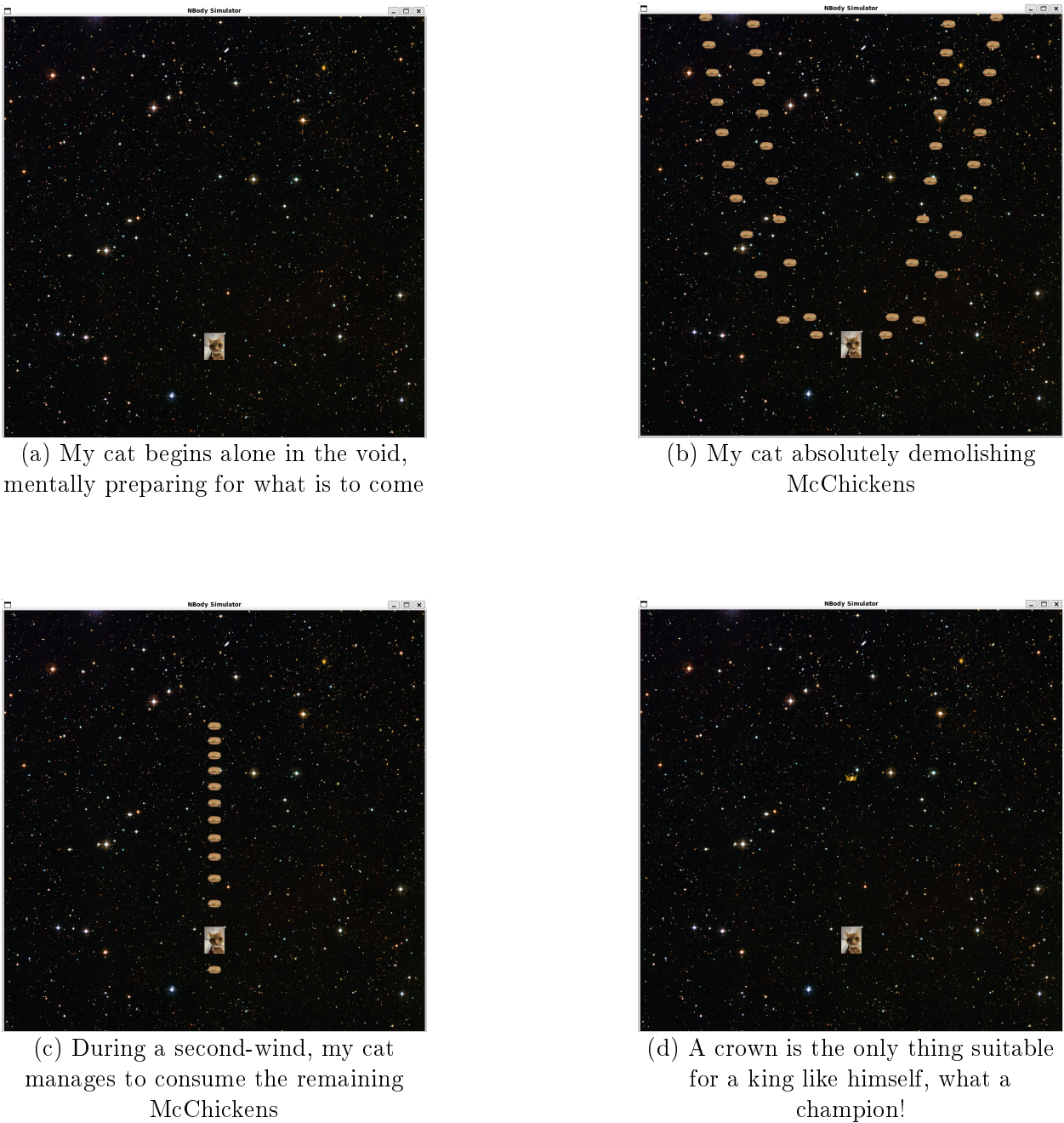
Typing *make* into the terminal while in the project directory will create an executable called **NBody**, as well as a static library **NBody.a**. The following format can be used to run the simulation

```
1 ./NBody T dt < <filename-with-extention>
```

T represents the simulation time to run until in seconds, and dt represents the incremental time intervals in seconds to step along, up until it reaches T. The input redirect (<) character should be used next to redirect input from the file which is specified immediately afterwards.

6.5 Proof of Functionality

Figure 12: A custom simulation I made which shows my cat attempting the 100 McChicken challenge. Further context can be found in section 6.8



6.6 Source Code

The N-Body Simulation program consisted of `Makefile` and `main.cpp` per usual, along with a set of `Universe.hpp` and `Universe.cpp` files, as well as `CelestialBody.hpp` and `CelestialBody.cpp` too. Below is the source code for viewing:

Makefile

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -lboost_unit_test_framework
4 # Your .hpp files
5 DEPS = Universe.hpp CelestialBody.hpp
6 # Your compiled .o files
7 OBJECTS = Universe.o CelestialBody.o
8 # The name of your programs
9 PROGRAM = NBody test
10
11 .PHONY: all clean lint
12
13 all: $(PROGRAM) NBody.a
14
15 # Wildcard recipe to make .o files from corresponding .cpp file
16 %.o: %.cpp $(DEPS)
17     $(CC) $(CFLAGS) -c $<
18
19 NBody: main.o $(OBJECTS)
20     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
21
22 test: test.o $(OBJECTS)
23     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
24
25 NBody.a: $(OBJECTS)
26     ar rcs $@ $^
27
28 clean:
29     rm *.o NBody test NBody.a
30 lint:
31     cpplint *.cpp *.hpp
```

main.cpp

```
1 // Copyright 2024 Nathan Khoury
2 #include "Universe.hpp"
3
4 int main(int argc, char *argv[]) {
5     if (argc < 2) {
6         std::cerr << "Error - Insufficient arguments supplied to command line"
7         << std::endl;
8         return 1;
9     }
10
11     const double T = std::stod(argv[1]);    // Total time to run simulation until
12     const double dt = std::stod(argv[2]);   // Delta Time
13
14     NB::Universe universe;
15     std::cin >> universe;
16
17     // Window position constants are global within CelestialBody.hpp
18
19     sf::RenderWindow window(sf::VideoMode(WINDOW_X, WINDOW_Y),
20         "NBody Simulator");
21     window.setPosition({0, 0});
22
23     sf::Texture bd;
24     if (!bd.loadFromFile("backdrop.jpg")) {
25         std::cerr << "couldn't load backdrop" << std::endl;
26     }
```

```

27     sf::Sprite backdrop(bd);
28     // default scale is 1 by 1 and is determined by the texture
29     backdrop.setScale(
30         WINDOW_X / backdrop.getLocalBounds().width,    // stretch/shrink to window width
31         WINDOW_Y / backdrop.getLocalBounds().height);    // stretch/shrink to window height
32
33     int timeElapsed = 0;
34
35     while (window.isOpen()) {
36         sf::Event event;
37         while (window.pollEvent(event)) {
38             if (event.type == sf::Event::Closed)
39                 window.close();
40         }
41         window.clear();
42         window.draw(backdrop); // draw background
43         window.draw(universe); // draw Universe and CelestialBody objects
44         window.display();
45         if (timeElapsed >= T) {
46             window.close();
47             break;
48         }
49         universe.step(dt);
50         timeElapsed += dt;
51     }
52
53     std::cout << universe << std::endl;
54     return 0;
55 }

```

Universe.hpp

```

1  // Copyright 2024 Nathan Khoury
2  #ifndef UNIVERSE_H
3  #define UNIVERSE_H
4
5  #include <vector>
6  #include <memory>
7
8  #include "CelestialBody.hpp"
9
10 namespace NB {
11
12 class Universe : public sf::Drawable {
13 public:
14     Universe() {} // default constructor (use >> to initialize)
15     explicit Universe(std::string filename); // value constructor (for boost testing)
16     size_t numPlanets() { return _numParticles; }
17     double radius() { return _rad; }
18     void step(double seconds);
19
20     void pairwise(CelestialBody& cb1, CelestialBody& cb2);
21
22     friend std::istream& operator>>(std::istream& in, Universe& universe);
23     friend std::ostream& operator<<(std::ostream& out, Universe& universe);
24
25     CelestialBody& operator[] (size_t);
26
27 protected:
28     void draw(sf::RenderTarget& target, sf::RenderStates states) const override;
29
30 private:
31     size_t _numParticles;
32     double _rad;
33     std::vector<std::shared_ptr<CelestialBody>> particles;
34 };
35

```

```

36 std::istream& operator>>(std::istream& in, Universe& universe);
37 std::ostream& operator<<(std::ostream& out, Universe& universe);
38 }; // namespace NB
39
40 #endif

```

Universe.cpp

```

1  // Copyright 2024 Nathan Khoury
2  #include <fstream>
3
4  #include "Universe.hpp"
5
6  NB::Universe::Universe(std::string filename) {
7      // Alternate string constructor
8      Universe u;
9      std::ifstream f;
10     f.open(filename);
11     // read in file
12     f >> u;
13     *this = std::move(u); // convert universe to rvalue
14     f.close();
15 }
16
17 void NB::Universe::step(double seconds) {
18     // Reset the internal netF count for all CB from previous step
19     for (auto pcb : particles) pcb->netFReset();
20     // Calculate net force for each CelestialBody
21     for (size_t i = 1; i < particles.size(); i++) {
22         CelestialBody& currCB = *particles.at(i - 1);
23         std::for_each(particles.begin() + i, particles.end(),
24             [&](auto pcb){ pairwise(currCB, *pcb); });
25     }
26     // CelestialBody::step() handles force, acceleration, velocity, and positionmak
27     for (auto pcb : particles) pcb->step(seconds);
28 }
29
30 void NB::Universe::pairwise(NB::CelestialBody& cb1, NB::CelestialBody& cb2) {
31     // distance r
32     double r = cb1.distance(cb2);
33     // force F
34     sf::Vector2<double> F = cb1.force(cb2, r);
35     // increment internal netF net force count
36     cb1.netFIncrement(F);
37     // Newton's 3rd law, invert direction
38     F.x *= -1; F.y *= -1;
39     // increment internal netF net force count
40     cb2.netFIncrement(F);
41 }
42
43 std::istream& NB::operator>>(std::istream& in, NB::Universe& universe) {
44     in >> universe._numParticles; // read n
45     in >> universe._rad; // read universe rad
46
47     for (size_t i = 0; i < universe._numParticles; i++) {
48         // call CelestialBody >> operator to read whole body at once
49         std::shared_ptr<CelestialBody> pcb(new CelestialBody(universe._rad));
50         in >> *pcb;
51         // add to internal vector
52         universe.particles.push_back(pcb);
53     }
54     return in;
55 }
56
57 std::ostream& NB::operator<<(std::ostream& out, NB::Universe& universe) {
58     out << universe._numParticles << std::endl; // output n
59     out << universe._rad; // output universe rad

```

```

60
61     // ranged for loop, calls CelestialBody << operator on all stored bodies
62     for (auto pcb : universe.particles)
63         out << std::endl << *pcb;
64     // does not end with new line (main must do so)
65     return out;
66 }
67
68 NB::CelestialBody& NB::Universe::operator[](size_t i) {
69     // index operator returns desired body (useful in boost testing)
70     // uses .at instead of [] with the vector for exception handling benefits
71     return *particles.at(i);
72 }
73
74 void NB::Universe::draw(sf::RenderTarget& target, sf::RenderStates states) const {
75     // calls CelestialBody draw method on all bodies using for_each
76     // (previously used a ranged for loop)
77     std::for_each(particles.begin(), particles.end(), [&](auto pcb){
78         target.draw(*pcb, states);
79     });
80 }

```

CelestialBody.hpp

```

1  // Copyright 2024 Nathan Khoury
2  #ifndef CELESTIALBODY_H
3  #define CELESTIALBODY_H
4
5  #include <iostream>
6  #include <string>
7  #include <cmath>
8  #include <algorithm>
9
10 #include <SFML/Graphics.hpp>
11
12 // Fixed Window Positions:
13 const size_t WINDOW_X = 1000;
14 const size_t WINDOW_Y = 1000;
15 // Gravitational Constant:
16 const double G = 6.67e-11;
17
18 namespace NB {
19 class CelestialBody : public sf::Drawable {
20 public:
21     // CelestialBody obtains Meters per Pixel by dividing the universe's
22     // radius by the radius of the window size to get the ratio
23     // universe-space : window-space
24     explicit CelestialBody(double universeRad = 0)
25         : M_PER_PX(universeRad / (WINDOW_X / 2)) {}
26
27     // getters:
28     const sf::Vector2<double> position() const { return _pos; }
29     const sf::Vector2<double> velocity() const { return _vel; }
30     const double mass() { return _mass; }
31     sf::Vector2<double> acceleration(); // calculate acceleration
32     // setters:
33     void position(sf::Vector2<double>& pos) { _pos = pos; }
34     void velocity(sf::Vector2<double>& vel) { _vel = vel; }
35     void netFReset() { _netF.x = 0; _netF.y = 0; } // reset calculated _netF net forces
36     void netFIncrement(sf::Vector2<double> F); // add F to internal net force count
37     double distance(CelestialBody& cb);
38     sf::Vector2<double> force(CelestialBody& cb, const double r);
39
40     std::string file() { return _imageSource; }
41     void step(double dt);
42
43     friend std::istream& operator>>(std::istream& in, CelestialBody& cb);

```

```

44     friend std::ostream& operator<<(std::ostream& out, CelestialBody& cb);
45
46 protected:
47     void draw(sf::RenderTarget& target, sf::RenderStates states) const override;
48
49 private:
50     sf::Vector2<double> _pos;    // xy position
51     sf::Vector2<double> _vel;    // xy velocities
52     sf::Vector2<double> _netF;   // xy net force
53     double _mass;
54     std::string _imageSource;
55     sf::Texture texture;
56     const double M_PER_PX;
57 };
58
59 std::istream& operator>>(std::istream& in, NB::CelestialBody& cb);
60 std::ostream& operator<<(std::ostream& out, NB::CelestialBody& cb);
61 };    // namespace NB
62
63 #endif

```

CelestialBody.cpp

```

1  // Copyright 2024 Nathan Khoury
2  #include "CelestialBody.hpp"
3
4  sf::Vector2<double> NB::CelestialBody::acceleration() {
5      sf::Vector2<double> a;
6      a.x = _netF.x / _mass;
7      a.y = _netF.y / _mass;
8      return a;
9  }
10
11 void NB::CelestialBody::netFIncrement(sf::Vector2<double> F) {
12     _netF.x += F.x;
13     _netF.y += F.y;
14 }
15
16 void NB::CelestialBody::step(double dt) {
17     sf::Vector2<double> a = acceleration();
18     sf::Vector2<double> v;
19     v.x = velocity().x + dt * a.x;
20     v.y = velocity().y + dt * a.y;
21     velocity(v);
22     sf::Vector2<double> p;
23     p.x = position().x + dt * velocity().x;
24     p.y = position().y + dt * velocity().y;
25     position(p);
26 }
27
28 double NB::CelestialBody::distance(CelestialBody& cb) {
29     // get change in x/y
30     double dx = std::abs(cb.position().x - this->position().x);
31     double dy = std::abs(cb.position().y - this->position().y);
32     // get distance r between planets
33     double r = std::sqrt(dx * dx + dy * dy);
34     return r;
35 }
36
37 sf::Vector2<double> NB::CelestialBody::force(CelestialBody& cb, const double r) {
38     double force = (G * this->mass() * cb.mass()) / (r * r);
39     sf::Vector2<double> F;
40     F.x = force * (cb.position().x - this->position().x) / r;
41     F.y = force * (cb.position().y - this->position().y) / r;
42     return F;
43 }
44

```

```

45 std::istream& NB::operator>>(std::istream& in, NB::CelestialBody& cb) {
46     // read in all the easy data in order first
47     in >> cb._pos.x;    // read x position
48     in >> cb._pos.y;    // read y position
49     in >> cb._vel.x;    // read x velocity
50     in >> cb._vel.y;    // read y velocity
51     in >> cb._mass;     // read mass of the celestial body
52     in >> cb._imageSource; // store filename in string
53
54     if (!cb.texture.loadFromFile(cb._imageSource)) {
55         // detect if texture loads properly
56         std::cerr << cb._imageSource << " failed to load" << std::endl;
57     }
58     // texture is now stored as a member of the CelestialBody
59     return in;
60 }
61
62 std::ostream& NB::operator<<(std::ostream& out, NB::CelestialBody& cb) {
63     // output with the same format as input
64     out.precision(4);
65     out << std::scientific;
66
67     // output x,y positions
68     out << cb._pos.x << " " << cb._pos.y << " ";
69     // output x,y velocities
70     out << cb._vel.x << " " << cb._vel.y << " ";
71     // output mass
72     out << cb._mass << " ";
73     // output filename
74     out << cb._imageSource;
75
76     return out;
77 }
78
79 void NB::CelestialBody::draw(sf::RenderTarget& target, sf::RenderStates states) const {
80     sf::Sprite body(this->texture);    // create a new sprite from texture
81     // move origin to center
82     body.setOrigin(body.getLocalBounds().width / 2,
83         body.getLocalBounds().height / 2);
84     // move position to calculated position
85     // (converted position in pixels) + (offset center of the universe)
86     body.setPosition((_pos.x / M_PER_PX) + (WINDOW_X / 2),
87         ((_pos.y * -1 / M_PER_PX) + (WINDOW_Y / 2)));
88     // draw the sprite
89     target.draw(body, states);
90 }

```

6.7 Unit Testing

The unit testing for the N-Body Simulation program was a bit challenging, mostly due to the imprecision which accrues after more and more approximations. Recall that this simulation is not operating under infinite precision, which means slight differences in expected values may occur due to internal rounding and the order in which one performs calculations. As a result of this, the tests verify that the values which are generated through various calculations are within 0.001 of the actual answer at any time. This sets a more realistic expectation for the tests, and still sufficiently verifies functionality.

```

1 // Copyright 2024 Nathan Khoury
2 #include <string>
3 #include <sstream>
4 #include <fstream>
5 #include "Universe.hpp"
6
7 #define BOOST_TEST_DYN_LINK
8 #define BOOST_TEST_MODULE NBody_Testing

```

```

9  #include <boost/test/unit_test.hpp>
10
11 BOOST_AUTO_TEST_CASE(testNumPlanetsAndRad) {
12     NB::Universe u("planets.txt");
13     // expected values
14     const size_t n = 5;
15     const double rad = 2.50e+11;
16     BOOST_REQUIRE_EQUAL(u.numPlanets(), n);
17     BOOST_REQUIRE_CLOSE(u.radius(), rad, 0.001);
18 }
19
20 BOOST_AUTO_TEST_CASE(testFirstPlanetPosition) {
21     NB::Universe u("planets.txt");
22     // expected values
23     const double posX = 1.4960e+11;
24     const double posY = 0.0000e+00;
25
26     NB::CelestialBody cb = u[0];
27
28     BOOST_REQUIRE_CLOSE(cb.position().x, posX, 0.001);
29     BOOST_REQUIRE_CLOSE(cb.position().y, posY, 0.001);
30 }
31
32 BOOST_AUTO_TEST_CASE(testFirstPlanetVelocity) {
33     NB::Universe u("planets.txt");
34     // expected values
35     const double velx = 0.0000e+00;
36     const double vely = 2.9800e+04;
37
38     NB::CelestialBody cb = u[0];
39
40     BOOST_REQUIRE_CLOSE(cb.velocity().x, velx, 0.001);
41     BOOST_REQUIRE_CLOSE(cb.velocity().y, vely, 0.001);
42 }
43
44 BOOST_AUTO_TEST_CASE(testFirstPlanetMass) {
45     NB::Universe u("planets.txt");
46     // expected values
47     const double mass = 5.9740e+24;
48
49     NB::CelestialBody cb = u[0];
50
51     BOOST_REQUIRE_CLOSE(cb.mass(), mass, 0.001);
52 }
53
54 BOOST_AUTO_TEST_CASE(testSpecificPos1) {
55     NB::Universe u("galaxy.txt");
56     // expected values
57     const double posX = -3.5E9;
58     const double posY = 0.0;
59
60     NB::CelestialBody cb = u[7];
61
62     BOOST_REQUIRE_CLOSE(cb.position().x, posX, 0.001);
63     BOOST_REQUIRE_CLOSE(cb.position().y, posY, 0.001);
64 }
65
66 BOOST_AUTO_TEST_CASE(testSpecificVel1) {
67     NB::Universe u("galaxy.txt");
68     // expected values
69     const double velx = 0.0;
70     const double vely = -414142.8325038183;
71
72     NB::CelestialBody cb = u[7];
73
74     BOOST_REQUIRE_CLOSE(cb.velocity().x, velx, 0.001);

```

```

75     BOOST_REQUIRE_CLOSE(cb.velocity().y, vely, 0.001);
76 }
77
78 BOOST_AUTO_TEST_CASE(testSpecificMass1) {
79     NB::Universe u("galaxy.txt");
80     // expected values
81     const double mass = 8.571428571428572E10;
82
83     NB::CelestialBody cb = u[7];
84
85     BOOST_REQUIRE_CLOSE(cb.mass(), mass, 0.001);
86 }
87
88 BOOST_AUTO_TEST_CASE(testSpecificPos2) {
89     NB::Universe u("uniform3.txt");
90     // expected values
91     const double posx = -2.500000e+08;
92     const double posy = 4.330127e+08;
93
94     NB::CelestialBody cb = u[1];
95
96     BOOST_REQUIRE_CLOSE(cb.position().x, posx, 0.001);
97     BOOST_REQUIRE_CLOSE(cb.position().y, posy, 0.001);
98 }
99
100 BOOST_AUTO_TEST_CASE(testSpecificVel2) {
101     NB::Universe u("uniform3.txt");
102     // expected values
103     const double velx = 2.149677e+02;
104     const double vely = 1.241117e+02;
105
106     NB::CelestialBody cb = u[1];
107
108     BOOST_REQUIRE_CLOSE(cb.velocity().x, velx, 0.001);
109     BOOST_REQUIRE_CLOSE(cb.velocity().y, vely, 0.001);
110 }
111
112 BOOST_AUTO_TEST_CASE(testSpecificMass2) {
113     NB::Universe u("uniform3.txt");
114     // expected values
115     const double mass = 8.00e+23;
116
117     NB::CelestialBody cb = u[1];
118
119     BOOST_REQUIRE_CLOSE(cb.mass(), mass, 0.001);
120 }
121
122 BOOST_AUTO_TEST_CASE(testOutput) {
123     NB::Universe u("1body.txt");
124     NB::CelestialBody cb = u[0];    // extract CelestialBody for testing
125     std::stringstream ss;
126     // store entire output in ss
127     ss << u;
128     int n;    // n planets
129     double x;    // for storing all doubles
130     ss >> n;    // check n bodies
131     BOOST_REQUIRE_EQUAL(n, u.numPlanets());
132     ss >> x;    // check radius
133     BOOST_REQUIRE_CLOSE(x, u.radius(), 0.001);
134     ss >> x;    // test posx
135     BOOST_REQUIRE_CLOSE(x, cb.position().x, 0.001);
136     ss >> x;    // test posy
137     BOOST_REQUIRE_CLOSE(x, cb.position().y, 0.001);
138     ss >> x;    // test velx
139     BOOST_REQUIRE_CLOSE(x, cb.velocity().x, 0.001);
140     ss >> x;    // test vely

```

```

141 BOOST_REQUIRE_CLOSE(x, cb.velocity().y, 0.001);
142 ss >> x;    // test mass
143 BOOST_REQUIRE_CLOSE(x, cb.mass(), 0.001);
144 }
145
146 BOOST_AUTO_TEST_CASE(oneStep) {
147     NB::Universe u("planets.txt");
148     double dt = 25000; // delta time
149     u.step(dt);
150     NB::CelestialBody cb = u[0];
151     BOOST_REQUIRE_EQUAL(5, u.numPlanets());
152     BOOST_REQUIRE_EQUAL(2.50e+11, u.radius());
153     BOOST_REQUIRE_CLOSE(1.4960e+11, cb.position().x, 0.01);
154     BOOST_REQUIRE_CLOSE(7.4500e+08, cb.position().y, 0.01);
155     BOOST_REQUIRE_CLOSE(-1.4820e+02, cb.velocity().x, 0.01);
156     BOOST_REQUIRE_CLOSE(2.9800e+04, cb.velocity().y, 0.01);
157     BOOST_REQUIRE_EQUAL(5.9740e+24, cb.mass());
158 }
159
160 BOOST_AUTO_TEST_CASE(twoSteps) {
161     NB::Universe u("planets.txt");
162     double dt = 25000; // delta time
163     u.step(dt);
164     u.step(dt);
165     NB::CelestialBody cb = u[0];
166     BOOST_REQUIRE_EQUAL(5, u.numPlanets());
167     BOOST_REQUIRE_EQUAL(2.50e+11, u.radius());
168     BOOST_REQUIRE_CLOSE(1.4959e+11, cb.position().x, 0.01);
169     BOOST_REQUIRE_CLOSE(1.4900e+09, cb.position().y, 0.01);
170     BOOST_REQUIRE_CLOSE(-2.9640e+02, cb.velocity().x, 0.01);
171     BOOST_REQUIRE_CLOSE(2.9799e+04, cb.velocity().y, 0.01);
172     BOOST_REQUIRE_EQUAL(5.9740e+24, cb.mass());
173 }
174
175 BOOST_AUTO_TEST_CASE(threeSteps) {
176     NB::Universe u("planets.txt");
177     double dt = 25000; // delta time
178     u.step(dt);
179     u.step(dt);
180     u.step(dt);
181     NB::CelestialBody cb = u[0];
182     BOOST_REQUIRE_EQUAL(5, u.numPlanets());
183     BOOST_REQUIRE_EQUAL(2.50e+11, u.radius());
184     BOOST_REQUIRE_CLOSE(1.4958e+11, cb.position().x, 0.01);
185     BOOST_REQUIRE_CLOSE(2.2349e+09, cb.position().y, 0.01);
186     BOOST_REQUIRE_CLOSE(-4.4460e+02, cb.velocity().x, 0.01);
187     BOOST_REQUIRE_CLOSE(2.9798e+04, cb.velocity().y, 0.01);
188     BOOST_REQUIRE_EQUAL(5.9740e+24, cb.mass());
189 }
190
191 BOOST_AUTO_TEST_CASE(oneYearEarth) {
192     NB::Universe u("planets.txt");
193     double T = 0;
194     double dt = 25000; // delta time
195     while (T < 31557600) {
196         u.step(dt);
197         T += 25000;
198     }
199     NB::CelestialBody cb = u[0];
200     BOOST_REQUIRE_EQUAL(5, u.numPlanets());
201     BOOST_REQUIRE_EQUAL(2.50e+11, u.radius());
202     BOOST_REQUIRE_CLOSE(1.4959e+11, cb.position().x, 0.01);
203     BOOST_REQUIRE_CLOSE(-1.6531e+09, cb.position().y, 0.01);
204     BOOST_REQUIRE_CLOSE(3.2949e+02, cb.velocity().x, 0.01);
205     BOOST_REQUIRE_CLOSE(2.9798e+04, cb.velocity().y, 0.01);
206     BOOST_REQUIRE_EQUAL(5.9740e+24, cb.mass());

```

```

207 }
208
209 BOOST_AUTO_TEST_CASE(oneYearMultiplePlanets) {
210     NB::Universe u("planets.txt");
211     double T = 0;
212     double dt = 25000; // delta time
213     while (T < 31557600) {
214         u.step(dt);
215         T += 25000;
216     }
217     BOOST_REQUIRE_EQUAL(5, u.numPlanets());
218     BOOST_REQUIRE_EQUAL(2.50e+11, u.radius());
219     NB::CelestialBody cb1 = u[0];
220     BOOST_REQUIRE_CLOSE(1.4959e+11, cb1.position().x, 0.01);
221     BOOST_REQUIRE_CLOSE(-1.6531e+09, cb1.position().y, 0.01);
222     BOOST_REQUIRE_CLOSE(3.2949e+02, cb1.velocity().x, 0.01);
223     BOOST_REQUIRE_CLOSE(2.9798e+04, cb1.velocity().y, 0.01);
224     BOOST_REQUIRE_EQUAL(5.9740e+24, cb1.mass());
225
226     NB::CelestialBody cb2 = u[1];
227     BOOST_REQUIRE_CLOSE(-2.2153e+11, cb2.position().x, 0.01);
228     BOOST_REQUIRE_CLOSE(-4.9263e+10, cb2.position().y, 0.01);
229     BOOST_REQUIRE_CLOSE(5.1805e+03, cb2.velocity().x, 0.01);
230     BOOST_REQUIRE_CLOSE(-2.3640e+04, cb2.velocity().y, 0.01);
231     BOOST_REQUIRE_EQUAL(6.4190e+23, cb2.mass());
232
233     NB::CelestialBody cb3 = u[2];
234     BOOST_REQUIRE_CLOSE(3.4771e+10, cb3.position().x, 0.01);
235     BOOST_REQUIRE_CLOSE(4.5752e+10, cb3.position().y, 0.01);
236     BOOST_REQUIRE_CLOSE(-3.8269e+04, cb3.velocity().x, 0.01);
237     BOOST_REQUIRE_CLOSE(2.9415e+04, cb3.velocity().y, 0.01);
238     BOOST_REQUIRE_EQUAL(3.3020e+23, cb3.mass());
239
240     NB::CelestialBody cb4 = u[3];
241     BOOST_REQUIRE_CLOSE(5.9426e+05, cb4.position().x, 0.01);
242     BOOST_REQUIRE_CLOSE(6.2357e+06, cb4.position().y, 0.01);
243     BOOST_REQUIRE_CLOSE(-5.8569e-02, cb4.velocity().x, 0.01);
244     BOOST_REQUIRE_CLOSE(1.6285e-01, cb4.velocity().y, 0.01);
245     BOOST_REQUIRE_EQUAL(1.9890e+30, cb4.mass());
246 }
247
248 BOOST_AUTO_TEST_CASE(oneBodyOneStep) {
249     NB::Universe u("1body.txt");
250     double dt = 1000; // delta time
251     u.step(dt);
252     NB::CelestialBody cb = u[0];
253     BOOST_REQUIRE_EQUAL(1, u.numPlanets());
254     BOOST_REQUIRE_EQUAL(100, u.radius());
255     BOOST_REQUIRE_EQUAL(2010, cb.position().x);
256     BOOST_REQUIRE_EQUAL(1020, cb.position().y);
257     BOOST_REQUIRE_EQUAL(2, cb.velocity().x);
258     BOOST_REQUIRE_EQUAL(1, cb.velocity().y);
259     BOOST_REQUIRE_EQUAL(1e20, cb.mass());
260 }
261
262
263 BOOST_AUTO_TEST_CASE(noMovementOneStep) {
264     NB::Universe u("1bodyNoMovement.txt");
265     double dt = 1000; // delta time
266     u.step(dt);
267     NB::CelestialBody cb = u[0];
268     BOOST_REQUIRE_EQUAL(1, u.numPlanets());
269     BOOST_REQUIRE_EQUAL(100, u.radius());
270     BOOST_REQUIRE_EQUAL(10, cb.position().x);
271     BOOST_REQUIRE_EQUAL(20, cb.position().y);
272     BOOST_REQUIRE_EQUAL(0, cb.velocity().x);

```

```
273 BOOST_REQUIRE_EQUAL(0, cb.velocity().y);
274 BOOST_REQUIRE_EQUAL(1e20, cb.mass());
275 }
276
277 BOOST_AUTO_TEST_CASE(twoStars) {
278     NB::Universe u("binary.txt");
279     double dt = 1000; // delta time
280     u.step(dt);
281     NB::CelestialBody cb1 = u[0];
282     BOOST_REQUIRE_EQUAL(2, u.numPlanets());
283     BOOST_REQUIRE_EQUAL(5e+10, u.radius());
284     BOOST_REQUIRE_CLOSE(1e+07, cb1.position().x, 0.01);
285     BOOST_REQUIRE_CLOSE(4.5000e+10, cb1.position().y, 0.01);
286     BOOST_REQUIRE_CLOSE(1.0000e+04, cb1.velocity().x, 0.01);
287     BOOST_REQUIRE_CLOSE(-1.2352e+01, cb1.velocity().y, 0.01);
288     BOOST_REQUIRE_EQUAL(1.5000e+30, cb1.mass());
289
290     NB::CelestialBody cb2 = u[1];
291     BOOST_REQUIRE_CLOSE(-1e+07, cb2.position().x, 0.01);
292     BOOST_REQUIRE_CLOSE(-4.5000e+10, cb2.position().y, 0.01);
293     BOOST_REQUIRE_CLOSE(-1.0000e+04, cb2.velocity().x, 0.01);
294     BOOST_REQUIRE_CLOSE(1.2352e+01, cb2.velocity().y, 0.01);
295     BOOST_REQUIRE_EQUAL(1.5000e+30, cb2.mass());
296 }
```

6.8 Bonus

For bonus I decided to add a backdrop of an image of a universe which I sourced and cited. I also decided to create my own simulation, which I called `mcchicken.txt` and features a lighthearted portrayal of what it may look like if my cat were to attempt to eat 100 McChickens in a row as a challenge. The contents of the `mcchicken.txt` file can be found below. You can get immersed in the story at the very bottom of the file where I explain the situation and a breakdown of what the simulation demonstrates.

```
1 102
2 5.275e+10
3 0.0000e+00 -3.0000e+10 0.0000e+00 0.0000e+00 5.0210e+31 cat.jpg
4 4.4000e+10 1.5000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
5 4.4000e+10 1.5500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
6 4.4000e+10 1.6000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
7 4.4000e+10 1.6500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
8 4.4000e+10 1.7000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
9 4.4000e+10 1.7500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
10 4.4000e+10 1.8000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
11 4.4000e+10 1.8500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
12 4.4000e+10 1.9000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
13 4.4000e+10 1.9500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
14 4.4000e+10 2.0000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
15 4.4000e+10 2.0500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
16 4.4000e+10 2.1000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
17 4.4000e+10 2.1500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
18 4.4000e+10 2.2000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
19 4.4000e+10 2.2500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
20 4.4000e+10 2.3000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
21 4.4000e+10 2.3500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
22 4.4000e+10 2.4000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
23 4.4000e+10 2.4500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
24 -4.4000e+10 1.5000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
25 -4.4000e+10 1.5500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
26 -4.4000e+10 1.6000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
27 -4.4000e+10 1.6500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
28 -4.4000e+10 1.7000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
29 -4.4000e+10 1.7500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
30 -4.4000e+10 1.8000e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
31 -4.4000e+10 1.8500e+11 0.0000e+00 -1.5000e+5 3.7500e+2 mcchicken.png
```

	-4.4000e+10	1.9000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
33	-4.4000e+10	1.9500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
34	-4.4000e+10	2.0000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
35	-4.4000e+10	2.0500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
36	-4.4000e+10	2.1000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
37	-4.4000e+10	2.1500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
38	-4.4000e+10	2.2000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
39	-4.4000e+10	2.2500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
40	-4.4000e+10	2.3000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
41	-4.4000e+10	2.3500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
42	-4.4000e+10	2.4000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
43	-4.4000e+10	2.4500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
44	3.0000e+10	1.5000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
45	3.0000e+10	1.5500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
46	3.0000e+10	1.6000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
47	3.0000e+10	1.6500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
48	3.0000e+10	1.7000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
49	3.0000e+10	1.7500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
50	3.0000e+10	1.8000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
51	3.0000e+10	1.8500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
52	3.0000e+10	1.9000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
53	3.0000e+10	1.9500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
54	3.0000e+10	2.0000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
55	3.0000e+10	2.0500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
56	3.0000e+10	2.1000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
57	3.0000e+10	2.1500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
58	3.0000e+10	2.2000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
59	3.0000e+10	2.2500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
60	3.0000e+10	2.3000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
61	3.0000e+10	2.3500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
62	3.0000e+10	2.4000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
63	3.0000e+10	2.4500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
64	-3.0000e+10	1.5000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
65	-3.0000e+10	1.5500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
66	-3.0000e+10	1.6000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
67	-3.0000e+10	1.6500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
68	-3.0000e+10	1.7000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
69	-3.0000e+10	1.7500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
70	-3.0000e+10	1.8000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
71	-3.0000e+10	1.8500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
72	-3.0000e+10	1.9000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
73	-3.0000e+10	1.9500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
74	-3.0000e+10	2.0000e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
75	-3.0000e+10	2.0500e+11	0.0000e+00	-1.5000e+5	3.7500e+2	mcchicken.png
7						

```

98 0.0000e+00 6.1000e+11 0.0000e+00 -3.0000e+5 3.7500e+2 mcchicken.png
99 0.0000e+00 6.1250e+11 0.0000e+00 -3.0000e+5 3.7500e+2 mcchicken.png
100 0.0000e+00 6.1500e+11 0.0000e+00 -3.0000e+5 3.7500e+2 mcchicken.png
101 0.0000e+00 6.1750e+11 0.0000e+00 -3.0000e+5 3.7500e+2 mcchicken.png
102 0.0000e+00 6.2000e+11 0.0000e+00 -3.0000e+5 3.7500e+2 mcchicken.png
103 0.0000e+00 6.2250e+11 0.0000e+00 -3.0000e+5 3.7500e+2 mcchicken.png
104 0.0000e+00 3.5000e+11 0.0000e+00 -8.0000e+4 2.0000e+2 crown.png
105
106 This simulation displays my very own cat (His name is Tigger) attempting
107 the 100 mcchicken challenge. As you may have guessed this is not an easy
108 challenge, although at times during this simulation it may seem that it is
109 easy (he is just very hungry). Watch as my cat Tigger demolishes 100
110 mcchickens, or perishes trying.
111
112 Recommended command line inputs to run with:
113 =====
114 T = 2495000
115 dt = 1000
116
117 Plot synopsis !!SPOILERS!!
118 =====
119 - We begin with Tigger floating in a nebulous void. It's not clear how he got
120 here, but it is abundantly clear he has quite the appetite.
121 - Before long, he begins gulping down mcchickens. He is like a bottomless pit,
122 his performance is truly astonishing.
123 - At the 80th mcchicken he grinds to a halt, his belly is so full, but he is so close.
124 It is unclear in this moment if he will finish the challenge.
125 - Then, after a brief rest, he resumes. He rapidly throws the remaining 20 down his hatch.
126 The silence in this moment during the simulation is deafening, it truly speaks volumes to
127 the emotional climax that is occurring.
128 - After a light intermission, a crown descends down upon his head deeming him champion of
129 the 100 mcchicken challenge. As soon as the crown lands on his head, the simulation
130 finishes.
131
132
133 mcchicken: https://upload.wikimedia.org/wikipedia/commons/c/c1/McD-McChicken\_%28infobox%29.png
134 crown: https://www.deviantart.com/prussiaart/art/Gold-crown-on-a-transparent-background-761936580
135 cat: my own :)

```

7 PS5: DNA Sequence Alignment

7.1 Functionality

For PS5 I engaged in pair programming with my partner Ishan Phadke to implement a **DNA sequence alignment** program. DNA sequence alignment is a real practice used in science to compare the similarities within two separate strands of DNA. Imagine, as a scientist, you discovered a new creature never before seen. You and your colleagues would certainly want to study all the aspects of it, DNA included. Rather than sitting down and manually comparing the nucleotides of the new DNA to some arbitrary known creatures' DNA, we can expedite the process by using a program to calculate how similar they are.

To understand the program we first must understand the method we use to perform DNA sequence alignment. The way our method works is by calculating the **edit distance** between the two DNA segments. When the two DNA segments are aligned, the nucleotides are likely not paired up in the most optimal arrangement automatically. While you can't change the order that the nucleotides are in, you *are* able to insert **gaps**, but it'll cost you. Below is a list of the *three* actions we can perform when trying to create the most optimal pairings. Each action has an associated cost which will serve as a **penalty** for performing the listed action. The edit distance, as was referenced before, is the sum of all penalties within the most optimal alignment of the two DNA segments.

- In the case that the nucleotides are lined up and are the same, if we choose to leave them as is we would pay a **penalty of 0**
- If the nucleotides are lined up and are different, choosing to leave as is would yield a **penalty of 1**
- Finally, we may choose to insert a gap and line up a nucleotide with an empty space. Doing this may create more optimal alignments for the rest of the DNA segment, but it comes with a hefty **penalty of 2**

Below is a table which visualizes what the optimal alignment between the words "plumber" and "summer" would yield. The benefit to writing our program is that it works with any character, not just the four designated for nucleotides (A, T, C, and G).

	p	l	u	m	b	e	r	-
s	4	2	3	5	7	8	10	12
u	5	3	1	3	5	6	8	10
m	7	5	3	1	3	4	6	8
m	8	6	4	2	1	2	4	6
e	10	8	6	4	2	0	2	4
r	12	10	8	6	4	2	0	2
-	14	12	10	8	6	4	2	0

Table 1: A comparison between "plumber" and "summer" with the optimal edit distance computed (green), and path highlighted (yellow)

As we can see "plumber" and "summer" align pretty well together. We can notice this mainly due to the "straightness" of the visualization of the optimal alignment path. Any movement straight up/down or left/right (depending on how you look at it) denotes that a gap exists there, which is the most expensive action that we can perform in this process and should be used relatively sparingly. Our edit distance is expressed in the top left corner (highlighted in green in table 1), which is pretty cheap relative to the length of the words compared. Below is another table which expresses only the edit path itself. The format of this table is identical to the format of the table generated by the program. The two characters being lined up in the step are provided, with the value in the third column being the cost for that

step itself. This allows us to see where our penalties are coming from, explaining how we obtained the edit distance value that we did.

X	Y	#
s	p	1
-	l	2
u	u	0
m	m	0
m	b	1
e	e	0
r	r	0

Table 2: The path of how to obtain the optimal edit distance when comparing "plumber" and "summer" (gaps are represented by the '-' character)

The next logical question might be: how do we decide what action is optimal given any situation? The method we implemented in PS5 uses a two-dimensional matrix to create a table which is ready to compare the two strings together, functionally identical to table 1. Supposed that the length of the string spanning the *width* of the matrix has a length **N**, and the current horizontal index being iterated on is given by **j**. Also suppose that the length of the string spanning the *height* of the matrix is given by **M**, and the current vertical index being iterated on is given by **j**. Let our $N \times M$ matrix be named `opt`, such that the current cell we are trying to solve for is given by `opt[i][j]` and our total edit distance will be placed in `opt[0][0]`.

The method used by my partner and I's implementation applies that the current cell that we would like to fill in, `opt[i][j]`, should take the *minimum* penalty of three adjacent cells. The cells it should compare should be one to the right of itself, one below itself, and one down *and* to the right of itself. This actualizes as roughly:

```

1 //           right           down           diagonal (down + right)
2 opt[i][j] = min{opt[i][j+1] + 2, opt[i+1][j] + 2,  opt[i+1][j+1] + 0/1}
```

Intuitively, one may consider a recursive approach which starts at `opt[0][0]` and solves all of the branches as is required by the recursion. For comparisons of smaller strings this is not unreasonable; However, when one is meant to compare DNA strings with hundreds of thousands of characters which represent the nucleotides, the operations very quickly become too large in quantity for pretty much any common computer to store. For this reason, we implemented a programming paradigm known as **dynamic programming**. Dynamic programming essentially requires that we compute all of the computations we will need to do ahead of time, such that when we would like to determine the edit distance, it has already been filled out for us. For our program, this results in us precomputing the entire $N \times M$ matrix, *then* we obtain the edit distance which is already calculated in `opt[0][0]` for us. The only responsibility we have left is to uncover what the optimal path was. This can be determined by starting at `opt[0][0]` and taking the same three adjacent cells, and solving for which cell (+ the respective penalty) is equal to our current cell. We can complete this until we reach `opt[M][N]`, at which point we have traced the entire path.

With all of this context out of the way, lets begin examining how my partner and I implemented this process. Continue to section 7.2 to learn more.

7.2 Features

The program begins by reading two strings from `stdin`, after which we pass them both to a constructor to initialize an object from our custom `EDistance` class. The constructor stores the strings, as well as their sizes, and then resizes our internal matrix `_opt` which is a `std::vector<std::vector<int> >`. Where I had opted to use a single vector back in Sokoban (section 5) to represent my matrix, my partner and I had decided to use a vector of vectors to represent our matrix in this program. We had felt that access to the index operators would be better for keeping the code easier to read and interpret for ourselves,

and this proved to be a wise choice.

With the `EDistance` object initialized, we started a SFML clock so that we can record the time it will have taken to calculate the edit distance and present it with the final path an edit distance at the end. Then, we called `EDistance::optDistance()` which fills out the entire matrix using the dynamic programming paradigm. `optDistance()` begins by filling out the far right column, as well as the bottommost row, first. We can do this with ease since, if you refer back to table 1, the far right column and bottommost row follow a distinct pattern: they both count up by two as they move down their respective axes. This occurs since we pad the ends of each provided string with an extra empty character in case the most optimal path requires that we begin either string with a gap. Due to the nature of this, this means that as you follow each gap up, they will always increment by exactly 2.

After filling out the freebies, we begin our main nested for loop at `opt[i][j]`, where `i = N - 1` and `j = M - 1`. This will place us comfortably in the bottom right corner, the position of the first unfilled cell, where our three adjacent cells that we will be taking the minimum of are already filled out. We can individually fill out each cell with ease by walking across the matrix right to left, from the bottom to the top, like if we were reading a book upside down. We can do this by using a function `EDistance::min3()` which is a static member function of our class that returns the minimum value of the three provided arguments. Following the rough idea presented earlier, our actual usage of `min3()` is shown below.

```
1 _opt[i][j] = min3(_opt[i + 1][j + 1] + penalty(_x.at(i), _y.at(j)), // diagonal + penalty
2   _opt[i + 1][j] + 2, // down + gap
3   _opt[i][j + 1] + 2); // right + gap
```

The `EDistance::penalty()` function is a static member function which checks if the two provided characters are the same. If the characters are the same, `penalty()` returns 0, whereas if they are different the function will return 1 as the appropriate penalty. Once we have iterated up to `_opt[0][0]` and completed the `min3()` calculation there, we will have our edit distance, which we can return back to main to be outputted to the terminal.

Once we have outputted our edit distance to the terminal for the user, we then must retrace our steps to figure out what path yields the most optimal alignment. We accomplish this by calling `EDistance::alignment()` which creates a `std::stringstream` object to construct a representation of the path within, visually similar to what is displayed in table 2. This is accomplished by beginning in `_opt[0][0]` and examining our three adjacent neighbors the same way we did before. This time, however, we are going to see if the value of our neighbor *with* the added applicable penalty is equal to the cell we are currently in. When we find which one is equal, or the first if multiple are equal, we then use a helper function `EDistance::printAlignment` to format a string based off of this fork in the path, which we will add to our stringstream for later, and iterate again. This continues until we have reached `_opt[M][N]` at which point we have the full path recorded in our stringstream, which we will return back to main to be outputted to the terminal.

With the edit distance and the string alignment path both shown to the user at this point, the only thing left to do is to capture the current time recorded in our `sf::Clock` within a `sf::Time` variable and output it to the string in terms of seconds. At this point the program will terminate and we have successfully calculated the edit distance and optimal string alignment path of our two strings!

7.3 Personal Reflection

7.3.1 The Struggles

My partner, Ishan, and I actually worked very well together, and it was very refreshing getting to collaborate with someone on one of these projects. That being said, no project is without its challenges. It is disappointing to say, but one of our largest bugs was due to the fact that we had tried to utilize a `size_t` data type as a stopping condition for our `for` loops in `EDistance::optDistance`. This is problematic due to the fact that our stopping conditions check for when `i/j` iterate below 0, which is a *very* bad stopping condition to use

a `size_t` data type for since the `size_t` type is **unsigned** meaning that it will only ever be positive. This lead to a couple hours of fighting off segmentation fault errors, but we made it past this eventually and persevered. The rest of the assignment actually proved not too challenging. The other most challenging aspect of the PS5 project in my opinion was the memory analysis we performed in the readme file. Using **valgrind** and **massif** was quite the task since there is so much informing being presented to the user, it can be hard to deduce where the information that we *want* is. Despite this we got everything working, and recorded all of the data.

7.3.2 What I Learned

I was very apprehensive to work in a group with someone because I have always been self-conscious about my skills, and I wouldn't want to be a burden on someone else who is expecting me to pull my weight evenly. This turned out to not be an issue at all and it was actually very pleasant. I attribute this to the fact that we both struggled in our own ways, and we were both uniquely helpful, which allowed us to compensate for each other's weaknesses not just for the sake of the project, but as a way for both of us to learn from each other. Being able to communicate ideas to someone in real time as you are both working on the same program proved to be an incredible learning experience, and I hope I can do collaborative work like that much more in the future

7.3.3 The Takeaway

I found that even though this assignment was conceptually a very challenging one to understand the methodology of, working with someone really helped to close the gaps in knowledge enough such that it overall reduced the amount of time spent debugging ridiculous mistakes (although evidently some still manage to sneak through the crack as was described at the start of section 7.3.1).

7.4 Running The Program

The program can be run in two different ways

```
1 ./EDistance < <filename-with-extention>
2 // OR
3 ./EDistance
```

The first method allows the user to use input redirection to compare strings which are stored in a file. This may be particularly useful for long DNA strings such as the *ecoli* ones that we had to test in this project. The second method is viable as well since the program uses `stdin` regardless, so one could simply just type two strings themselves.

7.5 Proof of Functionality

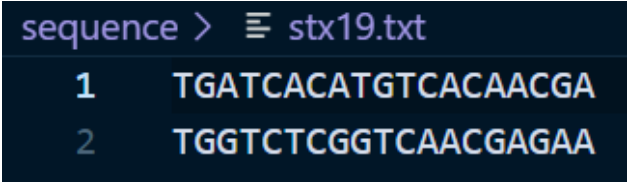


Figure 13: Contents of the `stx19.txt` file

```
nkhoury@nathan-laptop:~/comp4/ps5$ ./EDistance < sequence/stx19.txt
Edit distance = 10
T T 0
G G 0
A G 1
T T 0
C C 0
A T 1
C C 0
A G 1
T G 1
G T 1
T C 1
C A 1
A A 0
C C 0
A G 1
A A 0
C G 1
G A 1
A A 0

Execution time is 0.000277 seconds
nkhoury@nathan-laptop:~/comp4/ps5$
```

Figure 14: The result of running the `stx19.txt` file as redirected input

7.6 Source Code

Utilizing the provided makefile with the `make` command will yield an executable named **EDistance** and a static library called, suitably, **EDistance.a**. The DNA sequence alignment program features `EDistance.hpp` and `EDistance.cpp` supporting files, as well as a `main.cpp` for providing the main driver for this project. Below is the source code from the project

Makefile

```
1 CC = g++
2
3 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
4
5 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -lboost_unit_test_framework
6
7 # Your .hpp files
8
9 DEPS = EDistance.hpp
10
11 # Your compiled .o files
12
13 OBJECTS = EDistance.o
14
15 # The name of your programs
16
17 PROGRAM = EDistance test
18
19
20
21 .PHONY: all clean lint
22
23
24
25 all: $(PROGRAM) EDistance.a
26
27
28
29 # Wildcard recipe to make .o files from corresponding .cpp file
30
31 %.o: %.cpp $(DEPS)
32
33     $(CC) $(CFLAGS) -c $<
34
35
36
37 EDistance: main.o $(OBJECTS)
38
39     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
```

```

40
41
42
43 test: test.o $(OBJECTS)
44
45     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
46
47
48
49 EDistance.a: $(OBJECTS)
50
51     ar rcs $@ $^
52
53
54
55 clean:
56
57     rm *.o EDistance test EDistance.a
58
59 lint:
60
61     cpplint *.cpp *.hpp

```

main.cpp

```

1  // Copyright 2024 Nathan Khoury and Ishan Phadke
2
3  #include <iostream>
4  #include <SFML/System.hpp>
5  #include "EDistance.hpp"
6
7  int main(int argc, char* argv[]) {
8      std::string str1, str2;
9      // read strings from stdin
10     std::cin >> str1;
11     std::cin >> str2;
12     // create EDistance object using input strings
13     EDistance ed(str1, str2);
14     // begin clock
15     sf::Clock clock;
16     // print opt distance in opt[0][0]
17     std::cout << "Edit distance = " << ed.optDistance() << std::endl;
18     std::cout << ed.alignment() << std::endl;
19     sf::Time t = clock.getElapsedTime();
20     // display elapsed time
21     std::cout << "Execution time is " << t.asSeconds() << " seconds" << std::endl;
22     return 0;
23 }

```

EDistance.hpp

```

1  // Copyright 2024 Nathan Khoury and Ishan Phadke
2  #include <iostream>
3  #include <string>
4  #include <vector>
5
6  class EDistance {
7  public:
8      // value constructor allocates a matrix based off of two input strings
9      EDistance(std::string x, std::string y);
10     // returns the minimum value of three integer arguments
11     static int min3(int a, int b, int c) {
12         int& d = a < b ? a : b;
13         return d < c ? d : c;
14     }
15     // returns a penalty cost of 0 if characters are the same, 1 otherwise
16     static int penalty(char a, char b) {
17         return a == b ? 0 : 1;

```

```

18     }
19     // returns the optimal edit cost in _opt[0][0] after filling out the matrix
20     int optDistance();
21     // traces back the matrix and finds the optimal path from the calculated values
22     std::string alignment();
23     // helper function to print the matrix in the terminal
24     void dump();
25
26 private:
27     std::vector<std::vector<int>> _opt;
28     std::string _x, _y;
29     size_t _N, _M;
30     bool isSolved;
31     std::string printAlignment(char a, char b, int pen);
32 };

```

EDistance.cpp

```

1  // Copyright 2024 Nathan Khoury and Ishan Phadke
2
3  #include <iomanip>
4  #include <sstream>
5  #include <exception>
6  #include "EDistance.hpp"
7
8  EDistance::EDistance(std::string x, std::string y)
9      : _x(x), _y(y), _N(y.size()), _M(x.size()), isSolved(false) {
10     // string _y should be along the x-axis (same as pdf)
11     // string _x should be along the y-axis (same as pdf)
12     _opt.resize(_M + 1, std::vector<int>(_N + 1, 0));
13 }
14
15 int EDistance::optDistance() {
16     if (!isSolved) {
17         for (size_t i = 0; i < _M; i++) { // right column
18             _opt[i][_N] = 2 * (_M - i);
19         }
20         for (size_t j = 0; j < _N; j++) { // bottom row
21             _opt[_M][j] = 2 * (_N - j);
22         }
23         for (int j = _N - 1; j >= 0; j--) {
24             for (int i = _M - 1; i >= 0; i--) {
25                 _opt[i][j] = min3(_opt[i + 1][j + 1] +
26                     penalty(_x.at(i), _y.at(j)), // diagonal + penalty
27                     _opt[i + 1][j] + 2, // down + gap
28                     _opt[i][j + 1] + 2); // right + gap
29             }
30         }
31     }
32     isSolved = true;
33     return _opt[0][0];
34 }
35
36 std::string EDistance::alignment() {
37     size_t i = 0;
38     size_t j = 0;
39     std::stringstream ss;
40     while (i < _M || j < _N) {
41         if (i < _M && j < _N &&
42             _opt[i][j] == _opt[i + 1][j + 1] + penalty(_x.at(i), _y.at(j))) {
43             ss << printAlignment(_x.at(i), _y.at(j), penalty(_x.at(i), _y.at(j)));
44             i++;
45             j++;
46         } else if (i < _M && _opt[i][j] == _opt[i + 1][j] + 2) {
47             ss << printAlignment(_x.at(i), '-', 2);
48             i++;
49         } else if (j < _N && _opt[i][j] == _opt[i][j + 1] + 2) {

```

```

50         ss << printAlignment('-', _y.at(j), 2);
51         j++;
52     } else {
53         throw std::invalid_argument("Error in alignment\n");
54     }
55 }
56
57 return ss.str();
58 }
59
60 void EDistance::dump() {
61     std::cout << std::setw(5) << std::left << ' ';
62     // print out x-axis string's characters
63     for (size_t j = 0; j < _N; j++)
64         std::cout << std::setw(5) << std::left << _y.at(j);
65     // print out '-' for the extra
66     std::cout << std::setw(5) << '-' << std::endl;
67     for (size_t i = 0; i < _M + 1; i++) {
68         // i = row to print out
69         if (i >= _M) {
70             // if last row print the '-' character
71             std::cout << std::setw(5) << std::left << '-';
72         } else if (i < _M) {
73             // otherwise if not first row, print the character for the row
74             std::cout << std::setw(5) << std::left << _x.at(i);
75         }
76         for (size_t j = 0; j < _N + 1; j++) {
77             // j = collumn to print out
78             std::cout << std::setw(5) << std::left << _opt[i][j];
79         }
80         std::cout << std::endl;
81     }
82 }
83
84 std::string EDistance::printAlignment(char a, char b, int pen) {
85     std::stringstream line;
86     line << a << ' ' << b << ' ' << pen << '\n';
87     return line.str();
88 }

```

7.7 Unit Testing

The unit testing for this project was, in my opinion, much more challenging than the previous ones. I would credit this not just to the fact that the concept was harder to understand initially, but also the fact that there was a clear necessity to create some of your own sequence alignments with manually calculated answers. This is not only decently time consuming, but also prone to human error and could overall just prove to be a waste of time if it is redundant. It was definitely hard to cover all of our bases but I am very thankful that I was partnered with Ishan because he was not only very active and receptive to information regarding this project, but we made a pretty great team when it came to designing the tests. The tests verify that the functions `EDistance::optDistance()`, `EDistance::alignment()`, `EDistance::min3()`, and `EDistance::penalty()` yield appropriate results consistently. It was also challenging to test some of the more obscure cases since there are multiple correct answers for most sequence alignments, so it would be unwise to test for very specific results regarding the string outputted from `EDistance::alignment()` as it would generally fail good implementations that chose different, equally viable, paths. The next page presents the tests we implemented for this project.

```

1 // Copyright 2024 Nathan Khoury and Ishan Phadke
2
3 #include "EDistance.hpp"
4 #define BOOST_TEST_DYN_LINK
5 #define BOOST_TEST_MODULE PS5_EDistance_Testing
6 #include <boost/test/unit_test.hpp>
7
8 BOOST_AUTO_TEST_CASE(cost) {
9     std::string s1("CGGAT"), s2("ATACGGA");
10    EDistance ed(s1, s2);
11    BOOST_REQUIRE_EQUAL(ed.optDistance(), 8);
12 }
13
14 BOOST_AUTO_TEST_CASE(zeroCostValue) {
15     std::string s1("AT"), s2("AT");
16     EDistance ed(s1, s2);
17     BOOST_REQUIRE_EQUAL(ed.optDistance(), 0);
18 }
19
20 BOOST_AUTO_TEST_CASE(zeroCostAlignment) {
21     std::string s1("AT"), s2("AT");
22     EDistance ed(s1, s2);
23     std::string sol("A A O\nT T O\n");
24     BOOST_REQUIRE_EQUAL(ed.alignment(), sol);
25 }
26
27 BOOST_AUTO_TEST_CASE(penalty) {
28     BOOST_REQUIRE_EQUAL(EDistance::penalty('a', 'b'), 1);
29     BOOST_REQUIRE_EQUAL(EDistance::penalty('A', 'A'), 0);
30     BOOST_REQUIRE_EQUAL(EDistance::penalty('a', 'A'), 1);
31     BOOST_REQUIRE_EQUAL(EDistance::penalty('a', '-'), 1);
32     BOOST_REQUIRE_EQUAL(EDistance::penalty(' ', 'b'), 1);
33     BOOST_REQUIRE_EQUAL(EDistance::penalty('G', 'G'), 0);
34 }
35
36 BOOST_AUTO_TEST_CASE(min3) {
37     BOOST_REQUIRE_EQUAL(EDistance::min3(6, 4, 1), 1);
38     BOOST_REQUIRE_EQUAL(EDistance::min3(3252364, 32124434, 11111), 11111);
39     BOOST_REQUIRE_EQUAL(EDistance::min3(0, 2, 0), 0);
40     BOOST_REQUIRE_EQUAL(EDistance::min3(1, 0, 1), 0);
41     BOOST_REQUIRE_EQUAL(EDistance::min3(99, 22, -1), -1);
42 }
43
44 BOOST_AUTO_TEST_CASE(startGap) {
45     std::string s1("aaattt"), s2("ttt");
46     EDistance ed(s1, s2);
47     ed.optDistance();
48     std::string sol("a - 2\na - 2\nnt t O\nnt t O\nnt t O\n");
49     BOOST_REQUIRE_EQUAL(ed.alignment(), sol);
50 }
51
52 BOOST_AUTO_TEST_CASE(oneBlank) {
53     std::string s1("A"), s2("");
54     EDistance ed(s1, s2);
55     std::string sol("A - 2\n");
56     BOOST_REQUIRE_EQUAL(ed.optDistance(), 2);
57     BOOST_REQUIRE_EQUAL(ed.alignment(), sol);
58 }
59
60 BOOST_AUTO_TEST_CASE(bothBlank) {
61     std::string s1(""), s2("");
62     EDistance ed(s1, s2);
63     BOOST_REQUIRE_EQUAL(ed.optDistance(), 0);
64 }
65
66 BOOST_AUTO_TEST_CASE(generalEditDistanceTest1) {

```

```

67     std::string s1("AABBCCDDEEFFGG"), s2("FFGGAABBCCDDEE");
68     EDistance ed(s1, s2);
69     BOOST_REQUIRE_EQUAL(ed.optDistance(), 14);
70 }
71
72 BOOST_AUTO_TEST_CASE(generalEditDistanceTest2) {
73     std::string s1("hellohelloworldworld"), s2("helloworld");
74     EDistance ed(s1, s2);
75     BOOST_REQUIRE_EQUAL(ed.optDistance(), 20);
76 }
77
78
79 BOOST_AUTO_TEST_CASE(generalEditDistanceTest3) {
80     std::string s1("ATGCATCGGGTCACTCATGGATTTCACACGTCCAGGTCCAGGTCA"),
81     s2("TATTATAGGCACTTCAGCCCCATCGCGATCTTGAAGGGCATTGAGCATTAC");
82     EDistance ed(s1, s2);
83     BOOST_REQUIRE_EQUAL(ed.optDistance(), 38);
84 }
85
86 BOOST_AUTO_TEST_CASE(onlyPathUpEditDistanceTest) {
87     std::string s1("AAAAGG"),
88     s2("GGGAAAAGG");
89     EDistance ed(s1, s2);
90     std::string sol("- G 2\n- G 2\n- G 2\nA A 0\nA A 0\nA A 0\nG G 0\nG G 0\n");
91     BOOST_REQUIRE_EQUAL(ed.optDistance(), 8);
92 }
93
94 BOOST_AUTO_TEST_CASE(wrongDir) {
95     std::string s1("AACAGTTACC"),
96     s2("TAAGGTCA");
97     EDistance ed(s1, s2);
98     std::string sol("A T 1\nA A 0\nC - 2\nA A 0\nG G 0\nT G 1\nT T 0\nA - 2\nC C 0\nC A 1\n");
99     BOOST_REQUIRE_EQUAL(ed.optDistance(), 7);
100     BOOST_REQUIRE_EQUAL(ed.alignment(), sol);
101 }
102
103 BOOST_AUTO_TEST_CASE(noThrowGeneralConstructorCases) {
104     std::string s1a("hello"), s1b("world");
105     BOOST_REQUIRE_NO_THROW(EDistance ed1(s1a, s1b)); // general strings
106     std::string s2a("TESTINGAVERYLONGSTRING"), s2b("WITHANOTHERVERYLONGSTRING");
107     BOOST_REQUIRE_NO_THROW(EDistance ed2(s2a, s2b)); // long strings
108     std::string s3a("ATTAGACCAGAGCAGCGGCA"), s3b("ATGACTCAGCTAGACC");
109     BOOST_REQUIRE_NO_THROW(EDistance ed3(s3a, s3b)); // long strings with DNA characters
110     std::string s4a("A"), s4b("CTGCGTCTGCGTTCTGCTGTGC");
111     BOOST_REQUIRE_NO_THROW(EDistance ed4(s4a, s4b)); // one short one long
112 }
113
114 BOOST_AUTO_TEST_CASE(noThrowConstructorOneBlank) {
115     std::string s1(""), s2("ATAAGCAG");
116     BOOST_REQUIRE_NO_THROW(EDistance ed1(s1, s2));
117     std::string s3("ATAAGCAG"), s4("");
118     BOOST_REQUIRE_NO_THROW(EDistance ed2(s3, s4));
119 }
120
121 BOOST_AUTO_TEST_CASE(noThrowConstructorBothBlank) {
122     std::string s1(""), s2("");
123     BOOST_REQUIRE_NO_THROW(EDistance ed(s1, s2));
124 }
125
126 BOOST_AUTO_TEST_CASE(noThrowOptDist) {
127     std::string s1("AABBCCDDEEFFGG"), s2("FFGGAABBCCDDEE");
128     EDistance ed1(s1, s2);
129     BOOST_REQUIRE_NO_THROW(ed1.optDistance());
130     std::string s3("ATAG"), s4("");
131     EDistance ed2(s3, s4);
132     BOOST_REQUIRE_NO_THROW(ed2.optDistance());

```

```

133     std::string s5(""), s6("");
134     EDistance ed3(s5, s6);
135     BOOST_REQUIRE_NO_THROW(ed3.optDistance());
136 }
137
138 BOOST_AUTO_TEST_CASE(noThrowAlignmentAfterOptDist) {
139     std::string s1("AABBCCDDEEFFGG"), s2("FFGGAABBCCDDEE");
140     EDistance ed1(s1, s2);
141     ed1.optDistance();
142     BOOST_REQUIRE_NO_THROW(ed1.alignment());
143     std::string s3("ATAG"), s4("");
144     EDistance ed2(s3, s4);
145     ed2.optDistance();
146     BOOST_REQUIRE_NO_THROW(ed2.alignment());
147     std::string s5(""), s6("");
148     EDistance ed3(s5, s6);
149     ed3.optDistance();
150     BOOST_REQUIRE_NO_THROW(ed3.alignment());
151 }

```

8 PS6: RandWriter

8.1 Functionality

The RandWriter program is a program which takes a sample input, which is redirected to `stdin` from a text file, and derives new *randomly generated text* based on the input. This is done by following a **Markov model** which is a mathematical model which provides methodology for computing the probability of characters that follow an input string. A Markov model of *k-order* is said to take an input string, also known as a **k-gram**, of size **k** and use character probabilities from the input text to determine the *next* character. The next character becomes part of the next kgram which repeats until the newly generated output text is of size **L** specified by the user.

Essentially, the program will parse an entire input text, starting with the first *k* characters. A k-gram of length *k* is created and stored within the program. Only the *first character* immediately following the k-gram which is **not** part of the k-gram itself is recorded in correlation to the k-gram. The character that was just recorded becomes part of the k-gram, and the character at the front of the k-gram is removed to maintain a length of *k* characters. Only new instances of k-grams are recorded, but multiple instances of characters that follow k-grams should be stored as a total number of occurrences, such that the program can see how many times a given character had followed a specific k-gram. To further exemplify the functionality of an order-*k* markov model, the list below shows example text (200 characters long) generated by different order Markov models with the input text being the song *love story* by Taylor Swift

Order-1 takes a k-gram that is one character long, storing it internally if it is the first instance of it, and keeps track of which character followed that exact **1-gram**

```
1 // Order-1
2 Wous a Isous e condou
3 Woh
4 Rowaund I payou t s"
5 Lilco athifres thesse f I yor knd ando waind we proutsavesare
6 Ese "
7 Rou, in'mme
8 Ong snd Rony, dy, s ked dind boheveale tacery f Roust"
9 Ro Rome "Ye idde t
```

Order-2 takes a k-gram that is two characters long, storing it internally if it is the first instance of it, and keeps track of which character followed that exact **2-gram**

```
1 // Order-2
2 We st grom Jult be love yout theyess
3 I were oh, thead the we's me
4 Is rung the your cryna throme fircas re saway, som Juliet and I'lles
5 It's tryna rund try, "Yess
6 Is fory, "Starow
7
8 Romeo, sto is rince
```

Order-3 takes a k-gram that is three characters long, storing it internally if it is the first instance of it, and keeps track of which character followed that exact **3-gram**

```
1 // Order-3
2 We waith yound
3 My faiting
4 When I first said, "Yes"
5 Oh, oh, oh oh oh
6
7 I love mess
8 It's a love story, "Marry me, Juliet"
9 And say, "Yes"
10 Oh, oh, oh, oh
11
12 'Cause my head? I said
13
```

```
14 Romeo, I sneak out you
15 We
```

Order-5 takes a k-gram that is five characters long, storing it internally if it is the first instance of it, and keeps track of which character followed that exact **5-gram**

```
1 // Order-5
2 We were throwin' pebbles
3 And said, "Stay away from Juliet"
4 And I said, "Stay away from Juliet"
5 But you make your way throwin' pebbles
6 And say, "Yes"
7
8 So I sneak out of town for you, but you on the pri
```

Order-7 takes a k-gram that is seven characters long, storing it internally if it is the first instance of it, and keeps track of which character followed that exact **7-gram**

```
1 // Order-7
2 We were both young when I first saw you
3 I close your way through the crowd
4 And said, "Marry me, Juliet
5 You'll be the princess
6 It's a love story, baby, just say, "Hello"
7 Little did I know
8
9 That you wer
```

It is made fairly evident by these examples that the higher the order of the Markov model, the closer to the source text that the randomly generated text will seem. This is due to the fact that as the k-grams get larger, naturally the k-grams become way more specific and decrease in quantity. This gives the program less options to compare when trying to generate random text. Given a high enough Markov model (the tipping point for each input text varies), it will only be possible to yield the original text when running.

For this program, there are a few additional things worth mentioning. First of all, the randomly generated text will always begin with the first initial k-gram. Secondly, the input text is treated as if it were circular. This means that when the k-grams reach the end of the input text, and would have otherwise no other characters to maintain the k length of the k-gram, it will draw from the start of the input text again. Finally, a Markov model of order-0 *is possible*. Since the k-grams are of length 0, however, this means that the characters which are generated in the final output are determined by the frequency that the character appeared within the input text, rather than the probability being determined by the k-gram it follows and said character's respective frequency in that context. With all of this in mind, proceeding to section 8.2 will show you how this was all implemented within my RandWriter program.

8.2 Features

The program starts by verifying valid input from the user and converting the provided values into `size_t` variables for usage later. The user provides first a value `k` which tells the program what order of the Markov model we will be generating, and a second value `L` which informs the program how many characters long to make the final generated text (it is assumed that $L \geq K$). After this, a while loop utilizes `std::getline` to read the entire input into a `std::stringstream` named `text`. During this we also store the first k characters within `firstK` for usage later. Once the loop is finished, we create an object using the custom class I made called `RandWriter`, and we provide the input text by using `text.str()` to access the string.

Within the `RandWriter` constructor, it is verified that `k > text.size()`, since we cannot generate k-grams in this case. If all is well, we proceed to the main `for` loop within the constructor. This loop creates a k-gram starting at our current position in the input text (at first, this would be the first character) and going `k` characters out, with the resulting string being stored in a `kgram` stringstream. If it is not an order-0 model, this becomes our key,

otherwise the key should be an empty string "".

At this point it is important to note why we are concerned with keys. The way that our k-grams, as well as the characters the follow *and* respective frequencies of each character, are stored is by a private container in the class named `_kgrams` of type `std::map<std::string, std::map<char, size_t>`. This allows us to store our `kgram` as the first string, which we can use to access a secondary map which uses a given character that followed the `kgram` as a key to access the frequency of said character. This proved to be a very organized way to manage this sort of data, as we can very carefully store every `kgram`, and the characters that follow, appropriately and in the same place.

Returning back to the constructor now, we use our key `kgram` to create a new entry in the `_kgrams` map if one has not already been made. Then, we check what character comes next immediately *after* our current `kgram` ends. We will take this character and add it as a key to the submap within `_kgrams` if it has not been added already. Regardless, the value for this entry in the submap should now reflect the amount of times that this character has followed the `kgram` so far (this is achieved by setting the value to 1 if it is a new entry, or by incrementing the value by one if it is not). We then will add this character to a private `std::string` variable `_alphabet` which will be helpful for assuring valid input when randomly generating the text. At this point, the constructor is finished and we will return to `main()` to proceed.

Now that all of the `kgrams` have been stored internally, and the characters that followed with corresponding frequencies, all that is left to do is generate the new output text. We begin this process by calling `RandWriter::generate()` and providing the initial `kgram` which began the original input, and the `L` number of characters to write the final text until. Valid arguments are first assured in `generate()`, then we add our initial `kgram` to a running stringstream which will store our text as it generates. A while loop with the condition `(text.str().size() < L)` will iterate until we have a final output string of suitable length. Until that point, however, we generate a new character by calling `RandWriter::kRand()` and providing it our current `kgram`. After that, we update our current `kgram` to add the newly generated character to the back, and remove the oldest character from the front.

The function `kRand()` is where most of the magic happens, but it is also the most complex for that reason. After validating the length of the argument `kgram`, and assuring that it *does* exist in the map as an entry, we first obtain the amount of times that the `kgram` occurred in the input text by calling one of the two overloads for `RandWriter::freq()`. This first overload accepts a single string `kgram`, validates the argument `kgram`, and then adds up how many characters followed the `kgram`. This works since the text is circular which means that *any* `kgram` will *always* be followed by a character. Therefore, if we add up all of the characters that followed (which we tracked the quantities of as the value of the submap), we can easily determine the times that the `kgram` was found in the input text.

Once we have the number of times that the `kgram` occurred, we will generate a random number between 1 and the number of occurrences. This is accomplished by using an object from the `<random>` library, `std::uniform_int_distribution<int>`, to create a range of 1 to `max` (where `max` is our number of occurrences). We can then pass a private member variable named `gen` to this distribution to yield a value between 1 and `max`. The member variable `gen` is a `minstd_rand0` random number generator from the `<random>` library and it is initialized in the `RandWriter` constructor by using the `<chrono>` library to provide a unique seed. The initialization is displayed below, as the access to our unique seed may be a little... wordy...

```
1 /*
2  * gen is our minstd_rand0
3  * this retrieves the time since January 1st, 1970 in terms of the internal representation
4  * of the time within the std::chrono::duration's internal period
5  * interval which is not exactly a unit of time itself
6  * which proves to be a very effective unique seed for the generator
7  */
8 gen(std::chrono::system_clock::now().time_since_epoch().count())
```

I was torn between using `std::minstd_rand0` and `std::mt19937`, but I ended up using the former since it is a faster generator, as opposed to `std::mt19937` which produces higher quality random numbers. I felt that the time to generate was more crucial in this program, and I was fairly certain that the random numbers produced would be more than sufficient.

Returning back to `kRand()` with out new random number already within the range we need it, we store the number in `randNum` and utilize `std::find_if()` from the `<algorithm>` library. Below is the implementation for `std::find_if()` along with the lambda I created for it, which I believe to be one of my most complex, thoughtful, and useful ones yet.

```
1 auto selected = std::find_if(_alphabet.begin(), _alphabet.end(),
2     [&randNum, kgram, this](char ch){
3         randNum -= this->freq(kgram, ch);
4         if (randNum <= 0) return true;
5         return false;
6     });
```

Essentially, to determine what character we will choose based off our random number, I decided to take the random number generated which is no greater than the amount of times that `kgram` occurred, and subtract the frequencies of each character that followed the `kgram` individually. The custom lambda accomplishes this, then checks if the remaining value from the original random number is less than or equal to zero. If so, this should be the character that we choose, so we return a pointer to the character in `_alphabet`. To determine individual character frequency given the `kgram`, we utilize the second overload for `RandWriter::freq()` which accepts a `kgram` string and a character to check for. If the character is in the submap we return its value, otherwise we return zero since it must not have occurred in the input text. The value pointed to by `std::find_if()` by the end is our final character which we can dereference and return to `generate()`.

Within `generate()`, the character is added to the stringstream, and `generate` iterates again as was specified earlier. Once the final output string is of length `L`, it is extracted from the stringstream and returned, then promptly outputted to the terminal. The program terminates, and the user can now read whatever silly, strange, or incoherent text that has been produced by this process.

8.3 Personal Reflection

8.3.1 The Struggles

Working with maps was certainly a challenge. Not only had I never worked with a map before, but I was instantly trying to learn to work with a map *within* a map. An issue was that, within certain constant functions such as the two overloads for `RandWriter::freq()`, I was unable to use the index operator `[]` to reach within the map. This is because, naturally, the `[]` operator is not `const` which means that we may not utilize it when within a `const` function. Instead, after some head-scratching, I was required to use the `.at()` method for accessing values in the map.

Other than that I had one other major hurdle which was that an out of range error was occurring within both of my `RandWriter::freq()` implementations. This turned out to be due to the fact that I was not checking if `kgram` was actually within the map or not. This meant that a call such as `auto kg = _kgrams.at(kgram)` would reach randomly for a key-value pair that did not exist. After implementing the check for if `kgram` exists or not, it was pretty much smooth sailing (thankfully)

8.3.2 What I Learned

I actually found a lot of value and good practice in the exception throwing, and edge-case handling, which was a big focus in this project. Throwing exceptions I felt were applicable from the `<exception>` library not only made sense from a design standpoint, but it helped to make debugging so much easier, as I could tell where and why my exceptions were being thrown. I also learned how to not only iterate across all the elements of a map using a ranged `for` loop, but I also learned how to iterate across all of the elements in a map within a map.

Not to mention that I actually learned how to use a map, and consequently the pros and cons of using a map, during this project.

8.3.3 The Takeaway

I felt that this project was a really fun way to learn about maps and the `<random>` library. I feel that this project was maybe the most proud I've been of any of my end results. I feel that not only was I very prepared ahead of time to handle this assignment (I spent a very long time interpreting the task, understanding the Markov model, and planning my approach before I even started writing any code), but I feel I noticed a lot of growth during my skills in this project alone. Comparing the organization and sophistication of this program to that of PS2 (section 4) or PS3 (section 5), it really displays how far I've come in terms of the quality of the code I'm writing. As always I will acknowledge: this program is certainly this is imperfect, and my approach to some problems may be rudimentary and inefficient in some regards that maybe I still have not realized at the time of writing this. Despite this, I think my growth since the start of this semester is really impressive in my opinion. Regardless of any takeaway, it's pretty fun to use this program with friends to generate goofy recreations of familiar texts!

8.4 Running The Program

The program, like the rest, comes with a provided Makefile which will produce an executable named **TextWriter** and a static library **TextWriter.a**. Command line input of the following format can be used to run the program

```
1 ./TextWriter k L < <filename-with-extention>
```

The first argument `k` determines the order of the Markov model that will be produced, and the second argument `L` sets the length in terms of number of characters that the newly generated output text should be. The third argument uses input redirection to redirect the contents of a specified file to `stdin` to allow the program to easily read large existing pieces of text.

8.5 Proof of Functionality

Section 8.1 showed some example outputs that my own program outputted when I supplied `L = 200` and a variable `k` value with the input text being Taylor Swift's song *love story*. Below shows the contents of a file `woodchuck.txt`, with some examples produced with Markov models of differing *k-order* on the next page.

```
1 // contents of woodchuck.txt (this comment is not within the original file)
2 How much wood could a woodchuck chuck
3 If a woodchuck could chuck wood?
4 As much wood as a woodchuck could chuck,
5 If a woodchuck could chuck wood.
```

Figure 15: Order- k outputs using woodchuck.txt as input

```
● nkhoury@nathan-laptop:~/comp4/ps6$ ./TextWriter 0 100 < woodchuck.txt
u u?o facdosccda u oooacwuucwwwcocu wdu odudoo
dlu ?ca kalhscIc c olhouocow cdu d ol owd ocuAck
```

(a) Order-0, $L = 100$

```
● nkhoury@nathan-laptop:~/comp4/ps6$ ./TextWriter 1 100 < woodchuck.txt
Hod couck,
If ch woucodchuld chuld a a wodchucod ck wod?
If as chuldck ck a w ch wouck a chuck,
If a
```

(b) Order-1, $L = 100$

```
● nkhoury@nathan-laptop:~/comp4/ps6$ ./TextWriter 2 100 < woodchuck.txt
How muck could could could as muck
If a wood.How muchuck chuch wood?
As muck chuch wood chuchuck cou
```

(c) Order-2, $L = 100$

```
● nkhoury@nathan-laptop:~/comp4/ps6$ ./TextWriter 4 100 < woodchuck.txt
How much woodchuck,
If a wood?
As much woodchuck could chuck could chuck chuck could chuck
If a wood
```

(d) Order-4, $L = 100$

8.6 Source Code

The `main()` driver can be found in `TextWriter.cpp` rather than `main.cpp` this time, with the supporting class in `RandWriter.hpp` and `RandWriter.cpp`. A Makefile also accompanies the program as always. Below is the source code for my implementation of the `RandWriter` program

Makefile

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -lboost_unit_test_framework
4 # Your .hpp files
5 DEPS = RandWriter.hpp
6 # Your compiled .o files
7 OBJECTS = RandWriter.o
8 # The name of your programs
9 PROGRAM = TextWriter test
10
11 .PHONY: all clean lint
12
13 all: $(PROGRAM) TextWriter.a
14
15 # Wildcard recipe to make .o files from corresponding .cpp file
16 %.o: %.cpp $(DEPS)
17     $(CC) $(CFLAGS) -c $<
18
19 TextWriter: TextWriter.o $(OBJECTS)
20     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
21
22 test: test.o $(OBJECTS)
23     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
24
25 TextWriter.a: $(OBJECTS)
26     ar rcs $@ $^
27
28 clean:
29     rm *.o TextWriter test TextWriter.a
30 lint:
31     cpplint *.cpp *.hpp
```

TextWriter.cpp

```
1 // Copyright 2024 Nathan Khoury
2 #include "RandWriter.hpp"
3
4 int main(int argc, char* argv[]) {
5     if (argc < 2) {
6         // verify that the required command line argumes were provided
7         std::cerr << "Insufficient inputs supplied to command line"
8             << std::endl;
9     }
10    // convert arguments from strings to numerical values
11    size_t k = std::stoi(argv[1]);
12    size_t L = std::stoi(argv[2]);
13
14    // read text from redirected input, store in a singular string
15    std::stringstream text;
16    std::string buffer, firstK = "";
17    bool firstIter = true;
18    while (std::getline(std::cin, buffer, '\0')) { // keep newlines, read until end of input
19        text << buffer;
20        if (firstIter && k != 0) {
21            firstK = buffer.substr(0, k);
22            firstIter = false;
23        }
24    }
25
26    // create RandWriter object
27    RandWriter rw(text.str(), k);
28    std::cout << rw.generate(firstK, L) << std::endl;
29
30    return 0;
31 }
```

RandWriter.hpp

```
1 // Copyright 2024 Nathan Khoury
2 #include <iostream>
3 #include <sstream>
4 #include <string>
5 #include <map>
6 #include <random>
7 #include <chrono>
8 #include <algorithm>
9 #include <exception>
10
11 class RandWriter {
12 public:
13     RandWriter(const std::string& text, size_t k);
14
15     size_t orderK() const;
16
17     int freq(const std::string& kgram) const;
18
19     int freq(const std::string& kgram, char c) const;
20
21     char kRand(const std::string& kgram);
22
23     std::string generate(const std::string& kgram, size_t L);
24
25     friend std::ostream& operator<<(std::ostream& os, RandWriter& rw);
26
27 private:
28     // map of kgrams with inner_maps of subsequent characters and their frequencies
29     std::map<std::string, std::map<char, size_t>> _kgrams;
30     size_t _k;
31     size_t _textLen;
32     std::string _alphabet;
```

```

33     std::minstd_rand0 gen;    // rng (faster than mt19937 but worse number generation)
34     // trim indexes to be circular
35     size_t circular(const size_t& i) { return i % _textLen; }
36     // insert a key i
37     void insert(std::string key, char ch);
38     // add character to running alphabet if new character
39     void alphabet(char ch);
40     // check if the kgram is in the map _kgrams
41     bool inMap(const std::string& kgram) const;
42     // return the randomly generated number
43     int rng(const int& max);
44 };
45
46 std::ostream& operator<<(std::ostream& os, RandWriter& rw);

```

RandWriter.cpp

```

1  // Copyright 2024 Nathan Khoury
2  #include "RandWriter.hpp"
3
4  RandWriter::RandWriter(const std::string& text, size_t k)
5      : _k(k), _textLen(text.size()), _alphabet(""),
6      gen(std::chrono::system_clock::now().time_since_epoch().count()) {
7      if (_k > text.size()) throw std::invalid_argument("Bad constructor input: k < L");
8      for (size_t i = 0; i < text.size(); i++) {
9          // create a kgram starting at i, going k characters out
10         std::stringstream kgram;
11         std::string key = "";
12         for (size_t j = 0; j < _k; j++)
13             kgram << text.at(RandWriter::circular(i + j));
14         if (_k != 0)
15             key = kgram.str();
16         // attempt to add to map, with copy of blank submap (ignored if already present)
17         std::map<char, size_t> submap;
18         if (!inMap(key))
19             _kgrams[key] = submap;
20
21         // check the character at kgram + 1
22         char ch = text.at(RandWriter::circular(i + _k));
23         RandWriter::insert(key, ch);
24         RandWriter::alphabet(ch);
25     }
26 }
27
28 size_t RandWriter::orderK() const {
29     return _k;
30 }
31
32 int RandWriter::freq(const std::string& kgram) const {
33     // verify kgram is of length k
34     if (kgram.size() != _k) {
35         std::string errMsg = "length_error: kgram \"" + kgram
36             + "\" is not of length " + std::to_string(_k);
37         throw std::length_error(errMsg);
38     }
39     // return length of text if Order 0
40     if (_k == 0) return _textLen;
41     if (!inMap(kgram)) return 0;
42     int count = 0;
43     auto kg = _kgrams.at(kgram);
44     for (auto const& ent : kg)
45         count += ent.second;
46
47     return count;
48 }
49
50 int RandWriter::freq(const std::string& kgram, char c) const {

```

```

51     if (kgram.size() != _k) {
52         std::string errMsg = "length_error: kgram \"" + kgram
53             + "\" is not of length " + std::to_string(_k);
54         throw std::length_error(errMsg);
55     }
56     if (!inMap(kgram)) return 0;
57     auto const& kg = _kgrams.at(kgram);
58     for (auto const& ent : kg)
59         if (ent.first == c) return _kgrams.at(kgram).at(c);
60     return 0;
61 }
62
63 char RandWriter::kRand(const std::string& kgram) {
64     if (kgram.size() != _k) {
65         std::string errMsg = "length_error: kgram \"" + kgram
66             + "\" is not of length " + std::to_string(_k);
67         throw std::length_error(errMsg);
68     } else if (!inMap(kgram) && _k != 0) {
69         std::string errMsg = "invalid_argument: kgram \"" + kgram
70             + "\" does not exist in the map" + std::to_string(_k);
71         throw std::invalid_argument(errMsg);
72     }
73     // get total number of instances of kgram
74     int tot = freq(kgram);
75     // generate random number between 0 and kgram instances
76     int randNum = rng(tot);
77     // subtract the character's instances from the random number
78     auto selected = std::find_if(_alphabet.begin(), _alphabet.end(),
79         [&randNum, kgram, this](char ch){
80             randNum -= this->freq(kgram, ch);
81             if (randNum <= 0) return true;
82             return false;
83         });
84     if (selected != _alphabet.end())
85         return *selected;
86     throw std::range_error("range_error: left kRand() without selecting a character");
87 }
88
89 std::string RandWriter::generate(const std::string& kgram, size_t L) {
90     std::stringstream text;
91     if (L <= _k) return kgram;
92     if (kgram.size() != _k) {
93         std::string errMsg = "length_error: kgram \"" + kgram
94             + "\" is not of length " + std::to_string(_k);
95         throw std::length_error(errMsg);
96     }
97     text << kgram;
98     std::string currkg = kgram;
99     while (text.str().size() < L) {
100         text << kRand(currkg); // add randomly generated character to text
101         currkg = text.str().substr(text.str().size() - _k); // get next kgram
102     }
103     return text.str();
104 }
105
106 void RandWriter::insert(std::string key, char ch) {
107     if (_kgrams[key].count(ch)) { // has character occurred yet
108         // increment value
109         _kgrams[key][ch] += 1;
110     } else {
111         // first instance, new entry: key = char, value = 1
112         _kgrams[key][ch] = 1;
113     }
114 }
115
116 void RandWriter::alphabet(char ch) {

```

```

117     if (_alphabet.find(ch) == std::string::npos)
118         _alphabet += ch;
119 }
120
121 bool RandWriter::inMap(const std::string& kgram) const {
122     for (auto const &ent : _kgrams) {
123         if (ent.first == kgram)
124             return true;    // found the kgram, it exists in the map
125     }
126     return false;    // doesn't exist
127 }
128
129 int RandWriter::rng(const int& max) {
130     std::uniform_int_distribution<int> range(1, max);
131     return range(gen);
132 }
133
134 std::ostream& operator<<(std::ostream& os, RandWriter& rw) {
135     // credit to blackboard pdf for map iteration
136     for (auto const &ent1 : rw._kgrams) {
137         auto const &outer_key = ent1.first;
138         auto const &inner_map = ent1.second;
139         os << outer_key << ": (" << rw.freq(outer_key) << ') ' << std::endl;
140         for (auto const &ent2 : inner_map) {
141             auto const &inner_key = ent2.first;
142             auto const &inner_value = ent2.second;
143             os << "\t" << inner_key << " - " << inner_value << std::endl;
144         }
145     }
146     std::cout << "Order " << rw.orderK() << " Markov model" << std::endl;
147     std::cout << "Alphabet: " << rw._alphabet << std::endl;
148     return os;
149 }

```

8.7 Unit Testing

Unit testing for the RandWriter program actually proved to be not so bad. I'm not sure if this is just because I've been improving at them, or maybe it really just was a simple task. The tests check for valid outputs for all of the public, non-helper functions. The overridden stream insertion operator was not tested since it was only to assist in debugging, the program does not utilize it. The tests also assert that inappropriate arguments to functions throw reasonable exceptions. Probably my most complex test is one where I provide the string "AB AC DEF" as input, and I utilize an Order-1 model to test if, after 1000 calls to `kRand()` with the k-gram "A" provided, that the only characters generated are 'B' and 'C'. It also tests that out of the 1000 'B' and 'C' characters, the amount of either is not more than 100 from the other. That is to say, the split should not be greater than 40% to 60% in either character's favor. This creates a reasonable amount of leeway for the random nature of the generation, but the generation of 1000 characters should be enough to reasonably meet this standard every time. Below is the `test.cpp` file which contains my tests for this assignment

test.cpp

```

1 // Copyright 2024 Nathan Khoury
2 #include <cmath>
3 #include "RandWriter.hpp"
4
5
6 #define BOOST_TEST_DYN_LINK
7 #define BOOST_TEST_MODULE PS6_RandWriter
8 #include <boost/test/unit_test.hpp>
9
10 BOOST_AUTO_TEST_CASE(orderK_1) {
11     RandWriter rw("test text", 1);
12     BOOST_REQUIRE_EQUAL(rw.orderK(), 1);

```

```

13     BOOST_REQUIRE_NO_THROW(rw.orderK());
14 }
15
16 BOOST_AUTO_TEST_CASE(orderK_15) {
17     RandWriter rw("12345abcde67890", 15);
18     BOOST_REQUIRE_EQUAL(rw.orderK(), 15);
19     BOOST_REQUIRE_NO_THROW(rw.orderK());
20 }
21 BOOST_AUTO_TEST_CASE(orderK_0) {
22     RandWriter rw("text!!!", 0);
23     BOOST_REQUIRE_EQUAL(rw.orderK(), 0);
24     BOOST_REQUIRE_NO_THROW(rw.orderK());
25 }
26
27 BOOST_AUTO_TEST_CASE(kGreaterThanOrL) {
28     BOOST_REQUIRE_THROW(RandWriter rw("123", 4), std::logic_error);
29 }
30
31 BOOST_AUTO_TEST_CASE(kgram_freqGeneral) {
32     RandWriter rw("aabaa", 3);
33     BOOST_REQUIRE_EQUAL(rw.freq("aaa"), 2);
34     BOOST_REQUIRE_NO_THROW(rw.freq("aaa"));
35 }
36
37 BOOST_AUTO_TEST_CASE(kgram_freqSymbols) {
38     RandWriter rw("!!!!!!", 2);
39     BOOST_REQUIRE_EQUAL(rw.freq("!!!"), 7);
40     BOOST_REQUIRE_NO_THROW(rw.freq("!!!"));
41 }
42
43 BOOST_AUTO_TEST_CASE(kgram_freqOrder0) {
44     RandWriter rw(" unique text.", 0);
45     BOOST_REQUIRE_EQUAL(rw.freq(""), 13);
46     BOOST_REQUIRE_NO_THROW(rw.freq(""));
47 }
48
49 BOOST_AUTO_TEST_CASE(kgram_freqLongInput) {
50     RandWriter rw("pawns???speck specialspoken !spam! sports", 2);
51     BOOST_REQUIRE_EQUAL(rw.freq("sp"), 6);
52     BOOST_REQUIRE_NO_THROW(rw.freq("sp"));
53 }
54
55 BOOST_AUTO_TEST_CASE(kgram_freqBadInput) {
56     RandWriter rw("blahblahblah", 2);
57     BOOST_REQUIRE_THROW(rw.freq("a"), std::logic_error);
58 }
59
60 BOOST_AUTO_TEST_CASE(kgram_charFreqOrder1) {
61     RandWriter rw("hello", 1);
62     BOOST_REQUIRE_EQUAL(rw.freq("l", 'o'), 1);
63     BOOST_REQUIRE_NO_THROW(rw.freq("l", 'o'));
64 }
65
66 BOOST_AUTO_TEST_CASE(kgram_charFreqOrder2) {
67     RandWriter rw("man manual pam manage nam", 2);
68     BOOST_REQUIRE_EQUAL(rw.freq("ma", 'n'), 3);
69     BOOST_REQUIRE_NO_THROW(rw.freq("ma", 'n'));
70 }
71
72 BOOST_AUTO_TEST_CASE(kgram_charFreqOrder5) {
73     RandWriter rw("mmmmmm", 5);
74     BOOST_REQUIRE_EQUAL(rw.freq("mmmm", 'm'), 6);
75     BOOST_REQUIRE_NO_THROW(rw.freq("mmmm", 'm'));
76 }
77
78 BOOST_AUTO_TEST_CASE(kgram_charFreqOrder0) {

```

```

79     RandWriter rw("hopefully you pass this test :)", 0);
80     BOOST_REQUIRE_EQUAL(rw.freq("", ' '), 5);
81     BOOST_REQUIRE_NO_THROW(rw.freq("", ' '));
82 }
83
84 BOOST_AUTO_TEST_CASE(kgram_charFreqBadInput) {
85     RandWriter rw("-----", 5);
86     BOOST_REQUIRE_THROW(rw.freq(" ----- ", '-'), std::logic_error);
87 }
88
89 BOOST_AUTO_TEST_CASE(kgram_charFreqNoOccurrences) {
90     RandWriter rw("abcdefg", 3);
91     BOOST_REQUIRE_EQUAL(rw.freq("efg", 'h'), 0);
92     BOOST_REQUIRE_NO_THROW(rw.freq("efg", 'h'));
93 }
94
95 BOOST_AUTO_TEST_CASE(krand100PercentChanceGeneral) {
96     RandWriter rw("ABCDEFG", 2);
97     BOOST_REQUIRE_EQUAL(rw.kRand("CD"), 'E');
98     BOOST_REQUIRE_NO_THROW(rw.kRand("CD"));
99 }
100
101 BOOST_AUTO_TEST_CASE(krand100PercentChanceEdge) {
102     RandWriter rw("ABCDEFG", 1);
103     int GCount = 0;
104     for (int i = 0; i < 200; i++) {
105         char ch = rw.kRand("G");
106         if (ch == 'A')
107             GCount++;
108     }
109     BOOST_REQUIRE_EQUAL(GCount, 200);
110 }
111
112 BOOST_AUTO_TEST_CASE(krand50PercentChance) {
113     RandWriter rw("AB AC DEF", 1);
114     int BCount = 0;
115     int CCount = 0;
116     for (int i = 0; i < 1000; i++) {
117         char ch = rw.kRand("A");
118         if (ch == 'B') {
119             BCount++;
120         } else if (ch == 'C') {
121             CCount++;
122         } else {
123             std::stringstream errMsg;
124             errMsg << "Got character " << ch << " instead of B or C";
125             BOOST_FAIL(errMsg.str());
126         }
127     }
128     auto absDifference = std::abs(BCount - CCount);
129     BOOST_REQUIRE_LT(absDifference, 100);
130 }
131
132 BOOST_AUTO_TEST_CASE(generateOrder1) {
133     RandWriter rw("abcdefg", 1);
134     BOOST_REQUIRE_EQUAL(rw.generate("a", 7), "abcdefg");
135     BOOST_REQUIRE_NO_THROW(rw.generate("a", 7));
136 }
137
138 BOOST_AUTO_TEST_CASE(generateOrder5) {
139     RandWriter rw("1234567890", 5);
140     BOOST_REQUIRE_EQUAL(rw.generate("12345", 10), "1234567890");
141     BOOST_REQUIRE_NO_THROW(rw.generate("12345", 10));
142 }
143
144 BOOST_AUTO_TEST_CASE(generate100) {

```

```
145     RandWriter rw("AABCBBCAACCCBABCBAACCBACBACBACAACBBBBCCBCBCA", 3);
146     BOOST_REQUIRE_EQUAL(rw.generate("AAB", 100).size(), 100);
147 }
148
149 BOOST_AUTO_TEST_CASE(generateBadInput) {
150     RandWriter rw("hello all!", 2);
151     BOOST_REQUIRE_THROW(rw.generate("h", 40), std::logic_error);
152 }
```

9 PS7: Kronos Log Parsing

9.1 Functionality

PS7, the Kronos Log Parsing program, is a program which allows the user to supply a `.log` file containing information regarding a **Kronos time clock** and the status of its boot-up sequence. The program will look for key messages regarding the beginning, end, or inappropriate termination of a boot-up sequence, and create a report for the user to view listing the successes/failures in a file which appends the `.rpt` file extension to the file provided (note that this does not overwrite the `.log` extension, it adds to it). Below is a list of the important strings/formats that the program searches for when parsing the `.log` files

- `(log.c.166) server started`
 - A string containing this message denotes that the device is **beginning a boot-up sequence**
- `oejs.AbstractConnector:Started SelectChannelConnector@0.0.0.0:9080`
 - A string containing this message signifies that a boot-up sequence **completed successfully**
- `YYYY-MM-DD HH:MM:SS:`
 - This is the format for date and time, which preceeds the boot-up messages mentioned above. This can be used to determine the time it took to start the device

Now that it is understood what we are looking for when parsing, it should be understood what we would like to output as well. Below is another list explaining the lines of output, when they are used, and what they mean. Text of this format is output to the `.rpt` file and is written by the program. It is not read by the program like the previous formats.

- `=== Device boot ===`
 - This is the simple header which clearly separates a new boot-up sequence from the previous one. This is always output to the file when the program is about to begin reporting a new boot-up sequence, regardless of how the previous one terminated.
- `LINENUMBER(FILENAME.LOG): YYYY-MM-DD HH:MM:SS Boot Start`
 - Beginning with the line number that the startup message was found on, this is the message used in the output file to denote the start of a boot-up sequence. This always immediately follows the header "`=== Device boot ===`", and contains the name of the current `.log` file in parentheses, followed by a colon. After the colon, there is a space, the date and time, and then the message `Boot Start` to let the user know that a boot-up sequence is beginning.
- `LINENUMBER(FILENAME.LOG): YYYY-MM-DD HH:MM:SS Boot Completed`
 - The message for reporting a successful boot-up is almost identical to the previous message for reporting a boot-up beginning, apart from the message at the end reading `Boot Completed` instead.
- `Boot Time: Xms`
 - In the event that a boot-up sequence is completed successfully, the previous message is output, then the very next line prints this message indented. This message calculates the time that the boot-up took to complete by using the date and time stamps from the start/end messages and calculating the difference. This is printed in place of the `X` in the placeholder shown, and should be in *milliseconds* labelled with `ms`.

- **** Incomplete boot ****

- If a boot-up sequence is to fail for any reason, line immediately after the "Boot Start" message should contain "**** Incomplete boot ****", which concludes the report for the current boot-up sequence. The program will either begin another entry if a boot-up has already started, or wait for the next to start another entry.

With all of the messages that are significant for our task explained, the next logical inquiry as a reader may be: How do we determine if a boot-up has failed? A boot-up sequence will fail under two different conditions: The first is if it is interrupted by another boot-up sequence. If the program finds another boot-up message before the current boot-up sequence has ended, it will report an incomplete boot and start another separate report. The second way that a boot-up may fail is if the `.log` file ends before a success message is seen.

It should be mentioned at this time that for PS7, we were given the freedom to implement this process however we desired. It was up to us to determine what we needed to implement, the only requirements were that we used the **Boost Date-Time library** to track the date/time stamps seen in the `.log` files, and that we utilized the `<regex>` library which first began with the Boost library but was eventually implemented into the standard C++ library with C++11. Regular expressions, `regex`, are a great, reliable way to test formatting without the restriction of needing to have exact character match (as would be the issue with comparing two strings with `==`).

9.2 Features

My implementation of the Kronos Log Parsing program begins by assuring that the user provided a file to parse in the command-line (this is assumed to be a valid `.log` file). This filename is passed as an argument to instantiate an object of my class called `KronosBoot` within the namespace `KB`. The constructor simply sets a couple internal boolean flags, sets the current line number to 1, and stores the filename for usage when outputting. After this, within `main()`, a `std::ifstream` object named `input` is created with the provided `.log` filename as an argument. Afterwards, a `std::ofstream` object is created, named `output`. This takes the `.log` file with the added `.rpt` extension as an argument. My class provides a function `KB::KronosBoot::logToRpt()` to make this easy. After `input` and `output` are ready, references to them are passed as arguments to the function `KB::KronosBoot::scanLog()`.

`scanLog()` governs the entire process of reading the `.log` file and writing to the `.rpt` file. It begins by using a `boost::gregorian::date_facet` to inform `output` how we would like to print out dates that we read using the Boost Date-Time library. Once that is configured, the main loop begins by using `std::getline()` to read the first line from the file into a private string variable `_text`. The loop uses the `std::ifstream& input` as its stopping condition, so that it will keep reading lines until the end of the file. Between iterations of the loop, however, it first checks if `_text` is of the format of the boot-up message (`log.c.166`) `server started`. A function `isBeginningBootUp()` takes responsibility for this, using its inlined definition to check if `_text` matches the format of the private member `std::regex` named `_bootUp`.

```

1  const std::regex _bootUp
2      {"(\\d{4})-(\\d{2})-(\\d{2})\\s(\\d{2}:){3}"
3       "\\.(\\d{3})*\\s(\\(log\\.c\\.166\\))"};
4  const std::regex _bootFinish
5      {"oejs\\.AbstractConnector:Started SelectChannelConnector@0\\.\\.){3}0:9080"};
6  const std::regex _timeRegex{"(\\d{2}:){2}\\d{2}(:)*"};
7  const std::regex _dateRegex{"(\\d{4})-(\\d{2})-(\\d{2})"};

```

Above is the formats of all the regular expressions used during this program. It is worth breaking down precisely what each one is looking for now before we return to the flow of the program. Note that this is a breakdown of the actual regex anatomy itself, the formats were discussed in the previous section, section 9.1, if you want to look back at how some of these would appear in the `.log` file itself.

- `_bootUp` is used for detecting a boot-up beginning message
 - **date:** 4 digit year, '-' hyphen, 2 digit month, '-' hyphen, 2 digit day
 - whitespace
 - **time:** three sets of: (two digits, ':' colon)
 - *optionally* the time is followed by a period '.' and three more digits denoting milliseconds
 - whitespace
 - the exact string `(log.c.166)`
- `_bootFinish` is used for detecting a boot-up successfully finished message
 - the exact string `oejs.AbstractConnector:Started SelectChannelConnector@`
 - three sets of: (one '0' followed by one '.')
 - the exact string `0:9080`
- `_timeRegex` is used to determine if there is correctly formatted time, it is derived from the first regex
 - two sets of: (two digits, one ':' colon)
 - another two digits
 - *optionally* one colon on the end (in the event of trailing milliseconds)
- `_dateRegex` is used to assert proper date format, it is derived from the first regex
 - four sets of: (one digit) followed by one '-' hyphen
 - two sets of: (one digit) followed by one '-' hyphen
 - two sets of: (one digit)

The above lists expand the regexes fully such that you can see my intentions with the symbols, or so that you may interpret it yourself if you happen to be reading this and are unfamiliar. Returning back to `KB::KronosBoot::isBeginningBootUp()`, it uses `std::regex_search()` to see if the current line of text has a substring which matches the appropriate format, in which case we return `true`. Otherwise we naturally return `false`. This leads us back to `KB::KronosBoot::scanLog()`. If the device *is* beginning a new boot-up sequence, we enter `KB::KronosBoot::newBoot()`.

`newBoot()` first checks if the device was in the middle of boot-up already, but starting another boot-up sequence. If this is the case, we label it an "**** Incomplete boot ****" and continue. If not, the device was not booting and we just continue onward and create a new report. The date and time at the start of the line are recorded in `_startDate` and `_bootStart` respectively. `_startDate` is of type `boost::gregorian::date`. It's value is yielded by the function `KB::KronosBoot::bootDate()` which uses the regex `_dateRegex` and calls `std::regex_search()` with the extra parameter `match` which is of type `std::smatch`. The data type `std::smatch` automatically separates the substring that satisfies the regex, as well as all of the separate portions of the regex which are denoted by the `()` pairings in the regex. We can make a new `date` variable by calling `from_simple_string()` (from the boost date library) and passing in `match.str(0)` to convert from the substring to a date. Similarly, `_bootStart` gets its value from the function `KB::KronosBoot::bootTime()` which also uses the `smatch` data type and `std::regex_search()` to extract the substring, which it converts to a `time_duration` type by using `duration_from_string(match.str(0))` (from the boost time library). Wow, thats a mouthful.

With all the boring stuff discussed we can move on to greater things. Back in `newBoot()` again we can print the device boot header, and then output the boot start line by passing the current line number, the `.log` filename in parentheses, the start date, the start time, followed by "Boot Start". We then set the internal `_isBooting` flag to true.

Back in `scanLog()` now, if the device is not starting a boot-up sequence, we check if the current line expresses that we just finished a boot-up *and* the device truly was actually booting up (using the `_isBooting` bool flag). The function `KB::KronosBoot::completedBootUp()` uses `std::regex_search()` similarly to `isBeginningBootUp()` but uses the private member `regex _bootFinish` for the comparison. The function will return true if the current line contains the boot-up success message, and false otherwise. If the device did successfully complete a boot-up sequence, we call `KB::KronosBoot::finishBoot()` to output the message denoting completion and the time elapsed. In `finishBoot()`, the `_isBooting` flag gets set to false, and the end date and time are recorded the same way the start date and time was. The boot-up completed message follows the same format of the boot-up starting message except we follow it with the elapsed time, which we obtain from `KB::KronosBoot::elapsed().total_milliseconds()`.

`elapsed()` makes two variables of type `boost::posix_time::ptime` and stores the start date/time into one, and the end date/time in the other. We can then simply subtract the start `ptime` from the end `ptime` and let it calculate the difference for us. We return a `ptime` with the resulting value which we can use `.total_milliseconds()` to view the millisecond representation of. We can then output this at the end of our boot-up report, concluding `finishBoot()` and bringing us back to `scanLog()`

Regardless of if the line was significant or not, the main loop within `scanLog()` ends each iteration by incrementing the line number, getting a new line, and flushing the output. When the loop reaches the end of the file, one last `if` statement checks if the device was in the middle of a boot-up sequence, and outputs "**** Incomplete boot ****" if so. At this point the `std::ifstream` and `std::ofstream` objects are both closed and the program terminates, leaving the resulting report file in the same directory as the executable.

9.3 Personal Reflection

9.3.1 The Struggles

There were two large issues with my implementation at first which I have since fixed. The worse of the two was that when I would create a submission, it would time out with the autograder everytime. To time the autograder out you have to stall it for ten minutes, so this was an impressive feat. I ended up determining that the reason why this was happening was that I was recreating each regex object every single time I wanted to use it. Constructing a regex object is not cheap, especially when done every single comparison. By making the regex constant and storing them for the lifetime of the program, the program instantly got about 30x-40x faster, I am completely serious, I did the rough math.

The second problem with my approach was that, well, initially I had not used a class. For whatever reason my instinct was that a class was unnecessary for this, and I would just use a namespace instead. This proved to be a nuisance over time, and my tipping point was when I had to make the regexes permanent. I realized everything would be so much neater and intuitive if I just made a class, so I invested about an hour to refactor everything and organize the process within a class setting, and I am extremely thankful that I did. Not only was the `main()` function so much neater, but *everything* was incredibly more organized and intuitive. As I reread the code now, some of the names are still a little odd from the move, like private members `_bootStart` and `_bootEnd` should probably be named `_startTime` and `_endTime` respectively to make the units clear from the name.

9.3.2 What I Learned

I certainly learned that leveraging the strength of classes within C++ is not something to take for granted. The process would have been so much easier and more organized had I just began with a class in the first place. I also learned how to work with regexes, and what actions most symbols denote within them. I learned how useful and broad the Boost library is outside of just unit testing, such as the date and time types which made manipulating date and time values actually doable.

9.3.3 The Takeaway

I thought this was a pretty fun and simple way to finish off the year. I found it especially interesting because my place of work actually has a Kronos punch clock, so it was neat to think about applying programming to something real that I've encountered in my regular life. I also found it really great and fun to have the freedom to implement the program how I wanted. That being said, it also provided a good learning experience for me in regards to how important it is to leverage the abilities of C++ classes, and to think more carefully about in what form a program should be implemented in.

9.4 Running The Program

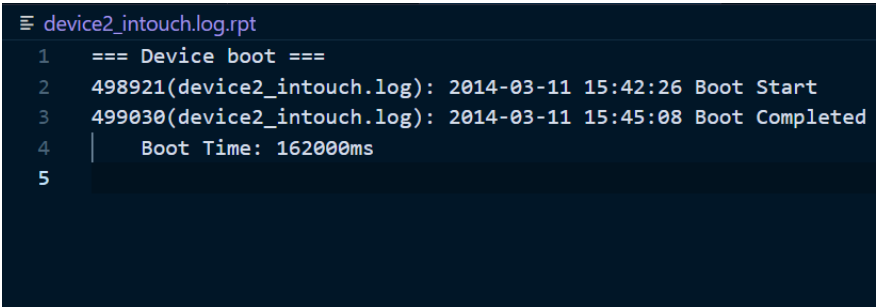
The executable **ps7** can be obtained by using *make* in correlation with the provided Makefile, and can be run using a command of the following format in the terminal.

```
1 ./ps7 <FILENAME-OF-LOG>
```

The only argument required is the name of the `.log` file you would like to parse, with the appropriate `.log` extension. The behavior of the program is undefined for file types other than `.log` files.

9.5 Proof of Functionality

Although it is a bit challenging to display output for this type of program, the image provided shows the resulting report after running the program on a 55MB `.log` file.



```
device2_intouch.log.rpt
1  === Device boot ===
2  498921(device2_intouch.log): 2014-03-11 15:42:26 Boot Start
3  499030(device2_intouch.log): 2014-03-11 15:45:08 Boot Completed
4  |      Boot Time: 162000ms
5
```

Figure 16: Sample output when parsing the `device2_intouch.log` file. There is only one boot-up sequence and it completed successfully

9.6 Source Code

My final program contained a Makefile like always, a `main.cpp` file containing the `main()` function, and supporting `KronosBoot.hpp` and `KronosBoot.cpp` files which contain the `KronosBoot` class within the `KB` namespace. Below is the source code for all of the mentioned files:

Makefile

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -lboost_unit_test_framework
4 # Your .hpp files
5 DEPS = KronosBoot.hpp
6 # Your compiled .o files
7 OBJECTS = KronosBoot.o
8 # The name of your programs
9 PROGRAM = ps7
10
11 .PHONY: all clean lint
12
13 all: $(PROGRAM)
14
15 # Wildcard recipe to make .o files from corresponding .cpp file
16 %.o: %.cpp $(DEPS)
17     $(CC) $(CFLAGS) -c $<
```

```

18
19 ps7: main.o $(OBJECTS)
20     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
21
22 clean:
23     rm *.o ps7
24
25 lint:
26     cpplint *.cpp *.hpp

```

main.cpp

```

1 // Copyright 2024 Nathan Khoury
2 #include <iostream>
3 #include <locale>
4 #include <fstream>
5
6 #include "KronosBoot.hpp"
7
8
9 int main(int argc, char* argv[]) {
10     // validate command line arguments
11     if (argc < 2) {
12         throw std::invalid_argument(".log file to parse is not supplied in the command line");
13     }
14     // create KronosBoot object
15     KB::KronosBoot kb(argv[1]);
16     // open .log file
17     std::ifstream input(argv[1]);
18     // open .rpt file
19     std::ofstream output(kb.logToRpt(argv[1]));
20
21     // main driver, scans input log and creates output file
22     kb.scanLog(input, output);
23
24     // close .log file
25     input.close();
26     // close .rpt file
27     output.close();
28     return 0;
29 }

```

KronosBoot.hpp

```

1 // Copyright 2024 Nathan Khoury
2 #ifndef KRONOS_BOOT_HPP
3 #define KRONOS_BOOT_HPP
4
5 #include <iostream>
6 #include <fstream>
7 #include <string>
8 #include <exception>
9 #include <regex>
10
11 #include "boost/date_time/gregorian/gregorian.hpp"
12 #include "boost/date_time/posix_time/posix_time.hpp"
13
14 using boost::gregorian::date_facet;
15
16 using boost::gregorian::date;
17 using boost::gregorian::from_simple_string;
18 using boost::gregorian::date_period;
19 using boost::gregorian::date_duration;
20
21 using boost::posix_time::ptime;
22 using boost::posix_time::time_duration;
23 using boost::posix_time::duration_from_string;
24 using boost::posix_time::time_from_string;
25 using boost::posix_time::to_simple_string;

```

```

26
27 namespace KB {
28 class KronosBoot {
29 public:
30     // value constructor requiring the name of the input file
31     explicit KronosBoot(std::string inputFileName) :
32         _isBooting(false), _firstBoot(true), _line(1), _logName(inputFileName) {}
33     // scanLog begins and manages the whole process of reading from .log and writing to .rpt
34     void scanLog(std::ifstream& input, std::ofstream& output);
35     // allows user to automatically add the .rpt extention
36     std::string logToRpt(std::string str) { return str + ".rpt"; }
37
38 private:
39     // check if a new bootup is starting
40     bool isBeginningBootUp() { return std::regex_search(_text, _bootUp); }
41     // check if a bootup finished successfully
42     bool completedBootUp() { return std::regex_search(_text, _bootFinish); }
43     // process for starting a new boot
44     void newBoot(std::ifstream& input, std::ofstream& output);
45     // process for successfully completeing a new boot
46     void finishBoot(std::ifstream& input, std::ofstream& output);
47     // read the timestamp on the current line
48     time_duration bootTime();
49     // read the date on the current line
50     date bootDate();
51     // calculate the elapsed time
52     time_duration elapsed(time_duration startT, time_duration endT, date startD, date endD);
53
54     // MEMBER VARIABLES
55     // conditional flags
56     bool _isBooting;
57     bool _firstBoot;
58     // Date/Time
59     date _startDate; // boot-up date
60     date _endDate; // success message date
61     time_duration _bootStart; // boot-up time
62     time_duration _bootEnd; // success message time
63     time_duration _elapsed; // time taken for whole boot up
64     // misc
65     size_t _line; // line # of current message
66     std::string _logName;
67     std::string _text; // current line of text
68     // regexs
69     const std::regex _bootUp
70         {"(\\d{4})-(\\d{2})-(\\d{2})\\s(\\d{2}:){3}"
71         "(\\.\\.\\d{3})*\\s(\\(log\\.c\\.166\\))"};
72     const std::regex _bootFinish
73         {"oejs\\.AbstractConnector:Started SelectChannelConnector@0\\.\\.\\.\\{3\\}0:9080"};
74     const std::regex _timeRegex{"(\\d{2}:){2}\\d{2}(\\:)*"};
75     const std::regex _dateRegex{"(\\d{4})-(\\d{2})-(\\d{2})"};
76 };
77 }; // namespace KB
78
79 #endif

```

KronosBoot.cpp

```

1 // Copyright 2024 Nathan Khoury
2 #include "KronosBoot.hpp"
3
4 void KB::KronosBoot::scanLog(std::ifstream& input, std::ofstream& output) {
5     // set date format configuration
6     date_facet* facet(new date_facet("%Y-%m-%d"));
7     output.imbue(std::locale(output.getloc(), facet));
8
9     // read input log
10    std::getline(input, _text);

```

```

11     while (input) {
12         // check if booting
13         if (isBeginningBootUp()) {
14             newBoot(input, output);
15         } else if (completedBootUp() && !_isBooting) {
16             finishBoot(input, output);
17         }
18         _line++;
19         std::getline(input, _text);
20         std::flush(output);
21     }
22     if (_isBooting) {
23         output << "**** Incomplete boot **** \n";
24     }
25 }
26
27 void KB::KronosBoot::newBoot(std::ifstream& input, std::ofstream& output) {
28     // check if already booting
29     if (_isBooting) {
30         // booting fails, new booting starts
31         output << "**** Incomplete boot **** \n"; // hanging space here after **** (???)
32     } // print newline if not first boot
33     if (!_firstBoot) {
34         output << '\n';
35         _firstBoot = false;
36     }
37     // begin boot-up sequence
38     _bootStart = bootTime();
39     _startDate = bootDate();
40     output << "=== Device boot ===\n"
41         << _line << '(' << _logName << "): "
42         << _startDate << ' ' << _bootStart << " Boot Start\n";
43     _isBooting = true;
44 }
45
46 void KB::KronosBoot::finishBoot(std::ifstream& input, std::ofstream& output) {
47     // boot-up completed successfully
48     _isBooting = false;
49     _bootEnd = bootTime();
50     _endDate = bootDate();
51     output << _line << '(' << _logName << "): "
52         << _endDate << ' ' << _bootEnd << " Boot Completed\n"
53         << "\tBoot Time: "
54         << elapsed(_bootStart, _bootEnd, _startDate, _endDate).total_milliseconds()
55         << "ms\n";
56 }
57
58 time_duration KB::KronosBoot::bootTime() {
59     std::smatch match;
60     if (std::regex_search(_text, match, _timeRegex)) {
61         time_duration time(duration_from_string(match.str(0)));
62         return time;
63     }
64     throw std::invalid_argument("provided string did not contain correctly formatted time");
65 }
66
67 date KB::KronosBoot::bootDate() {
68     std::smatch match;
69     if (std::regex_search(_text, match, _dateRegex)) {
70         date d(from_simple_string(match.str(0)));
71         return d;
72     }
73     throw std::invalid_argument("provided string did not contain correctly formatted date");
74 }
75
76 time_duration KB::KronosBoot::elapsed

```

```
77     (time_duration startT, time_duration endT, date startD, date endD) {  
78         ptime t1(startD, startT);  
79         ptime t2(endD, endT);  
80         return t2 - t1;  
81     }
```

9.7 Unit Testing

Unit testing was not required for this project.